

P0.1 Import the libraries

```
import hashlib

import numpy as np
import pandas as pd

pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 160)

print("pandas version:", pd.__version__)
print("numpy version:", np.__version__)
```

P0.2 Load the raw data

```
RAW_CSV = "25740123.csv"

df_raw = pd.read_csv(RAW_CSV)

print("Dataset shape:", df_raw.shape)
print("Number of rows:", df_raw.shape[0])
print("Number of columns:", df_raw.shape[1])
print("Ordered column names:")
for position, column_name in enumerate(df_raw.columns, start=1):
    print(position, column_name)
df_raw.head()
```

P0.3 Attribute audit table

```
total_rows = len(df_raw)
audit_rows = []

for column_name in df_raw.columns:
    column = df_raw[column_name]
    missing_count = int(column.isna().sum())
    audit_rows.append({
        "Attribute": column_name,
        "Pandas dtype": str(column.dtype),
        "Non-missing count": int(column.notna().sum()),
        "Missing count": missing_count,
        "Missing percentage": round(100 * missing_count / total_rows, 2),
        "Unique values": int(column.nunique(dropna=True)),
    })

audit_table = pd.DataFrame(audit_rows)
audit_table
audit_table.to_csv("outputs/P0_data_audit.csv", index=False)
print("Saved outputs/P0_data_audit.csv with", len(audit_table), "attribute rows.")
```

P0.4 Duplicate rows

```
duplicate_row_count = int(df_raw.duplicated().sum())
print("Exact duplicate rows:", duplicate_row_count)
```

P0.5 Missing values and unique values for every attribute

```
print("Missing values per attribute:")
print(df_raw.isna().sum())
print("Unique values per attribute:")
print(df_raw.nunique(dropna=True))
```

P0.6 Cancelled and diverted flights

```
print("CANCELLED unique values:", sorted(df_raw["CANCELLED"].unique()))
print("DIVERTED unique values:", sorted(df_raw["DIVERTED"].unique()))

cancelled_count = int((df_raw["CANCELLED"] == 1).sum())
diverted_count = int((df_raw["DIVERTED"] == 1).sum())

print("Rows where CANCELLED == 1:", cancelled_count)
print("Rows where DIVERTED == 1:", diverted_count)
```

P0.7 Airlines

```
airline_names = sorted(df_raw["AIRLINE_NAME"].dropna().unique())
number_of_airlines = int(df_raw["AIRLINE_NAME"].nunique(dropna=True))

print("Number of airlines:", number_of_airlines)
for airline in airline_names:
    print("-", airline)
```

P0.8 Date range

```
print("Original DATE dtype:", df_raw["DATE"].dtype)

date_parsed = pd.to_datetime(df_raw["DATE"], format="%Y-%m-%d")

print("Parsed minimum date:", date_parsed.min().date())
print("Parsed maximum date:", date_parsed.max().date())
print("Dates that failed to parse:", int(date_parsed.isna().sum()))

print("DATE dtype in df_raw after parsing:", df_raw["DATE"].dtype)
print("First DATE value in df_raw:", repr(df_raw["DATE"].iloc[0]))
```

P0.9 ARRIVAL_DELAY audit

```
arrival_delay = df_raw["ARRIVAL_DELAY"]

arrival_delay_non_missing = int(arrival_delay.notna().sum())
```

```

arrival_delay_min = float(arrival_delay.min())
arrival_delay_max = float(arrival_delay.max())
arrival_delay_negative = int((arrival_delay < 0).sum())

print("Non-missing ARRIVAL_DELAY values:", arrival_delay_non_missing)
print("Missing ARRIVAL_DELAY values:", int(arrival_delay.isna().sum()))
print("Minimum ARRIVAL_DELAY:", arrival_delay_min, "minutes")
print("Maximum ARRIVAL_DELAY:", arrival_delay_max, "minutes")
print("Non-missing values below zero (early arrivals):", arrival_delay_negative)

```

P0.10 Why the values are missing

```

is_cancelled = df_raw["CANCELLED"] == 1
is_diverted = df_raw["DIVERTED"] == 1
cancelled_or_diverted = is_cancelled | is_diverted

arrival_delay_missing = df_raw["ARRIVAL_DELAY"].isna()
elapsed_time_missing = df_raw["ELAPSED_TIME"].isna()

print("Cancelled flights:", int(is_cancelled.sum()))
print("Diverted flights:", int(is_diverted.sum()))
print("Cancelled or diverted flights:", int(cancelled_or_diverted.sum()))
print("Rows with missing ARRIVAL_DELAY:", int(arrival_delay_missing.sum()))
print("Rows with missing ELAPSED_TIME:", int(elapsed_time_missing.sum()))
arrival_rows_match = bool(arrival_delay_missing.equals(cancelled_or_diverted))
elapsed_rows_match = bool(elapsed_time_missing.equals(cancelled_or_diverted))

flagged_index = set(df_raw.index[cancelled_or_diverted])
arrival_index_match = set(df_raw.index[arrival_delay_missing]) == flagged_index
elapsed_index_match = set(df_raw.index[elapsed_time_missing]) == flagged_index

print("Missing ARRIVAL_DELAY rows equal cancelled or diverted rows:", arrival_rows_match)
print("Missing ELAPSED_TIME rows equal cancelled or diverted rows:", elapsed_rows_match)
print("Same result using row index sets, ARRIVAL_DELAY:", arrival_index_match)
print("Same result using row index sets, ELAPSED_TIME:", elapsed_index_match)

```

P0.11 The three delay-cause attributes

```

air_system_missing = df_raw["AIR_SYSTEM_DELAY"].isna()
security_missing = df_raw["SECURITY_DELAY"].isna()
weather_missing = df_raw["WEATHER_DELAY"].isna()

same_as_security = bool(air_system_missing.equals(security_missing))
same_as_weather = bool(air_system_missing.equals(weather_missing))

print("AIR_SYSTEM_DELAY missing on the same rows as SECURITY_DELAY:", same_as_security)
print("AIR_SYSTEM_DELAY missing on the same rows as WEATHER_DELAY:", same_as_weather)
print("Rows where the delay causes are recorded:", int((~air_system_missing).sum()))
delay_15_or_more = df_raw["ARRIVAL_DELAY"] >= 15
delay_causes_recorded = ~air_system_missing

delay_cause_rows_match = bool(delay_causes_recorded.equals(delay_15_or_more))

print("Rows where ARRIVAL_DELAY is 15 minutes or more:", int(delay_15_or_more.sum()))
print("Recorded delay-cause rows equal those rows:", delay_cause_rows_match)

```

P0.12 Checks for the audit

```

expected_missing_counts = {
    "DEPARTURE_TIME": 61,
    "DEPARTURE_DELAY": 61,
    "TAXI_OUT": 65,
    "WHEELS_OFF": 65,
    "WHEELS_ON": 67,
    "TAXI_IN": 67,
    "ARRIVAL_TIME": 67,
    "ELAPSED_TIME": 84,
    "AIR_TIME": 84,
    "ARRIVAL_DELAY": 84,
    "AIR_SYSTEM_DELAY": 3522,
    "SECURITY_DELAY": 3522,
    "WEATHER_DELAY": 3522,
}

validation_rows = []

def record_check(check_name, expected_value, observed_value):
    validation_rows.append({
        "Check": check_name,
        "Expected": str(expected_value),
        "Observed": str(observed_value),
        "Match": str(expected_value) == str(observed_value),
    })

record_check("Dataset rows", 4349, df_raw.shape[0])
record_check("Dataset columns", 25, df_raw.shape[1])
record_check("Date minimum", "2015-01-01", date_parsed.min().date())
record_check("Date maximum", "2015-12-31", date_parsed.max().date())
record_check("Number of airlines", 6, number_of_airlines)

for column_name, expected_count in expected_missing_counts.items():
    record_check("Missing values in " + column_name,
                 expected_count,
                 int(df_raw[column_name].isna().sum()))

```

```

other_columns = [c for c in df_raw.columns if c not in expected_missing_counts]
record_check("Attributes expected to have no missing values",
0,
int(df_raw[other_columns].isna().sum().sum()))
record_check("Cancelled flights", 66, cancelled_count)
record_check("Diverted flights", 18, diverted_count)
record_check("Cancelled or diverted rows", 84, int(cancelled_or_diverted.sum()))
record_check("Missing ARRIVAL_DELAY count", 84, int(arrival_delay_missing.sum()))
record_check("ARRIVAL_DELAY non-missing count", 4265, arrival_delay_non_missing)
record_check("ARRIVAL_DELAY minimum", -61.0, arrival_delay_min)
record_check("ARRIVAL_DELAY maximum", 865.0, arrival_delay_max)
record_check("Negative ARRIVAL_DELAY count", 2633, arrival_delay_negative)
record_check("Missing ARRIVAL_DELAY row set equals cancelled or diverted", True, arrival_rows_match)
record_check("Missing ELAPSED_TIME row set equals cancelled or diverted", True, elapsed_rows_match)
record_check("Delay causes missing on identical rows", True, same_as_security and same_as_weather)
record_check("Delay-cause rows equal ARRIVAL_DELAY of 15 or more", True, delay_cause_rows_match)
record_check("Rows with ARRIVAL_DELAY of 15 or more", 827, int(delay_15_or_more.sum()))
record_check("Exact duplicate rows", 0, duplicate_row_count)

```

```

validation_table = pd.DataFrame(validation_rows)
validation_table
failed_checks = validation_table[validation_table["Match"] == False]

```

```

print("Total checks:", len(validation_table))
print("Checks passed:", int((validation_table["Match"] == True).sum()))
print("Checks failed:", len(failed_checks))

```

```

if len(failed_checks) == 0:
    print("All expected dataset facts were confirmed from the raw file.")
else:
    print("The following checks did not match the expected values:")
    print(failed_checks)

```

P0.13 Raw file integrity

```
RECORDED_CSV_HASH = "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de"
```

```

with open(RAW_CSV, "rb") as raw_file:
    current_csv_hash = hashlib.sha256(raw_file.read()).hexdigest()

```

```

print("Recorded SHA-256:", RECORDED_CSV_HASH)
print("Current SHA-256: ", current_csv_hash)
print("Raw file unchanged:", current_csv_hash == RECORDED_CSV_HASH)
print("Rows still loaded from the raw file:", len(df_raw))

```

A1.1 Nominal attributes

```
a1_rows = []
```

```

a1_rows.append(["O_CITY", "Nominal",
    "The origin city is a distinct place label, so only equality comparisons apply and no order or numerical
↳ distance exists."])
a1_rows.append(["O_STATE", "Nominal",
    "State names distinguish one origin region from another but carry no measurement order, so only equality and
↳ inequality are meaningful."])
a1_rows.append(["D_CITY", "Nominal",
    "The destination city is a distinct place label with no inherent ranking, and differences between city names
↳ cannot be measured."])
a1_rows.append(["D_STATE", "Nominal",
    "Destination states are distinct regional categories, and no state sits at a measurable interval above or
↳ below another."])
a1_rows.append(["DIVERTED", "Nominal",
    "The codes 0 and 1 label two flight outcomes rather than measuring an amount, so only distinctness is
↳ meaningful."])
a1_rows.append(["CANCELLED", "Nominal",
    "The value 1 marks a cancelled flight and 0 a flight that operated, so the digits act purely as category
↳ labels."])
a1_rows.append(["AIRLINE_NAME", "Nominal",
    "Each airline name identifies a distinct carrier, and airlines cannot be placed in a measurement order or
↳ spaced by intervals."])

```

```
print("Nominal attributes added:", len(a1_rows))
```

A1.2 Interval attributes

```

a1_rows.append(["SCHEDULED_DEPARTURE", "Interval",
    "This is a clock point, so times can be ordered and gaps between them compared, but midnight is a
↳ conventional origin."])
a1_rows.append(["DEPARTURE_TIME", "Interval",
    "The actual departure clock time can be ordered and differences between two times are meaningful, yet the
↳ zero point is conventional."])
a1_rows.append(["DEPARTURE_DELAY", "Interval",
    "Delay is a signed offset in minutes from the schedule, where zero means on time rather than an absence of
↳ quantity."])
a1_rows.append(["WHEELS_OFF", "Interval",
    "The moment the wheels leave the ground is a point on the clock, so differences are meaningful but ratios are
↳ not."])
a1_rows.append(["WHEELS_ON", "Interval",
    "Touchdown is recorded as a clock point, so intervals between events are meaningful while a doubled clock
↳ value has no interpretation."])
a1_rows.append(["SCHEDULED_ARRIVAL", "Interval",
    "The planned arrival clock time is ordered with meaningful gaps between times, but it has no true zero for
↳ arrival."])
a1_rows.append(["ARRIVAL_TIME", "Interval",

```

```

    "The actual arrival clock time supports ordering and meaningful differences, although the origin of the clock
↳ is set by convention.")])
a1_rows.append(["ARRIVAL_DELAY", "Interval",
    "Arrival delay is a signed deviation from schedule and includes negative values for early arrivals, so ratio
↳ comparisons are not meaningful.")])
a1_rows.append(["DATE", "Interval",
    "Calendar dates can be ordered and differences in days are meaningful, but the start of the calendar is a
↳ convention."])

print("Attributes added so far:", len(a1_rows))

```

A1.3 Ratio attributes

```

a1_rows.append(["TAXI_OUT", "Ratio",
    "This is a duration in minutes where zero means no time elapsed taxiing, so differences and ratios are both
↳ meaningful."])
a1_rows.append(["SCHEDULED_TIME", "Ratio",
    "The planned trip length is a duration with a true zero, so one flight can be meaningfully twice as long as
↳ another."])
a1_rows.append(["ELAPSED_TIME", "Ratio",
    "Total elapsed time measures an amount of time with a genuine zero, which allows meaningful differences and
↳ ratio comparisons."])
a1_rows.append(["AIR_TIME", "Ratio",
    "Time airborne is a measured duration where zero represents no time in the air, so ratio statements are
↳ meaningful."])
a1_rows.append(["DISTANCE", "Ratio",
    "Distance has a true zero at no separation and equal units, so one route can be meaningfully twice as far as
↳ another."])
a1_rows.append(["TAXI_IN", "Ratio",
    "The taxi-in duration counts minutes from a genuine zero, so both differences and ratio comparisons between
↳ flights are meaningful."])
a1_rows.append(["AIR_SYSTEM_DELAY", "Ratio",
    "When recorded, this is an amount of delay caused by the air system, and zero minutes is a meaningful absence
↳ of it."])
a1_rows.append(["SECURITY_DELAY", "Ratio",
    "When recorded, this measures minutes of delay attributed to security, with zero representing no such delay
↳ contribution."])
a1_rows.append(["WEATHER_DELAY", "Ratio",
    "When recorded, this measures minutes of delay attributed to weather, and zero marks a true absence of
↳ weather delay."])

print("Total attributes classified:", len(a1_rows))

```

A1.4 The complete A1 classification table

```

a1_table = pd.DataFrame(a1_rows, columns=["Attribute", "Attribute Type", "Justification"])

# Sort the rows into the original dataset column order, taken from the data itself.
attribute_order = list(df_raw.columns)
a1_table["Dataset position"] = a1_table["Attribute"].map(attribute_order.index)
a1_table = a1_table.sort_values("Dataset position").reset_index(drop=True)
a1_table = a1_table.drop(columns="Dataset position")

print("A1 table shape:", a1_table.shape)
a1_table
for attribute, scale, justification in zip(a1_table["Attribute"],
                                          a1_table["Attribute Type"],
                                          a1_table["Justification"]):
    print(f"{attribute:<20} {scale:<9} {justification}")
print("Attributes per measurement scale:")
print(a1_table["Attribute Type"].value_counts())
print()
print("Total attributes classified:", len(a1_table))

```

A1.5 Checks for the classification table

```

approved_nominal = ["O_CITY", "O_STATE", "D_CITY", "D_STATE",
    "DIVERTED", "CANCELLED", "AIRLINE_NAME"]
approved_interval = ["SCHEDULED_DEPARTURE", "DEPARTURE_TIME", "DEPARTURE_DELAY",
    "WHEELS_OFF", "WHEELS_ON", "SCHEDULED_ARRIVAL",
    "ARRIVAL_TIME", "ARRIVAL_DELAY", "DATE"]
approved_ratio = ["TAXI_OUT", "SCHEDULED_TIME", "ELAPSED_TIME", "AIR_TIME",
    "DISTANCE", "TAXI_IN", "AIR_SYSTEM_DELAY",
    "SECURITY_DELAY", "WEATHER_DELAY"]

approved_types = {}
for attribute in approved_nominal:
    approved_types[attribute] = "Nominal"
for attribute in approved_interval:
    approved_types[attribute] = "Interval"
for attribute in approved_ratio:
    approved_types[attribute] = "Ratio"

print("Approved classifications declared:", len(approved_types))
allowed_scales = {"Nominal", "Ordinal", "Interval", "Ratio"}
dataset_attributes = list(df_raw.columns)
table_attributes = list(a1_table["Attribute"])
table_types = list(a1_table["Attribute Type"])
justifications = list(a1_table["Justification"])

a1_checks = {
    "1. A1 row count is 25": len(a1_table) == 25,
    "2. A1 attribute order equals df_raw.columns": table_attributes == dataset_attributes,
    "3. Unique A1 attributes is 25": a1_table["Attribute"].nunique() == 25,
}

```

```

"4. No dataset attribute is missing": set(dataset_attributes) - set(table_attributes) == set(),
"5. No extra attribute is present": set(table_attributes) - set(dataset_attributes) == set(),
"6. Only the four measurement scales are used": set(table_types) <= allowed_scales,
"7. No missing Attribute Type": a1_table["Attribute Type"].notna().all(),
"8. No missing Justification": a1_table["Justification"].notna().all(),
"9. Every justification is non-empty text": all(isinstance(text, str) and text.strip() != "" for text in
justifications),
"10. Types match the approved classification": all(approved_types[name] == scale for name, scale in
zip(table_attributes, table_types)),
}

```

```

for description, passed in a1_checks.items():
    print("PASS" if passed else "FAIL", description)

```

```

print()
print("Checks passed:", sum(a1_checks.values()), "of", len(a1_checks))
a1_table.to_csv("outputs/A1_attribute_types.csv", index=False)

a1_saved = pd.read_csv("outputs/A1_attribute_types.csv")

print("Saved outputs/A1_attribute_types.csv")
print("Rows read back:", len(a1_saved))
print("Columns read back:", list(a1_saved.columns))
print("Saved file matches the table in memory:", a1_saved.equals(a1_table))

```

A2.1 Categorical attributes: locations and airlines

```
import os
```

```
import matplotlib.pyplot as plt
```

```
batch1_attributes = ["O_CITY", "O_STATE", "D_CITY", "D_STATE", "AIRLINE_NAME"]
```

```

print("Raw data shape:", df_raw.shape)
for attribute in batch1_attributes:
    present = attribute in df_raw.columns
    missing = int(df_raw[attribute].isna().sum())
    print(f"{attribute:<14} present={present} missing={missing}")

print()
print("All five attributes present:", all(a in df_raw.columns for a in batch1_attributes))
print("All five attributes complete:", all(df_raw[a].isna().sum() == 0 for a in batch1_attributes))

```

```

def frequency_table(attribute_name):
    """Return the complete frequency and percentage table for one nominal attribute."""
    counts = df_raw[attribute_name].value_counts(dropna=True)

    table = counts.reset_index()
    table.columns = ["Category", "Frequency"]

    valid_count = int(df_raw[attribute_name].notna().sum())
    table["Percentage"] = round(100 * table["Frequency"] / valid_count, 2)

    # Highest frequency first, with ties broken alphabetically so the order is repeatable.
    table = table.sort_values(["Frequency", "Category"], ascending=[False, True])
    return table.reset_index(drop=True)

def report_table(full_table, top_n=10):
    """Return the largest categories plus one Others row holding every remaining category."""
    total_count = int(full_table["Frequency"].sum())
    top_rows = full_table.head(top_n).copy()
    remaining_rows = full_table.iloc[top_n:]

```

```

    if len(remaining_rows) == 0:
        return top_rows.reset_index(drop=True)

    others_frequency = int(remaining_rows["Frequency"].sum())
    others_row = pd.DataFrame([
        "Category": "Others",
        "Frequency": others_frequency,
        "Percentage": round(100 * others_frequency / total_count, 2),
    ])
    return pd.concat([top_rows, others_row], ignore_index=True)

```

```

def describe_categorical(attribute_name, full_table):
    """Print the summary statistics appropriate for a nominal attribute."""
    column = df_raw[attribute_name]
    valid_count = int(column.notna().sum())
    top_ten_share = round(100 * full_table["Frequency"].head(10).sum() / valid_count, 2)

```

```

    print(f"Attribute: {attribute_name}")
    print(f"  Total observations n   : {len(df_raw)}")
    print(f"  Non-missing count       : {valid_count}")
    print(f"  Missing count          : {int(column.isna().sum())}")
    print(f"  Unique categories      : {int(column.nunique(dropna=True))}")
    print(f"  Mode                   : {full_table.loc[0, 'Category']}")
    print(f"  Mode frequency         : {int(full_table.loc[0, 'Frequency'])}")
    print(f"  Mode percentage        : {full_table.loc[0, 'Percentage']}%")
    print(f"  Share of top ten       : {top_ten_share}%")
    print(f"  Categories seen once   : {int((full_table['Frequency'] == 1).sum())}")

```

```

def save_bar_chart(table, attribute_name, title, category_axis_label, horizontal=True):
    """Draw one bar chart of category frequencies and save it as a PNG for the report."""
    figure, axes = plt.subplots(figsize=(9, 5.5))

```

```

if horizontal:
    plot_order = table.iloc[::-1] # reversed so the largest bar appears at the top
    bars = axes.barh(plot_order["Category"], plot_order["Frequency"])
    axes.set_xlabel("Number of flights")
    axes.set_ylabel(category_axis_label)
    axes.margins(x=0.12)
else:
    bars = axes.bar(table["Category"], table["Frequency"])
    axes.set_xlabel(category_axis_label)
    axes.set_ylabel("Number of flights")
    axes.margins(y=0.12)

axes.bar_label(bars, padding=2, fontsize=8)
axes.set_title(title)
figure.tight_layout()

figure_path = f"figures/A2_{attribute_name}_bar.png"
figure.savefig(figure_path, dpi=200)
plt.show()
print("Saved", figure_path)
o_city_full = frequency_table("O_CITY")
o_city_report = report_table(o_city_full)

o_city_full.to_csv("outputs/A2_O_CITY_frequency_full.csv", index=False)
o_city_report.to_csv("outputs/A2_O_CITY_report_table.csv", index=False)

describe_categorical("O_CITY", o_city_full)
print()
print("Categories in the full table:", len(o_city_full))
print("Rows in the report table:", len(o_city_report))

o_city_report
save_bar_chart(o_city_report, "O_CITY",
               "Origin city distribution: ten most frequent cities and all others",
               "Origin city",
               horizontal=True)
o_state_full = frequency_table("O_STATE")
o_state_report = report_table(o_state_full)

o_state_full.to_csv("outputs/A2_O_STATE_frequency_full.csv", index=False)
o_state_report.to_csv("outputs/A2_O_STATE_report_table.csv", index=False)

describe_categorical("O_STATE", o_state_full)
print()
print("Categories in the full table:", len(o_state_full))
print("Rows in the report table:", len(o_state_report))

o_state_report
save_bar_chart(o_state_report, "O_STATE",
               "Origin state distribution: ten most frequent states and all others",
               "Origin state",
               horizontal=False)
d_city_full = frequency_table("D_CITY")
d_city_report = report_table(d_city_full)

d_city_full.to_csv("outputs/A2_D_CITY_frequency_full.csv", index=False)
d_city_report.to_csv("outputs/A2_D_CITY_report_table.csv", index=False)

describe_categorical("D_CITY", d_city_full)
print()
print("Categories in the full table:", len(d_city_full))
print("Rows in the report table:", len(d_city_report))

d_city_report
save_bar_chart(d_city_report, "D_CITY",
               "Destination city distribution: ten most frequent cities and all others",
               "Destination city",
               horizontal=True)
d_state_full = frequency_table("D_STATE")
d_state_report = report_table(d_state_full)

d_state_full.to_csv("outputs/A2_D_STATE_frequency_full.csv", index=False)
d_state_report.to_csv("outputs/A2_D_STATE_report_table.csv", index=False)

describe_categorical("D_STATE", d_state_full)
print()
print("Categories in the full table:", len(d_state_full))
print("Rows in the report table:", len(d_state_report))

d_state_report
save_bar_chart(d_state_report, "D_STATE",
               "Destination state distribution: ten most frequent states and all others",
               "Destination state",
               horizontal=False)
airline_name_full = frequency_table("AIRLINE_NAME")
airline_name_report = report_table(airline_name_full)

airline_name_full.to_csv("outputs/A2_AIRLINE_NAME_frequency_full.csv", index=False)
airline_name_report.to_csv("outputs/A2_AIRLINE_NAME_report_table.csv", index=False)

```

```

describe_categorical("AIRLINE_NAME", airline_name_full)
print()
print("Categories in the full table:", len(airline_name_full))
print("Rows in the report table:", len(airline_name_report))

airline_name_report
save_bar_chart(airline_name_report, "AIRLINE_NAME",
               "Flight counts by airline, all six airlines",
               "Airline",
               horizontal=True)

```

A2.2 Summary of the nominal attributes

```

summary_rows = []
full_tables = {
    "O_CITY": o_city_full,
    "O_STATE": o_state_full,
    "D_CITY": d_city_full,
    "D_STATE": d_state_full,
    "AIRLINE_NAME": airline_name_full,
}

for attribute in batch1_attributes:
    full_table = full_tables[attribute]
    column = df_raw[attribute]
    summary_rows.append({
        "Attribute": attribute,
        "N": len(df_raw),
        "Missing": int(column.isna().sum()),
        "Unique Categories": int(column.nunique(dropna=True)),
        "Mode": full_table.loc[0, "Category"],
        "Mode Frequency": int(full_table.loc[0, "Frequency"]),
        "Mode Percentage": full_table.loc[0, "Percentage"],
    })

batch1_summary = pd.DataFrame(summary_rows)
batch1_summary.to_csv("outputs/A2_batch1_categorical_summary.csv", index=False)
batch1_summary

```

A2.3 Checks for the nominal attributes

```

def percentages_sum_to_100(table):
    """Allow for the rounding error introduced by displaying each percentage to two decimals."""
    tolerance = 0.005 * len(table) + 0.01
    return abs(table["Percentage"].sum() - 100) <= tolerance

def report_table_is_top10_plus_others(full_table, report):
    """Confirm a report table holds the top ten rows plus one correct Others row."""
    if len(report) != 11 or report.iloc[-1]["Category"] != "Others":
        return False
    top_matches = report.head(10).reset_index(drop=True).equals(full_table.head(10).reset_index(drop=True))
    expected_others = int(full_table["Frequency"].sum()) - int(full_table["Frequency"].head(10).sum())
    return top_matches and int(report.iloc[-1]["Frequency"]) == expected_others

expected_figures = [f"figures/A2_{a}_bar.png" for a in batch1_attributes]
expected_outputs = [f"outputs/A2_{a}_frequency_full.csv" for a in batch1_attributes]
expected_outputs += [f"outputs/A2_{a}_report_table.csv" for a in batch1_attributes]
expected_outputs += ["outputs/A2_batch1_categorical_summary.csv"]

report_tables = {"O_CITY": o_city_report, "O_STATE": o_state_report,
                 "D_CITY": d_city_report, "D_STATE": d_state_report,
                 "AIRLINE_NAME": airline_name_report}
print("Files expected:", len(expected_figures), "figures and", len(expected_outputs), "tables")

a2_checks = {
    "1. All five attributes exist": all(a in df_raw.columns for a in batch1_attributes),
    "2. Missing count is zero for all five": all(df_raw[a].isna().sum() == 0 for a in batch1_attributes),
    "3. Frequency totals equal 4349 for every attribute": all(int(full_tables[a]["Frequency"].sum()) == 4349 for
    a in batch1_attributes),
    "4. Percentage totals are 100 within rounding": all(percentages_sum_to_100(full_tables[a]) for a in
    batch1_attributes)
    and all(percentages_sum_to_100(report_tables[a]) for a in
    batch1_attributes),
    "5. Full table row counts equal the unique counts": all(len(full_tables[a]) == df_raw[a].nunique(dropna=True)
    for a in batch1_attributes),
    "6. O_CITY report table is top 10 plus correct Others": report_table_is_top10_plus_others(o_city_full,
    o_city_report),
    "7. O_STATE report table is top 10 plus correct Others": report_table_is_top10_plus_others(o_state_full,
    o_state_report),
    "8. D_CITY report table is top 10 plus correct Others": report_table_is_top10_plus_others(d_city_full,
    d_city_report),
    "9. D_STATE report table is top 10 plus correct Others": report_table_is_top10_plus_others(d_state_full,
    d_state_report),
    "10. AIRLINE_NAME report table holds all six airlines": len(airline_name_report) == 6 and
    df_raw["AIRLINE_NAME"].nunique() == 6,
    "11. AIRLINE_NAME has no Others row": "Others" not in list(airline_name_report["Category"]),
    "12. Summary mode equals the highest-frequency category": all(
        batch1_summary.loc[i, "Mode"] == full_tables[a]["Category"].iloc[0]
        and int(batch1_summary.loc[i, "Mode Frequency"]) == int(full_tables[a]["Frequency"].max())
        for i, a in enumerate(batch1_attributes)),
    "13. All five figure files exist": all(os.path.exists(p) for p in expected_figures),
}

```

```

    "14. All expected output CSV files exist": all(os.path.exists(p) for p in expected_outputs),
    "15. df_raw remains 4349 by 25": df_raw.shape == (4349, 25),
}

for description, passed in a2_checks.items():
    print("PASS" if passed else "FAIL", description)

print()
print("Checks passed:", sum(a2_checks.values()), "of", len(a2_checks))

A2.4 Numeric trip measures and binary flight-status attributes
batch2_attributes = ["SCHEDULED_TIME", "DISTANCE", "DIVERTED", "CANCELLED"]

print("Raw data shape:", df_raw.shape)
for attribute in batch2_attributes:
    column = df_raw[attribute]
    print(f"{attribute:<16} present={attribute in df_raw.columns} "
          f"missing={int(column.isna().sum())} dtype={column.dtype}")

print()
print("All four present:", all(a in df_raw.columns for a in batch2_attributes))
print("All four complete:", all(df_raw[a].isna().sum() == 0 for a in batch2_attributes))
print("Shape is 4349 by 25:", df_raw.shape == (4349, 25))
def numeric_summary(attribute_name):
    """Return the summary statistics that are meaningful for a quantitative ratio attribute."""
    column = df_raw[attribute_name]
    return {
        "N": int(column.notna().sum()),
        "Missing": int(column.isna().sum()),
        "Mean": column.mean(),
        "Median": column.median(),
        "Minimum": column.min(),
        "Maximum": column.max(),
        "Range": column.max() - column.min(),
        "Variance": column.var(ddof=1), # sample variance, divides by n - 1
        "Standard deviation": column.std(ddof=1), # sample standard deviation
        "Q1": column.quantile(0.25),
        "Q3": column.quantile(0.75),
        "IQR": column.quantile(0.75) - column.quantile(0.25),
        "Skewness": column.skew(), # pandas sample skewness, no extra library needed
    }

def format_value(value):
    """Show whole numbers without decimals and all other values to two decimal places."""
    if float(value).is_integer():
        return str(int(value))
    return f"{value:.2f}"

def summary_to_table(summary):
    """Turn a summary dictionary into a two-column report-ready table."""
    rows = [{"Statistic": name, "Value": format_value(value)} for name, value in summary.items()]
    return pd.DataFrame(rows)

scheduled_time_summary = numeric_summary("SCHEDULED_TIME")

print("Full precision values for SCHEDULED_TIME:")
for name, value in scheduled_time_summary.items():
    print(f" {name:<20} {value}")

scheduled_time_table = summary_to_table(scheduled_time_summary)
scheduled_time_table.to_csv("outputs/A2_SCHEDULED_TIME_summary.csv", index=False)
scheduled_time_table

def save_histogram(attribute_name, bin_edges, title, x_label):
    """Draw one histogram of an attribute and save it as a PNG for the report."""
    figure, axes = plt.subplots(figsize=(8, 5))
    axes.hist(df_raw[attribute_name], bins=bin_edges, edgecolor="white")
    axes.set_title(title)
    axes.set_xlabel(x_label)
    axes.set_ylabel("Number of flights")
    figure.tight_layout()

    figure_path = f"figures/A2_{attribute_name}_hist.png"
    figure.savefig(figure_path, dpi=200)
    plt.show()
    print("Saved", figure_path)

scheduled_time_bins = np.arange(30, 631, 30) # 30 minute wide bins covering 37 to 602

save_histogram("SCHEDULED_TIME", scheduled_time_bins,
              "Distribution of scheduled flight time",
              "Scheduled flight time (minutes)")

distance_summary = numeric_summary("DISTANCE")

print("Full precision values for DISTANCE:")
for name, value in distance_summary.items():
    print(f" {name:<20} {value}")

distance_table = summary_to_table(distance_summary)
distance_table.to_csv("outputs/A2_DISTANCE_summary.csv", index=False)
distance_table

distance_bins = np.arange(0, 5001, 250) # 250 unit wide bins covering 67 to 4983

```

```

save_histogram("DISTANCE", distance_bins,
              "Distribution of flight distance",
              "Distance")
def binary_frequency_table(attribute_name, meanings):
    """Frequency table for a binary nominal attribute, with readable display labels."""
    counts = df_raw[attribute_name].value_counts().sort_index()
    valid_count = int(df_raw[attribute_name].notna().sum())

    rows = []
    for category, frequency in counts.items():
        rows.append({
            "Category": int(category),
            "Meaning": meanings[int(category)],
            "Frequency": int(frequency),
            "Percentage": round(100 * frequency / valid_count, 2),
        })
    return pd.DataFrame(rows)
def binary_summary(attribute_name, frequency):
    """Compact nominal summary. No mean or standard deviation is calculated here."""
    column = df_raw[attribute_name]
    most_common = frequency.sort_values("Frequency", ascending=False).iloc[0]

    return pd.DataFrame([
        {
            "Attribute": attribute_name,
            "N": len(df_raw),
            "Missing": int(column.isna().sum()),
            "Unique Categories": int(column.nunique(dropna=True)),
            "Mode": int(most_common["Category"]),
            "Mode Meaning": most_common["Meaning"],
            "Mode Frequency": int(most_common["Frequency"]),
            "Mode Percentage": most_common["Percentage"],
        }
    ])
diverted_meanings = {0: "Not diverted", 1: "Diverted"}

diverted_frequency = binary_frequency_table("DIVERTED", diverted_meanings)
diverted_summary = binary_summary("DIVERTED", diverted_frequency)

diverted_frequency.to_csv("outputs/A2_DIVERTED_frequency.csv", index=False)
diverted_summary.to_csv("outputs/A2_DIVERTED_summary.csv", index=False)

print("Frequencies sum to:", int(diverted_frequency["Frequency"].sum()))
print("Percentages sum to:", round(diverted_frequency["Percentage"].sum(), 2))
diverted_frequency
diverted_summary
def save_binary_bar_chart(frequency, attribute_name, title):
    """Draw one bar chart of a binary flight-status attribute using readable labels."""
    figure, axes = plt.subplots(figsize=(7, 5))
    bars = axes.bar(frequency["Meaning"], frequency["Frequency"])

    axes.bar_label(bars, padding=2)
    axes.set_title(title)
    axes.set_xlabel("Flight status")
    axes.set_ylabel("Number of flights")
    axes.margins(y=0.15)
    figure.tight_layout()

    figure_path = f"figures/A2_{attribute_name}_bar.png"
    figure.savefig(figure_path, dpi=200)
    plt.show()
    print("Saved", figure_path)
save_binary_bar_chart(diverted_frequency, "DIVERTED",
                      "Distribution of diverted flights")
cancelled_meanings = {0: "Not cancelled", 1: "Cancelled"}

cancelled_frequency = binary_frequency_table("CANCELLED", cancelled_meanings)
cancelled_summary = binary_summary("CANCELLED", cancelled_frequency)

cancelled_frequency.to_csv("outputs/A2_CANCELLED_frequency.csv", index=False)
cancelled_summary.to_csv("outputs/A2_CANCELLED_summary.csv", index=False)

print("Frequencies sum to:", int(cancelled_frequency["Frequency"].sum()))
print("Percentages sum to:", round(cancelled_frequency["Percentage"].sum(), 2))
cancelled_frequency
cancelled_summary
save_binary_bar_chart(cancelled_frequency, "CANCELLED",
                      "Distribution of cancelled flights")

```

A2.5 Summary of the numeric and binary attributes

```

numeric_comparison = pd.DataFrame({
    "Statistic": list(scheduled_time_summary.keys()),
    "SCHEDULED_TIME": [format_value(v) for v in scheduled_time_summary.values()],
    "DISTANCE": [format_value(v) for v in distance_summary.values()],
})
numeric_comparison
overview_rows = [
    {"Attribute": "SCHEDULED_TIME", "Attribute Type": "Ratio",
     "N": scheduled_time_summary["N"], "Missing": scheduled_time_summary["Missing"],
     "Primary Summary": f"Mean {scheduled_time_summary['Mean']:.2f} minutes, "
                       f"median {format_value(scheduled_time_summary['Median'])}, "

```

```

        f"range {format_value(scheduled_time_summary['Minimum'])} to "
        f"{format_value(scheduled_time_summary['Maximum'])}",
{"Attribute": "DISTANCE", "Attribute Type": "Ratio",
 "N": distance_summary["N"], "Missing": distance_summary["Missing"],
 "Primary Summary": f"Mean {distance_summary['Mean']:.2f}, "
                    f"median {format_value(distance_summary['Median'])}, "
                    f"range {format_value(distance_summary['Minimum'])} to "
                    f"{format_value(distance_summary['Maximum'])}",
]

for name, frequency in [("DIVERTED", diverted_frequency), ("CANCELLED", cancelled_frequency)]:
    flagged = frequency[frequency["Category"] == 1].iloc[0]
    not_flagged = frequency[frequency["Category"] == 0].iloc[0]
    overview_rows.append({
        "Attribute": name, "Attribute Type": "Nominal",
        "N": len(df_raw), "Missing": int(df_raw[name].isna().sum()),
        "Primary Summary": f"({not_flagged['Meaning']}) {not_flagged['Frequency']} "
                          f"({not_flagged['Percentage']}%), {flagged['Meaning']} "
                          f"({flagged['Frequency']}) ({flagged['Percentage']}%)")

batch2_overview = pd.DataFrame(overview_rows)
batch2_overview.to_csv("outputs/A2_batch2_overview.csv", index=False)
batch2_overview

```

A2.6 Checks for the numeric and binary attributes

```

def is_sample_variance(attribute_name, reported_variance):
    """Confirm a reported variance uses the sample formula with n - 1 in the denominator."""
    column = df_raw[attribute_name]
    n = int(column.notna().sum())
    manual_sample = ((column - column.mean()) ** 2).sum() / (n - 1)
    population = column.var(ddof=0)
    return bool(np.isclose(reported_variance, manual_sample)
                and not np.isclose(reported_variance, population))

expected_figures_b2 = ["figures/A2_SCHEDULED_TIME_hist.png", "figures/A2_DISTANCE_hist.png",
                      "figures/A2_DIVERTED_bar.png", "figures/A2_CANCELLED_bar.png"]
expected_outputs_b2 = ["outputs/A2_SCHEDULED_TIME_summary.csv", "outputs/A2_DISTANCE_summary.csv",
                      "outputs/A2_DIVERTED_frequency.csv", "outputs/A2_DIVERTED_summary.csv",
                      "outputs/A2_CANCELLED_frequency.csv", "outputs/A2_CANCELLED_summary.csv",
                      "outputs/A2_batch2_overview.csv"]

# Field names used anywhere in the two nominal summaries, checked against numeric statistics.
nominal_fields = (list(diverted_frequency.columns) + list(cancelled_frequency.columns)
                  + list(diverted_summary.columns) + list(cancelled_summary.columns))
numeric_statistic_names = {"mean", "median", "variance", "standard deviation",
                           "std", "q1", "q3", "iqr", "range", "skew"}

fresh_copy = pd.read_csv(RAW_CSV)
print("Nominal field names checked:", sorted(set(nominal_fields)))
b2_checks = {
    "1. All four attributes exist": all(a in df_raw.columns for a in batch2_attributes),
    "2. All four have zero missing values": all(df_raw[a].isna().sum() == 0 for a in batch2_attributes),
    "3. df_raw remains 4349 by 25": df_raw.shape == (4349, 25),
    "4. SCHEDULED_TIME N equals 4349": scheduled_time_summary["N"] == 4349,
    "5. DISTANCE N equals 4349": distance_summary["N"] == 4349,
    "6. Ranges equal maximum minus minimum": (
        scheduled_time_summary["Range"] == scheduled_time_summary["Maximum"] - scheduled_time_summary["Minimum"]
        and distance_summary["Range"] == distance_summary["Maximum"] - distance_summary["Minimum"]),
    "7. Variance uses ddof=1 for both attributes": (
        is_sample_variance("SCHEDULED_TIME", scheduled_time_summary["Variance"])
        and is_sample_variance("DISTANCE", distance_summary["Variance"])),
    "8. Standard deviation is the square root of the sample variance": (
        np.isclose(scheduled_time_summary["Standard deviation"], np.sqrt(scheduled_time_summary["Variance"]))
        and np.isclose(distance_summary["Standard deviation"], np.sqrt(distance_summary["Variance"]))),
    "9. Q1 <= Median <= Q3 for both attributes": (
        scheduled_time_summary["Q1"] <= scheduled_time_summary["Median"] <= scheduled_time_summary["Q3"]
        and distance_summary["Q1"] <= distance_summary["Median"] <= distance_summary["Q3"]),
    "10. DIVERTED frequencies sum to 4349": int(diverted_frequency["Frequency"].sum()) == 4349,
    "11. DIVERTED percentages sum to about 100": abs(diverted_frequency["Percentage"].sum() - 100) <= 0.05,
    "12. CANCELLED frequencies sum to 4349": int(cancelled_frequency["Frequency"].sum()) == 4349,
    "13. CANCELLED percentages sum to about 100": abs(cancelled_frequency["Percentage"].sum() - 100) <= 0.05,
    "14. DIVERTED count for 1 matches the P0 value of 18": int((df_raw["DIVERTED"] == 1).sum()) == 18,
    "15. CANCELLED count for 1 matches the P0 value of 66": int((df_raw["CANCELLED"] == 1).sum()) == 66,
    "16. No numeric statistics applied to the two binary attributes": all(
        field.strip().lower() not in numeric_statistic_names for field in nominal_fields),
    "17. All four figure files exist": all(os.path.exists(p) for p in expected_figures_b2),
    "18. All expected output CSV files exist": all(os.path.exists(p) for p in expected_outputs_b2),
    "19. df_raw still matches the file on disk": df_raw.equals(fresh_copy),
    "20. IQR equals Q3 minus Q1 for both attributes": (
        scheduled_time_summary["IQR"] == scheduled_time_summary["Q3"] - scheduled_time_summary["Q1"]
        and distance_summary["IQR"] == distance_summary["Q3"] - distance_summary["Q1"]),
    "21. IQR is 84 for SCHEDULED_TIME and 659 for DISTANCE": (
        scheduled_time_summary["IQR"] == 84 and distance_summary["IQR"] == 659),
    "22. Skewness rounds to 1.43 for SCHEDULED_TIME and 1.52 for DISTANCE": (
        round(scheduled_time_summary["Skewness"], 2) == 1.43
        and round(distance_summary["Skewness"], 2) == 1.52),
}

validation_b2 = pd.DataFrame({"Check": list(b2_checks.keys()),
                              "Result": ["PASS" if v else "FAIL" for v in b2_checks.values()]})

```

```
print("Checks passed:", sum(b2_checks.values()), "of", len(b2_checks))
validation_b2
```

A2.7 Taxi and Flight Durations

```
duration_attributes = ["TAXI_OUT", "ELAPSED_TIME", "AIR_TIME", "TAXI_IN"]
```

```
print("Raw data shape:", df_raw.shape)
for attribute in duration_attributes:
    column = df_raw[attribute]
    missing = int(column.isna().sum())
    print(f"{attribute:<14} present={attribute in df_raw.columns} "
          f"valid={int(column.notna().sum())} missing={missing} "
          f"({missing / len(df_raw) * 100:.2f} per cent)")

print()
print("All four present:", all(a in df_raw.columns for a in duration_attributes))
print("Shape is 4349 by 25:", df_raw.shape == (4349, 25))
def describe_duration(series):
    """Return every statistic required for a duration attribute, ignoring missing values."""
    total_rows = len(series)
    missing = int(series.isna().sum())

    return {
        "Total rows": total_rows,
        "Valid N": int(series.notna().sum()),
        "Missing": missing,
        "Missing percentage": missing / total_rows * 100,
        "Mean": series.mean(),
        "Median": series.median(),
        "Minimum": series.min(),
        "Maximum": series.max(),
        "Range": series.max() - series.min(),
        "Variance": series.var(ddof=1), # sample variance, divides by n - 1
        "Standard deviation": series.std(ddof=1), # sample standard deviation
        "Q1": series.quantile(0.25),
        "Q3": series.quantile(0.75),
        "IQR": series.quantile(0.75) - series.quantile(0.25),
        "Skewness": series.skew(),
    }

def missing_value_comparison(attribute_name):
    """Describe one attribute before and after attribute-wise removal of its missing values."""
    original = df_raw[attribute_name]
    clean = original.dropna() # temporary Series only, df_raw is never changed
    return describe_duration(original), describe_duration(clean)

def comparison_table(before, after):
    """Place the before and after statistics side by side for the report."""
    return pd.DataFrame({
        "Statistic": list(before.keys()),
        "Original column": [format_value(value) for value in before.values()],
        "After removal": [format_value(after[name]) for name in before],
    })

duration_before, duration_after, duration_clean = {}, {}, {}
for attribute in duration_attributes:
    duration_before[attribute], duration_after[attribute] = missing_value_comparison(attribute)
    duration_clean[attribute] = df_raw[attribute].dropna()

print("Described attributes:", list(duration_before))
def save_missing_comparison_figure(attribute_name):
    """Draw the recorded values before and after removal using one shared set of bin edges."""
    original = df_raw[attribute_name]
    clean = duration_clean[attribute_name]
    bin_edges = np.histogram_bin_edges(clean, bins="fd") # Freedman-Diaconis, no manual count
    before, after = duration_before[attribute_name], duration_after[attribute_name]

    notes = [f"Total rows {before['Total rows']}\nValid N {before['Valid N']}\n"
             f"Missing {before['Missing']}\n"
             f"Missing {before['Missing percentage']:.2f} per cent",
             f"Rows after removal {after['Total rows']}\nMissing 0"]
    panel_titles = [f"{attribute_name}: original recorded values",
                    f"{attribute_name}: after removal"]

    figure, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
    for panel, values, title, note in zip(axes, [original.dropna(), clean], panel_titles, notes):
        # A thin edge line keeps the bars visible when the rule returns many narrow bins.
        panel.hist(values, bins=bin_edges, edgecolor="white", linewidth=0.3)
        panel.set_xlim(bin_edges[0], bin_edges[-1])
        panel.set_title(title)
        panel.set_xlabel(f"{attribute_name} (minutes)")
        panel.annotate(note, xy=(0.97, 0.95), xycoords="axes fraction",
                       ha="right", va="top", fontsize=9)

    axes[0].set_ylabel("Number of flights")
    figure.tight_layout()
    figure_path = f"figures/A2_{attribute_name}_missing_comparison.png"
    figure.savefig(figure_path, dpi=200)
    plt.show()
    print("Saved", figure_path, "using", len(bin_edges) - 1, "shared bins")
```

```

taxi_out_before, taxi_out_after = duration_before["TAXI_OUT"], duration_after["TAXI_OUT"]

print("Full precision values for TAXI_OUT after removal:")
for name, value in taxi_out_after.items():
    print(f" {name:<20} {value}")

taxi_out_comparison = comparison_table(taxi_out_before, taxi_out_after)
taxi_out_comparison.to_csv("outputs/A2_TAXI_OUT_missing_comparison.csv", index=False)
taxi_out_comparison
save_missing_comparison_figure("TAXI_OUT")
elapsed_before, elapsed_after = duration_before["ELAPSED_TIME"], duration_after["ELAPSED_TIME"]

print("Full precision values for ELAPSED_TIME after removal:")
for name, value in elapsed_after.items():
    print(f" {name:<20} {value}")

elapsed_comparison = comparison_table(elapsed_before, elapsed_after)
elapsed_comparison.to_csv("outputs/A2_ELAPSED_TIME_missing_comparison.csv", index=False)
elapsed_comparison
save_missing_comparison_figure("ELAPSED_TIME")
air_time_before, air_time_after = duration_before["AIR_TIME"], duration_after["AIR_TIME"]

print("Full precision values for AIR_TIME after removal:")
for name, value in air_time_after.items():
    print(f" {name:<20} {value}")

air_time_comparison = comparison_table(air_time_before, air_time_after)
air_time_comparison.to_csv("outputs/A2_AIR_TIME_missing_comparison.csv", index=False)
air_time_comparison
save_missing_comparison_figure("AIR_TIME")
taxi_in_before, taxi_in_after = duration_before["TAXI_IN"], duration_after["TAXI_IN"]

print("Full precision values for TAXI_IN after removal:")
for name, value in taxi_in_after.items():
    print(f" {name:<20} {value}")

taxi_in_comparison = comparison_table(taxi_in_before, taxi_in_after)
taxi_in_comparison.to_csv("outputs/A2_TAXI_IN_missing_comparison.csv", index=False)
taxi_in_comparison
save_missing_comparison_figure("TAXI_IN")
combined_statistics = ["N", "Missing", "Mean", "Median", "Minimum", "Maximum", "Range",
                       "Variance", "Standard deviation", "Q1", "Q3", "IQR", "Skewness"]

def after_removal_value(attribute_name, statistic):
    """Read one after-removal statistic, mapping the report label N onto Valid N."""
    key = "Valid N" if statistic == "N" else statistic
    return duration_after[attribute_name][key]

duration_clean_summary = pd.DataFrame({"Statistic": combined_statistics})
for attribute in duration_attributes:
    duration_clean_summary[attribute] = [format_value(after_removal_value(attribute, statistic))
                                         for statistic in combined_statistics]

duration_clean_summary.to_csv("outputs/A2_duration_summary_after_removal.csv", index=False)
duration_clean_summary
overview_rows = []
for attribute in duration_attributes:
    before = duration_before[attribute]
    overview_rows.append({
        "Attribute": attribute,
        "Total Rows": before["Total rows"],
        "Valid N": before["Valid N"],
        "Missing": before["Missing"],
        "Missing Percentage": round(before["Missing percentage"], 2),
        "Rows After Removal": duration_after[attribute]["Total rows"],
    })

duration_missing_overview = pd.DataFrame(overview_rows)
duration_missing_overview.to_csv("outputs/A2_duration_missing_overview.csv", index=False)
duration_missing_overview
duration_status_mask = (df_raw["CANCELLED"] == 1) | (df_raw["DIVERTED"] == 1)
print("Rows that are cancelled or diverted:", int(duration_status_mask.sum()))

status_rows = []
for attribute in duration_attributes:
    missing_mask = df_raw[attribute].isna()
    status_rows.append({
        "Attribute": attribute,
        "Missing": int(missing_mask.sum()),
        "Missing and cancelled": int((missing_mask & (df_raw["CANCELLED"] == 1)).sum()),
        "Missing and diverted": int((missing_mask & (df_raw["DIVERTED"] == 1)).sum()),
        "Missing and neither": int((missing_mask & ~duration_status_mask).sum()),
        "Missing rows equal cancelled or diverted rows":
            bool((missing_mask == duration_status_mask).all()),
    })

duration_status_check = pd.DataFrame(status_rows)

```

```

duration_status_check.to_csv("outputs/A2_duration_missing_status_check.csv", index=False)
duration_status_check
def recorded_values_match(attribute_name):
    """Confirm the cleaned Series holds exactly the recorded values, in the original order."""
    original = df_raw[attribute_name]
    recorded = original[original.notna()] # selected with a mask, not with a second dropna
    clean = duration_clean[attribute_name]
    return bool(recorded.index.equals(clean.index)
                and np.array_equal(recorded.to_numpy(), clean.to_numpy()))

def status_counts_match(attribute_name):
    """Recompute the status split of the missing rows and compare it with the saved table."""
    missing_mask = df_raw[attribute_name].isna()
    cancelled = int((missing_mask & (df_raw["CANCELLED"] == 1)).sum())
    diverted = int((missing_mask & (df_raw["DIVERTED"] == 1)).sum())
    neither = int((missing_mask & ~duration_status_mask).sum())
    row = duration_status_check.set_index("Attribute").loc[attribute_name]
    return bool(cancelled == row["Missing and cancelled"]
                and diverted == row["Missing and diverted"]
                and neither == row["Missing and neither"]
                and cancelled + diverted + neither == int(missing_mask.sum()))

expected_figures_a27 = [f"figures/A2_{a}_missing_comparison.png" for a in duration_attributes]
expected_outputs_a27 = ([f"outputs/A2_{a}_missing_comparison.csv" for a in duration_attributes]
                        + [f"outputs/A2_duration_summary_after_removal.csv",
                           "outputs/A2_duration_missing_overview.csv",
                           "outputs/A2_duration_missing_status_check.csv"])

fresh_duration_copy = pd.read_csv(RAW_CSV)
print("Files to check:", len(expected_figures_a27), "figures and", len(expected_outputs_a27), "tables")
a27_checks = {
    "1. All four attributes exist": all(a in df_raw.columns for a in duration_attributes),
    "2. df_raw remains 4349 by 25": df_raw.shape == (4349, 25),
    "3. TAXI_OUT missing equals 65": duration_before["TAXI_OUT"]["Missing"] == 65,
    "4. ELAPSED_TIME missing equals 84": duration_before["ELAPSED_TIME"]["Missing"] == 84,
    "5. AIR_TIME missing equals 84": duration_before["AIR_TIME"]["Missing"] == 84,
    "6. TAXI_IN missing equals 67": duration_before["TAXI_IN"]["Missing"] == 67,
    "7. TAXI_OUT cleaned N equals 4284": duration_after["TAXI_OUT"]["Valid N"] == 4284,
    "8. ELAPSED_TIME cleaned N equals 4265": duration_after["ELAPSED_TIME"]["Valid N"] == 4265,
    "9. AIR_TIME cleaned N equals 4265": duration_after["AIR_TIME"]["Valid N"] == 4265,
    "10. TAXI_IN cleaned N equals 4282": duration_after["TAXI_IN"]["Valid N"] == 4282,
    "11. Every cleaned Series has zero missing values": all(
        duration_after[a]["Missing"] == 0 for a in duration_attributes),
    "12. Cleaned values equal the recorded values in the same order": all(
        recorded_values_match(a) for a in duration_attributes),
    "13. Means unchanged by removal": all(
        np.isclose(duration_before[a]["Mean"], duration_after[a]["Mean"])
        for a in duration_attributes),
    "14. Medians unchanged by removal": all(
        duration_before[a]["Median"] == duration_after[a]["Median"] for a in duration_attributes),
    "15. Variances unchanged by removal": all(
        np.isclose(duration_before[a]["Variance"], duration_after[a]["Variance"])
        for a in duration_attributes),
    "16. Standard deviations unchanged by removal": all(
        np.isclose(duration_before[a]["Standard deviation"], duration_after[a]["Standard deviation"])
        for a in duration_attributes),
    "17. Quartiles unchanged by removal": all(
        duration_before[a]["Q1"] == duration_after[a]["Q1"]
        and duration_before[a]["Q3"] == duration_after[a]["Q3"] for a in duration_attributes),
    "18. IQR values unchanged by removal": all(
        duration_before[a]["IQR"] == duration_after[a]["IQR"] for a in duration_attributes),
    "19. Skewness values unchanged by removal": all(
        np.isclose(duration_before[a]["Skewness"], duration_after[a]["Skewness"])
        for a in duration_attributes),
    "20. Variance uses ddof=1 for all four attributes": all(
        is_sample_variance(a, duration_after[a]["Variance"]) for a in duration_attributes),
    "21. ELAPSED_TIME missing rows equal the cancelled or diverted rows": bool(
        (df_raw["ELAPSED_TIME"].isna() == duration_status_mask).all()),
    "22. AIR_TIME missing rows equal the cancelled or diverted rows": bool(
        (df_raw["AIR_TIME"].isna() == duration_status_mask).all()),
    "23. TAXI_OUT missing status counts match the recomputed data": status_counts_match("TAXI_OUT"),
    "24. TAXI_IN missing status counts match the recomputed data": status_counts_match("TAXI_IN"),
    "25. All four comparison figures exist": all(
        os.path.exists(p) for p in expected_figures_a27),
    "26. All expected output CSV files exist": all(
        os.path.exists(p) for p in expected_outputs_a27),
    "27. df_raw still matches the file on disk": df_raw.equals(fresh_duration_copy),
}

validation_a27 = pd.DataFrame({"Check": list(a27_checks.keys()),
                              "Result": ["PASS" if v else "FAIL" for v in a27_checks.values()]})
print("Checks passed:", sum(a27_checks.values()), "of", len(a27_checks))
validation_a27

A2.8 Delay Attributes
delay_attributes = ["DEPARTURE_DELAY", "ARRIVAL_DELAY",
                   "AIR_SYSTEM_DELAY", "SECURITY_DELAY", "WEATHER_DELAY"]
signed_delay_attributes = ["DEPARTURE_DELAY", "ARRIVAL_DELAY"]
cause_delay_attributes = ["AIR_SYSTEM_DELAY", "SECURITY_DELAY", "WEATHER_DELAY"]

```

```

print("Raw data shape:", df_raw.shape)
for attribute in delay_attributes:
    column = df_raw[attribute]
    missing = int(column.isna().sum())
    print(f"{attribute:<18} present={attribute in df_raw.columns} "
          f"valid={int(column.notna().sum())} missing={missing} "
          f"({missing / len(df_raw) * 100:.2f} per cent)")

print()
print("All five present:", all(a in df_raw.columns for a in delay_attributes))
print("Shape is 4349 by 25:", df_raw.shape == (4349, 25))

def describe_numeric_series(series):
    """Return the full set of numeric summary statistics for one Series."""
    total_rows = len(series)
    missing = int(series.isna().sum())

    return {
        "Total rows": total_rows,
        "Valid N": int(series.notna().sum()),
        "Missing": missing,
        "Missing percentage": missing / total_rows * 100,
        "Mean": series.mean(),
        "Median": series.median(),
        "Minimum": series.min(),
        "Maximum": series.max(),
        "Range": series.max() - series.min(),
        "Variance": series.var(ddof=1), # sample variance, divides by n - 1
        "Standard deviation": series.std(ddof=1), # sample standard deviation
        "Q1": series.quantile(0.25),
        "Q3": series.quantile(0.75),
        "IQR": series.quantile(0.75) - series.quantile(0.25),
        "Skewness": series.skew(),
    }

def delay_missing_comparison(attribute_name):
    """Describe one delay attribute before and after attribute-wise removal."""
    original = df_raw[attribute_name]
    clean = original.dropna() # temporary Series only, df_raw is never changed
    return describe_numeric_series(original), describe_numeric_series(clean)

delay_before, delay_after, delay_clean = {}, {}, {}
for attribute in delay_attributes:
    delay_before[attribute], delay_after[attribute] = delay_missing_comparison(attribute)
    delay_clean[attribute] = df_raw[attribute].dropna()

print("Described attributes:", list(delay_before))

def save_delay_comparison_figure(attribute_name):
    """Draw the recorded values before and after removal using one shared set of bin edges."""
    original = df_raw[attribute_name]
    clean = delay_clean[attribute_name]
    bin_edges = np.histogram_bin_edges(clean, bins="auto")
    before, after = delay_before[attribute_name], delay_after[attribute_name]

    notes = [f"Total rows {before['Total rows']}\nValid N {before['Valid N']}\n"
             f"Missing {before['Missing']}\n"
             f"Missing {before['Missing percentage']:.2f} per cent",
             f"Rows after removal {after['Total rows']}\nMissing 0"]
    panel_titles = [f"{attribute_name}: original recorded values",
                    f"{attribute_name}: after removal"]

    figure, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
    for panel, values, title, note in zip(axes, [original.dropna(), clean], panel_titles, notes):
        # A thin edge line keeps the bars visible when the rule returns many narrow bins.
        panel.hist(values, bins=bin_edges, edgecolor="white", linewidth=0.3)
        panel.set_xlim(bin_edges[0], bin_edges[-1])
        panel.set_title(title)
        panel.set_xlabel(f"{attribute_name} (minutes)")
        panel.annotate(note, xy=(0.97, 0.95), xycoords="axes fraction",
                       ha="right", va="top", fontsize=9)

    axes[0].set_ylabel("Number of flights")
    figure.tight_layout()
    figure_path = f"figures/A2_{attribute_name}_missing_comparison.png"
    figure.savefig(figure_path, dpi=200)
    plt.show()
    print("Saved", figure_path, "using", len(bin_edges) - 1, "shared bins")

departure_before = delay_before["DEPARTURE_DELAY"]
departure_after = delay_after["DEPARTURE_DELAY"]

print("Full precision values for DEPARTURE_DELAY after removal:")
for name, value in departure_after.items():
    print(f" {name:<20} {value}")

departure_comparison = comparison_table(departure_before, departure_after)
departure_comparison.to_csv("outputs/A2_DEPARTURE_DELAY_missing_comparison.csv", index=False)
departure_comparison
save_delay_comparison_figure("DEPARTURE_DELAY")

```

```

arrival_before = delay_before["ARRIVAL_DELAY"]
arrival_after = delay_after["ARRIVAL_DELAY"]

print("Full precision values for ARRIVAL_DELAY after removal:")
for name, value in arrival_after.items():
    print(f" {name:<20} {value}")

arrival_comparison = comparison_table(arrival_before, arrival_after)
arrival_comparison.to_csv("outputs/A2_ARRIVAL_DELAY_missing_comparison.csv", index=False)
arrival_comparison
save_delay_comparison_figure("ARRIVAL_DELAY")

air_system_before = delay_before["AIR_SYSTEM_DELAY"]
air_system_after = delay_after["AIR_SYSTEM_DELAY"]

print("Full precision values for AIR_SYSTEM_DELAY after removal:")
for name, value in air_system_after.items():
    print(f" {name:<20} {value}")

print()
print("Recorded zeros:", int((delay_clean["AIR_SYSTEM_DELAY"] == 0).sum()))
print("Recorded positive values:", int((delay_clean["AIR_SYSTEM_DELAY"] > 0).sum()))

air_system_comparison = comparison_table(air_system_before, air_system_after)
air_system_comparison.to_csv("outputs/A2_AIR_SYSTEM_DELAY_missing_comparison.csv", index=False)
air_system_comparison
save_delay_comparison_figure("AIR_SYSTEM_DELAY")

security_before = delay_before["SECURITY_DELAY"]
security_after = delay_after["SECURITY_DELAY"]

print("Full precision values for SECURITY_DELAY after removal:")
for name, value in security_after.items():
    print(f" {name:<20} {value}")

print()
print("Recorded zeros:", int((delay_clean["SECURITY_DELAY"] == 0).sum()))
print("Recorded positive values:", int((delay_clean["SECURITY_DELAY"] > 0).sum()))

security_comparison = comparison_table(security_before, security_after)
security_comparison.to_csv("outputs/A2_SECURITY_DELAY_missing_comparison.csv", index=False)
security_comparison
save_delay_comparison_figure("SECURITY_DELAY")

weather_before = delay_before["WEATHER_DELAY"]
weather_after = delay_after["WEATHER_DELAY"]

print("Full precision values for WEATHER_DELAY after removal:")
for name, value in weather_after.items():
    print(f" {name:<20} {value}")

print()
print("Recorded zeros:", int((delay_clean["WEATHER_DELAY"] == 0).sum()))
print("Recorded positive values:", int((delay_clean["WEATHER_DELAY"] > 0).sum()))

weather_comparison = comparison_table(weather_before, weather_after)
weather_comparison.to_csv("outputs/A2_WEATHER_DELAY_missing_comparison.csv", index=False)
weather_comparison
save_delay_comparison_figure("WEATHER_DELAY")

delay_combined_statistics = ["N", "Missing", "Mean", "Median", "Minimum", "Maximum", "Range",
                             "Variance", "Standard deviation", "Q1", "Q3", "IQR", "Skewness"]

def delay_after_value(attribute_name, statistic):
    """Read one after-removal statistic, mapping the report label N onto Valid N."""
    key = "Valid N" if statistic == "N" else statistic
    return delay_after[attribute_name][key]

delay_clean_summary = pd.DataFrame({"Statistic": delay_combined_statistics})
for attribute in delay_attributes:
    delay_clean_summary[attribute] = [
        format_value(delay_after_value(attribute, statistic))
        for statistic in delay_combined_statistics
    ]

delay_clean_summary.to_csv("outputs/A2_delay_summary_after_removal.csv", index=False)
delay_clean_summary
delay_overview_rows = []
for attribute in delay_attributes:
    before = delay_before[attribute]
    delay_overview_rows.append({
        "Attribute": attribute,
        "Total Rows": before["Total rows"],
        "Valid N": before["Valid N"],
        "Missing": before["Missing"],
        "Missing Percentage": round(before["Missing percentage"], 2),
        "Rows After Removal": delay_after[attribute]["Total rows"],
    })

delay_missing_overview = pd.DataFrame(delay_overview_rows)
delay_missing_overview.to_csv("outputs/A2_delay_missing_overview.csv", index=False)

```

```

delay_missing_overview
signed_rows = []
for attribute in signed_delay_attributes:
    clean = delay_clean[attribute]
    negative = int((clean < 0).sum())
    zero = int((clean == 0).sum())
    positive = int((clean > 0).sum())
    signed_rows.append({
        "Attribute": attribute,
        "Negative": negative,
        "Zero": zero,
        "Positive": positive,
        "Valid N": int(clean.notna().sum()),
    })

signed_delay_counts = pd.DataFrame(signed_rows)
signed_delay_counts.to_csv("outputs/A2_signed_delay_counts.csv", index=False)
print("Each signed split sums to Valid N:",
      bool(((signed_delay_counts["Negative"]
        + signed_delay_counts["Zero"]
        + signed_delay_counts["Positive"])
        == signed_delay_counts["Valid N"]).all()))

signed_delay_counts
delay_status_mask = (df_raw["CANCELLED"] == 1) | (df_raw["DIVERTED"] == 1)
departure_missing = df_raw["DEPARTURE_DELAY"].isna()
arrival_missing = df_raw["ARRIVAL_DELAY"].isna()

delay_status_check = pd.DataFrame([
    {
        "Attribute": "DEPARTURE_DELAY",
        "Missing": int(departure_missing.sum()),
        "Missing and cancelled": int((departure_missing & (df_raw["CANCELLED"] == 1)).sum()),
        "Missing and diverted": int((departure_missing & (df_raw["DIVERTED"] == 1)).sum()),
        "Missing and neither": int((departure_missing & ~delay_status_mask).sum()),
        "Cancelled with recorded value": int(
            ((df_raw["CANCELLED"] == 1) & df_raw["DEPARTURE_DELAY"].notna()).sum()),
        "All missing rows are cancelled": bool(
            (departure_missing & (df_raw["CANCELLED"] == 1)).sum() == departure_missing.sum()),
        "Missing rows equal all cancelled rows": bool(
            (departure_missing == (df_raw["CANCELLED"] == 1)).all()),
        "Missing rows equal cancelled or diverted rows": bool(
            (departure_missing == delay_status_mask).all()),
    },
    {
        "Attribute": "ARRIVAL_DELAY",
        "Missing": int(arrival_missing.sum()),
        "Missing and cancelled": int((arrival_missing & (df_raw["CANCELLED"] == 1)).sum()),
        "Missing and diverted": int((arrival_missing & (df_raw["DIVERTED"] == 1)).sum()),
        "Missing and neither": int((arrival_missing & ~delay_status_mask).sum()),
        "Cancelled with recorded value": int(
            ((df_raw["CANCELLED"] == 1) & df_raw["ARRIVAL_DELAY"].notna()).sum()),
        "All missing rows are cancelled": bool(
            (arrival_missing & (df_raw["CANCELLED"] == 1)).sum() == arrival_missing.sum()),
        "Missing rows equal all cancelled rows": bool(
            (arrival_missing == (df_raw["CANCELLED"] == 1)).all()),
        "Missing rows equal cancelled or diverted rows": bool(
            (arrival_missing == delay_status_mask).all()),
    }
])

delay_status_check.to_csv("outputs/A2_delay_status_check.csv", index=False)
delay_status_check
arrival_delay_ge_15 = df_raw["ARRIVAL_DELAY"].ge(15)
cause_missing_masks_identical = all(
    df_raw[attribute].isna().equals(df_raw["AIR_SYSTEM_DELAY"].isna())
    for attribute in cause_delay_attributes
)

cause_rows = []
for attribute in cause_delay_attributes:
    clean = delay_clean[attribute]
    cause_rows.append({
        "Attribute": attribute,
        "Recorded N": int(clean.notna().sum()),
        "Missing": int(df_raw[attribute].isna().sum()),
        "Missing Percentage": round(df_raw[attribute].isna().mean() * 100, 2),
        "Recorded Zero": int((clean == 0).sum()),
        "Recorded Positive": int((clean > 0).sum()),
        "Recorded Rows Equal ARRIVAL_DELAY >= 15": bool(
            (df_raw[attribute].notna() == arrival_delay_ge_15).all()),
    })

delay_cause_eligibility = pd.DataFrame(cause_rows)
delay_cause_eligibility.to_csv("outputs/A2_delay_cause_eligibility_check.csv", index=False)

print("Identical missing masks across the three cause columns:",
      cause_missing_masks_identical)
print("ARRIVAL_DELAY >= 15 count:", int(arrival_delay_ge_15.sum()))
delay_cause_eligibility
def delay_recorded_values_match(attribute_name):

```

```

"""Confirm the cleaned Series holds exactly the recorded values, in the original order."""
original = df_raw[attribute_name]
recorded = original[original.notna()]
clean = delay_clean[attribute_name]
return bool(recorded.index.equals(clean.index)
            and np.array_equal(recorded.to_numpy(), clean.to_numpy()))

expected_figures_a28 = [f"figures/A2_{a}_missing_comparison.png" for a in delay_attributes]
expected_outputs_a28 = ([f"outputs/A2_{a}_missing_comparison.csv" for a in delay_attributes]
                        + [f"outputs/A2_delay_missing_overview.csv",
                           f"outputs/A2_delay_summary_after_removal.csv",
                           f"outputs/A2_signed_delay_counts.csv",
                           f"outputs/A2_delay_status_check.csv",
                           f"outputs/A2_delay_cause_eligibility_check.csv"])

fresh_delay_copy = pd.read_csv(RAW_CSV)
print("Files to check:", len(expected_figures_a28), "figures and", len(expected_outputs_a28), "tables")
a28_checks = {
    "1. All five attributes exist": all(a in df_raw.columns for a in delay_attributes),
    "2. df_raw remains 4349 by 25": df_raw.shape == (4349, 25),
    "3. DEPARTURE_DELAY missing equals 61": delay_before["DEPARTURE_DELAY"]["Missing"] == 61,
    "4. ARRIVAL_DELAY missing equals 84": delay_before["ARRIVAL_DELAY"]["Missing"] == 84,
    "5. AIR_SYSTEM_DELAY missing equals 3522": delay_before["AIR_SYSTEM_DELAY"]["Missing"] == 3522,
    "6. SECURITY_DELAY missing equals 3522": delay_before["SECURITY_DELAY"]["Missing"] == 3522,
    "7. WEATHER_DELAY missing equals 3522": delay_before["WEATHER_DELAY"]["Missing"] == 3522,
    "8. DEPARTURE_DELAY cleaned N equals 4288": delay_after["DEPARTURE_DELAY"]["Valid N"] == 4288,
    "9. ARRIVAL_DELAY cleaned N equals 4265": delay_after["ARRIVAL_DELAY"]["Valid N"] == 4265,
    "10. Each delay-cause cleaned N equals 827": all(
        delay_after[a]["Valid N"] == 827 for a in cause_delay_attributes),
    "11. Every cleaned Series has zero missing values": all(
        delay_after[a]["Missing"] == 0 for a in delay_attributes),
    "12. Cleaned values equal the recorded values in the same order": all(
        delay_recorded_values_match(a) for a in delay_attributes),
    "13. Means unchanged by removal": all(
        np.isclose(delay_before[a]["Mean"], delay_after[a]["Mean"]) for a in delay_attributes),
    "14. Medians unchanged by removal": all(
        delay_before[a]["Median"] == delay_after[a]["Median"] for a in delay_attributes),
    "15. Variances unchanged by removal": all(
        np.isclose(delay_before[a]["Variance"], delay_after[a]["Variance"])
        for a in delay_attributes),
    "16. Standard deviations unchanged by removal": all(
        np.isclose(delay_before[a]["Standard deviation"], delay_after[a]["Standard deviation"])
        for a in delay_attributes),
    "17. Quartiles unchanged by removal": all(
        delay_before[a]["Q1"] == delay_after[a]["Q1"]
        and delay_before[a]["Q3"] == delay_after[a]["Q3"] for a in delay_attributes),
    "18. IQR values unchanged by removal": all(
        delay_before[a]["IQR"] == delay_after[a]["IQR"] for a in delay_attributes),
    "19. Skewness values unchanged by removal": all(
        np.isclose(delay_before[a]["Skewness"], delay_after[a]["Skewness"])
        for a in delay_attributes),
    "20. Variance uses ddof=1 for all five attributes": all(
        is_sample_variance(a, delay_after[a]["Variance"]) for a in delay_attributes),
    "21. All 61 missing DEPARTURE_DELAY rows are cancelled": bool(
        (df_raw["DEPARTURE_DELAY"].isna() & (df_raw["CANCELLED"] == 1)).sum() == 61),
    "22. DEPARTURE_DELAY missing rows are not equal to all cancelled rows": bool(
        not (df_raw["DEPARTURE_DELAY"].isna() == (df_raw["CANCELLED"] == 1)).all()),
    "23. Exactly 5 cancelled flights retain recorded DEPARTURE_DELAY": int(
        ((df_raw["CANCELLED"] == 1) & df_raw["DEPARTURE_DELAY"].notna()).sum()) == 5,
    "24. Missing ARRIVAL_DELAY rows equal cancelled or diverted rows": bool(
        (df_raw["ARRIVAL_DELAY"].isna() == delay_status_mask).all()),
    "25. All three delay-cause missing masks are identical": cause_missing_masks_identical,
    "26. Each delay-cause recorded mask equals ARRIVAL_DELAY >= 15": all(
        (df_raw[a].notna() == arrival_delay_ge_15).all() for a in cause_delay_attributes),
    "27. ARRIVAL_DELAY >= 15 count equals 827": int(arrival_delay_ge_15.sum()) == 827,
    "28. AIR_SYSTEM_DELAY recorded zero count equals 432":
        int((delay_clean["AIR_SYSTEM_DELAY"] == 0).sum()) == 432,
    "29. SECURITY_DELAY recorded zero count equals 823":
        int((delay_clean["SECURITY_DELAY"] == 0).sum()) == 823,
    "30. WEATHER_DELAY recorded zero count equals 777":
        int((delay_clean["WEATHER_DELAY"] == 0).sum()) == 777,
    "31. Signed-delay negative, zero and positive counts sum to Valid N": bool(
        ((signed_delay_counts["Negative"]
          + signed_delay_counts["Zero"]
          + signed_delay_counts["Positive"])
         == signed_delay_counts["Valid N"]).all()),
    "32. All five comparison figures exist": all(
        os.path.exists(p) for p in expected_figures_a28),
    "33. All expected output CSV files exist": all(
        os.path.exists(p) for p in expected_outputs_a28),
    "34. df_raw still matches the file on disk": df_raw.equals(fresh_delay_copy),
}

validation_a28 = pd.DataFrame({"Check": list(a28_checks.keys()),
                              "Result": ["PASS" if v else "FAIL" for v in a28_checks.values()]})
print("Checks passed:", sum(a28_checks.values()), "of", len(a28_checks))
validation_a28

```

A2.9 Impact of Missing Values and Revised Figures

```
median_impact_attributes = [
```

```

    "TAXI_OUT",
    "ELAPSED_TIME",
    "AIR_TIME",
    "TAXI_IN",
    "DEPARTURE_DELAY",
    "ARRIVAL_DELAY",
]

median_impact_rows = []
for attribute in median_impact_attributes:
    removal = df_raw[attribute].dropna()
    imputed = df_raw[attribute].fillna(df_raw[attribute].median())

    sd_removal = removal.std(ddof=1)
    sd_imputed = imputed.std(ddof=1)
    sd_change_percent = (sd_imputed - sd_removal) / sd_removal * 100

    median_impact_rows.append({
        "Attribute": attribute,
        "N removal": int(len(removal)),
        "N imputed": int(len(imputed)),
        "Mean removal": round(removal.mean(), 2),
        "Mean imputed": round(imputed.mean(), 2),
        "SD removal": round(sd_removal, 2),
        "SD imputed": round(sd_imputed, 2),
        "Q1 removal": round(removal.quantile(0.25), 2),
        "Q1 imputed": round(imputed.quantile(0.25), 2),
        "Q3 removal": round(removal.quantile(0.75), 2),
        "Q3 imputed": round(imputed.quantile(0.75), 2),
        "Skewness removal": round(removal.skew(), 2),
        "Skewness imputed": round(imputed.skew(), 2),
        "SD change percent": round(sd_change_percent, 2),
    })

median_imputation_impact = pd.DataFrame(median_impact_rows)
median_imputation_impact.to_csv(
    "outputs/A2_missing_impact_median_imputation.csv",
    index=False
)

median_imputation_impact
blank_as_zero_attributes = [
    "AIR_SYSTEM_DELAY",
    "SECURITY_DELAY",
    "WEATHER_DELAY",
]

blank_as_zero_rows = []
for attribute in blank_as_zero_attributes:
    recorded = df_raw[attribute].dropna()
    zero_demo = df_raw[attribute].fillna(0)

    mean_recorded = recorded.mean()
    mean_zero_demo = zero_demo.mean()
    mean_fall_percent = (mean_recorded - mean_zero_demo) / mean_recorded * 100

    blank_as_zero_rows.append({
        "Attribute": attribute,
        "N blanks missing": int(len(recorded)),
        "N blanks as zero": int(len(zero_demo)),
        "Mean blanks missing": round(mean_recorded, 2),
        "Mean blanks as zero": round(mean_zero_demo, 2),
        "SD blanks missing": round(recorded.std(ddof=1), 2),
        "SD blanks as zero": round(zero_demo.std(ddof=1), 2),
        "Mean fall percent": round(mean_fall_percent, 2),
    })

blank_as_zero_demo = pd.DataFrame(blank_as_zero_rows)
blank_as_zero_demo.to_csv(
    "outputs/A2_delay_cause_blank_as_zero_demo.csv",
    index=False
)

blank_as_zero_demo
delayed = df_raw[df_raw["ARRIVAL_DELAY"] >= 15].copy()
assert len(delayed) == 827, f"Expected 827 delayed flights, found {len(delayed)}"

cause_sum = (
    delayed["AIR_SYSTEM_DELAY"]
    + delayed["SECURITY_DELAY"]
    + delayed["WEATHER_DELAY"]
)
assert cause_sum.isna().sum() == 0, "Unexpected blanks among the delayed-flight cause fields"

delay_cause_coverage = pd.DataFrame([
    "Delayed flights": int(len(delayed)),
    "Cause sum below arrival delay": int((cause_sum < delayed["ARRIVAL_DELAY"]).sum()),
    "Cause sum equal to arrival delay": int((cause_sum == delayed["ARRIVAL_DELAY"]).sum()),
    "Cause sum above arrival delay": int((cause_sum > delayed["ARRIVAL_DELAY"]).sum()),
    "All three causes zero": int(
        ((delayed["AIR_SYSTEM_DELAY"] == 0)
         & (delayed["SECURITY_DELAY"] == 0)
         & (delayed["WEATHER_DELAY"] == 0)).sum()
    )
])

```

```

    ),
    "Share of delay minutes explained percent": round(
        cause_sum.sum() / delayed["ARRIVAL_DELAY"].sum() * 100,
        2
    ),
})

delay_cause_coverage.to_csv(
    "outputs/A2_delay_cause_coverage.csv",
    index=False
)

delay_cause_coverage
duration_complete_mask = df_raw[[
    "TAXI_OUT",
    "AIR_TIME",
    "TAXI_IN",
    "ELAPSED_TIME",
]].notna().all(axis=1)

duration_complete = df_raw.loc[duration_complete_mask]
duration_identity_holds = (
    duration_complete["ELAPSED_TIME"]
    == (
        duration_complete["TAXI_OUT"]
        + duration_complete["AIR_TIME"]
        + duration_complete["TAXI_IN"]
    )
)

duration_consistency = pd.DataFrame([
    "Rows with all four recorded": int(duration_complete_mask.sum()),
    "Rows where the identity holds": int(duration_identity_holds.sum()),
])

duration_consistency.to_csv(
    "outputs/A2_duration_consistency_check.csv",
    index=False
)

duration_consistency

def save_imputation_figure(attribute_name, bin_width, display_max=None, display_min=None):
    """Compare removal with median imputation for one attribute, using shared bins."""
    with plt.rc_context({"font.size": 12}):
        column = df_raw[attribute_name]
        recorded = column.dropna()
        median_value = recorded.median()
        n_filled = int(column.isna().sum())
        filled = pd.Series([median_value] * n_filled)

        low = np.floor(recorded.min() / bin_width) * bin_width
        high = np.ceil((recorded.max() + 1) / bin_width) * bin_width
        bin_edges = np.arange(low, high + bin_width, bin_width)

        figure, axes = plt.subplots(1, 2, figsize=(10, 4.2), sharey=True)

        axes[0].hist(
            recorded,
            bins=bin_edges,
            color="tab:blue"
        )

        axes[0].set_title(
            f"(a) After removal (n = {len(recorded):,})"
        )

        axes[1].hist(
            [recorded, filled],
            bins=bin_edges,
            stacked=True,
            color=["tab:blue", "tab:orange"],
            label=["Recorded values", "Filled with the median"]
        )

        axes[1].set_title(
            f"(b) After median imputation (n = {len(column):,})"
        )

        axes[1].legend(
            loc="upper right",
            fontsize=10
        )

        left = bin_edges[0] if display_min is None else display_min
        right = bin_edges[-1] if display_max is None else display_max

        for panel in axes:
            panel.set_xlim(left, right)
            panel.set_xlabel(f"{attribute_name} (minutes)")

        axes[0].set_ylabel("Number of flights")

    def plural(count, word):

```

```

        return f"{count} {word}" if count == 1 else f"{count} {word}s"
unit = "minute" if abs(median_value) == 1 else "minutes"

notes = [
    f"{plural(n_filled, 'missing value')} filled\n"
    f"with the median ({median_value:g} {unit})"
]

hidden_high = int((recorded > right).sum())
hidden_low = int((recorded < left).sum())

if hidden_high:
    notes.append(
        f"Not shown: {plural(hidden_high, 'value')} above {right:g}\n"
        f"(maximum {recorded.max():g})"
    )

if hidden_low:
    notes.append(
        f"Not shown: {plural(hidden_low, 'value')} below {left:g}\n"
        f"(minimum {recorded.min():g})"
    )

axes[1].text(
    0.97,
    0.80,
    "\n".join(notes),
    transform=axes[1].transAxes,
    ha="right",
    va="top",
    fontsize=10
)

figure.tight_layout()

path = f"figures/A2_{attribute_name}_removal_vs_median.png"

figure.savefig(
    path,
    dpi=200
)

plt.show()

return path

def save_delay_cause_status_figure():
    """Show blank, recorded zero and recorded positive counts for the three delay causes."""
    with plt.rc_context({"font.size": 12}):

        causes = [
            "AIR_SYSTEM_DELAY",
            "SECURITY_DELAY",
            "WEATHER_DELAY"
        ]

        blank = [
            int(df_raw[c].isna().sum())
            for c in causes
        ]

        zero = [
            int((df_raw[c] == 0).sum())
            for c in causes
        ]

        positive = [
            int((df_raw[c] > 0).sum())
            for c in causes
        ]

        positions = np.arange(len(causes))
        width = 0.27

        figure, axis = plt.subplots(
            figsize=(10, 4.2)
        )

        for offset, values, label, colour in [
            (-width, blank, "Blank (no cause recorded)", "tab:gray"),
            (0, zero, "Recorded zero", "tab:blue"),
            (width, positive, "Recorded positive", "tab:orange")
        ]:
            bars = axis.bar(
                positions + offset,
                values,
                width,
                label=label,
                color=colour
            )

```

```

        axis.bar_label(
            bars,
            fontsize=9,
            padding=2
        )

    axis.set_xticks(positions)
    axis.set_xticklabels(causes)

    axis.set_ylabel("Number of flights")

    axis.set_title(
        "Record status of the three delay-cause attributes"
    )

    axis.set_ylim(
        0,
        max(blank) * 1.3
    )

    axis.legend(
        loc="upper center",
        ncol=3,
        fontsize=10
    )

    figure.tight_layout()

    path = "figures/A2_delay_cause_record_status.png"

    figure.savefig(
        path,
        dpi=200
    )

    plt.show()

    return path
revised_figure_paths = [
    save_imputation_figure(
        "TAXI_OUT",
        bin_width=1,
        display_min=0,
        display_max=60
    ),
    save_imputation_figure(
        "ELAPSED_TIME",
        bin_width=10
    ),
    save_imputation_figure(
        "AIR_TIME",
        bin_width=10
    ),
    save_imputation_figure(
        "TAXI_IN",
        bin_width=1,
        display_min=0,
        display_max=40
    ),
    save_imputation_figure(
        "DEPARTURE_DELAY",
        bin_width=5,
        display_min=-60,
        display_max=180
    ),
    save_imputation_figure(
        "ARRIVAL_DELAY",
        bin_width=5,
        display_min=-60,
        display_max=180
    ),
    save_delay_cause_status_figure(),
]

revised_figure_paths
from PIL import Image

expected_figures_a29 = [
    "figures/A2_TAXI_OUT_removal_vs_median.png",
    "figures/A2_ELAPSED_TIME_removal_vs_median.png",
    "figures/A2_AIR_TIME_removal_vs_median.png",
    "figures/A2_TAXI_IN_removal_vs_median.png",
    "figures/A2_DEPARTURE_DELAY_removal_vs_median.png",
    "figures/A2_ARRIVAL_DELAY_removal_vs_median.png",
    "figures/A2_delay_cause_record_status.png",
]

expected_outputs_a29 = [
    "outputs/A2_missing_impact_median_imputation.csv",
    "outputs/A2_delay_cause_blank_as_zero_demo.csv",
    "outputs/A2_delay_cause_coverage.csv",

```

```

    "outputs/A2_duration_consistency_check.csv",
]

expected_median_means = {
    "TAXI_OUT": (16.15, 16.11),
    "ELAPSED_TIME": (135.39, 135.01),
    "AIR_TIME": (111.76, 111.38),
    "TAXI_IN": (7.49, 7.47),
    "DEPARTURE_DELAY": (10.81, 10.64),
    "ARRIVAL_DELAY": (5.41, 5.21),
}

expected_median_sds = {
    "TAXI_OUT": (8.93, 8.87),
    "ELAPSED_TIME": (72.00, 71.35),
    "AIR_TIME": (69.50, 68.88),
    "TAXI_IN": (6.57, 6.52),
    "DEPARTURE_DELAY": (39.21, 38.96),
    "ARRIVAL_DELAY": (41.60, 41.22),
}

expected_median_skew = {
    "TAXI_OUT": (3.16, 3.19),
    "ELAPSED_TIME": (1.44, 1.46),
    "AIR_TIME": (1.45, 1.48),
    "TAXI_IN": (9.75, 9.82),
    "DEPARTURE_DELAY": (7.20, 7.25),
    "ARRIVAL_DELAY": (6.44, 6.50),
}

expected_blank_means = {
    "AIR_SYSTEM_DELAY": (12.64, 2.40),
    "SECURITY_DELAY": (0.07, 0.01),
    "WEATHER_DELAY": (3.28, 0.62),
}

expected_blank_sds = {
    "AIR_SYSTEM_DELAY": (29.71, 13.87),
    "SECURITY_DELAY": (1.29, 0.56),
    "WEATHER_DELAY": (21.33, 9.39),
}

hidden_expected = {
    "TAXI_OUT": (0, 60, 0, 24),
    "TAXI_IN": (0, 40, 0, 15),
    "DEPARTURE_DELAY": (-60, 180, 0, 32),
    "ARRIVAL_DELAY": (-60, 180, 1, 33),
}

def a29_hidden_counts(attribute_name, left, right):
    recorded = df_raw[attribute_name].dropna()
    return int((recorded < left).sum()), int((recorded > right).sum())

def a29_median_row(attribute_name):
    return median_imputation_impact.set_index("Attribute").loc[attribute_name]

def a29_blank_row(attribute_name):
    return blank_as_zero_demo.set_index("Attribute").loc[attribute_name]

df_raw_fresh_a29 = pd.read_csv(RAW_CSV)
with open(RAW_CSV, "rb") as raw_file_a29:
    raw_hash_a29 = hashlib.sha256(raw_file_a29.read()).hexdigest()

a29_checks = {
    "1. Median imputation table exists": os.path.exists(
        "outputs/A2_missing_impact_median_imputation.csv"
    ),
    "2. Blank-as-zero table exists": os.path.exists(
        "outputs/A2_delay_cause_blank_as_zero_demo.csv"
    ),
    "3. Delay-cause coverage table exists": os.path.exists(
        "outputs/A2_delay_cause_coverage.csv"
    ),
    "4. Duration consistency table exists": os.path.exists(
        "outputs/A2_duration_consistency_check.csv"
    ),
    "5. All seven revised figures exist": all(
        os.path.exists(path) for path in expected_figures_a29
    ),
    "6. Delayed flights equal 827": int(
        delay_cause_coverage.loc[0, "Delayed flights"]
    ) == 827,
    "7. Cause sum below equals 661": int(
        delay_cause_coverage.loc[0, "Cause sum below arrival delay"]
    ) == 661,
    "8. Cause sum equal equals 166": int(
        delay_cause_coverage.loc[0, "Cause sum equal to arrival delay"]
    ) == 166,
    "9. Cause sum above equals 0": int(

```

```

    delay_cause_coverage.loc[0, "Cause sum above arrival delay"]
) == 0,
"10. All three causes zero equals 412": int(
    delay_cause_coverage.loc[0, "All three causes zero"]
) == 412,
"11. Share explained equals 25.83": float(
    delay_cause_coverage.loc[0, "Share of delay minutes explained percent"]
) == 25.83,
"12. Duration complete rows equal 4265": int(
    duration_consistency.loc[0, "Rows with all four recorded"]
) == 4265,
"13. Duration identity holds for 4265 rows": int(
    duration_consistency.loc[0, "Rows where the identity holds"]
) == 4265,
"14. Mean fall percent about 80.98 for all three": bool(
    ((blank_as_zero_demo["Mean fall percent"] - 80.98).abs() < 0.01).all()
),
"15. df_raw equals fresh CSV read": df_raw.equals(df_raw_fresh_a29),
"16. Raw CSV hash unchanged": raw_hash_a29 == (
    "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de"
),
"17. df_raw shape still 4349 by 25": df_raw.shape == (4349, 25),
}

for attribute, (mean_rem, mean_imp) in expected_median_means.items():
    row = a29_median_row(attribute)
    a29_checks[f"18. {attribute} mean removal"] = float(row["Mean removal"]) == mean_rem
    a29_checks[f"19. {attribute} mean imputed"] = float(row["Mean imputed"]) == mean_imp

for attribute, (sd_rem, sd_imp) in expected_median_sds.items():
    row = a29_median_row(attribute)
    a29_checks[f"20. {attribute} SD removal"] = float(row["SD removal"]) == sd_rem
    a29_checks[f"21. {attribute} SD imputed"] = float(row["SD imputed"]) == sd_imp

for attribute, (skew_rem, skew_imp) in expected_median_skew.items():
    row = a29_median_row(attribute)
    a29_checks[f"22. {attribute} skew removal"] = float(row["Skewness removal"]) == skew_rem
    a29_checks[f"23. {attribute} skew imputed"] = float(row["Skewness imputed"]) == skew_imp

for attribute, (mean_miss, mean_zero) in expected_blank_means.items():
    row = a29_blank_row(attribute)
    a29_checks[f"24. {attribute} mean blanks missing"] = (
        float(row["Mean blanks missing"]) == mean_miss
    )
    a29_checks[f"25. {attribute} mean blanks as zero"] = (
        float(row["Mean blanks as zero"]) == mean_zero
    )

for attribute, (sd_miss, sd_zero) in expected_blank_sds.items():
    row = a29_blank_row(attribute)
    a29_checks[f"26. {attribute} SD blanks missing"] = (
        float(row["SD blanks missing"]) == sd_miss
    )
    a29_checks[f"27. {attribute} SD blanks as zero"] = (
        float(row["SD blanks as zero"]) == sd_zero
    )

for attribute, (left, right, low_count, high_count) in hidden_expected.items():
    low, high = a29_hidden_counts(attribute, left, right)
    a29_checks[f"28. {attribute} hidden below"] = low == low_count
    a29_checks[f"29. {attribute} hidden above"] = high == high_count

for path in expected_figures_a29:
    with Image.open(path) as image:
        dpi = image.info.get("dpi", (None, None))
        name = os.path.basename(path)
        a29_checks[f"30. {name} readable"] = image.size[0] > 0 and image.size[1] > 0
        a29_checks[f"31. {name} near 200 dpi"] = (
            dpi[0] is not None and abs(dpi[0] - 200) < 1
        )

a29_checks["32. Old TAXI_OUT comparison figure still exists"] = os.path.exists(
    "figures/A2_TAXI_OUT_missing_comparison.png"
)
a29_checks["33. Old ARRIVAL_DELAY comparison figure still exists"] = os.path.exists(
    "figures/A2_ARRIVAL_DELAY_missing_comparison.png"
)

a29_check_table = pd.DataFrame({
    "Check": list(a29_checks.keys()),
    "Passed": list(a29_checks.values()),
})
print("A2.9 checks passed:", int(a29_check_table["Passed"].sum()),
      "of", len(a29_check_table))
assert a29_check_table["Passed"].all(), a29_check_table.loc[~a29_check_table["Passed"]]
a29_check_table

A2.10 Clock Times and Date
import math

clock_attributes = [

```

```

    "SCHEDULED_DEPARTURE",
    "DEPARTURE_TIME",
    "WHEELS_OFF",
    "WHEELS_ON",
    "SCHEDULED_ARRIVAL",
    "ARRIVAL_TIME",
]

def round_half_up(value):
    return math.floor(float(value) + 0.5)

def minutes_to_hhmm(value):
    rounded = math.floor(float(value) + 0.5)
    hours = rounded // 60
    minutes = rounded % 60
    return f"{hours:02d}:{minutes:02d}"

def hhmm_to_minutes(series):
    converted = series.astype(float).copy()
    valid = converted.notna()
    hhmm = converted.loc[valid]
    converted.loc[valid] = (hhmm // 100) * 60 + (hhmm % 100)
    return converted

hhmm_validity_rows = []

for attribute_name in clock_attributes:
    recorded = df_raw[attribute_name].dropna()
    hours = (recorded // 100).astype(int)
    minutes = (recorded % 100).astype(int)
    invalid_hours = int(((hours < 0) | (hours > 23)).sum())
    invalid_minutes = int(((minutes < 0) | (minutes > 59)).sum())
    hhmm_validity_rows.append({
        "Attribute": attribute_name,
        "Recorded N": len(recorded),
        "Invalid hours": invalid_hours,
        "Invalid minutes": invalid_minutes,
    })

hhmm_validity = pd.DataFrame(hhmm_validity_rows)
assert (hhmm_validity["Invalid hours"] == 0).all()
assert (hhmm_validity["Invalid minutes"] == 0).all()
hhmm_validity

clock_minutes = {
    attribute_name: hhmm_to_minutes(df_raw[attribute_name])
    for attribute_name in clock_attributes
}

for attribute_name, series in clock_minutes.items():
    print(
        attribute_name,
        "valid N =", int(series.notna().sum()),
        "missing =", int(series.isna().sum()),
        "minute range =", float(series.min()), "to", float(series.max()),
    )

clock_summary_rows = []

for attribute_name in clock_attributes:
    series = clock_minutes[attribute_name]
    recorded = series.dropna()
    missing = int(series.isna().sum())
    q1 = recorded.quantile(0.25)
    q3 = recorded.quantile(0.75)
    clock_summary_rows.append({
        "Attribute": attribute_name,
        "Total Rows": len(series),
        "Valid N": int(recorded.shape[0]),
        "Missing": missing,
        "Missing Percentage": round(100 * missing / len(series), 2),
        "Mean Time": minutes_to_hhmm(recorded.mean()),
        "Median Time": minutes_to_hhmm(recorded.median()),
        "Minimum Time": minutes_to_hhmm(recorded.min()),
        "Maximum Time": minutes_to_hhmm(recorded.max()),
        "Range (minutes)": float(recorded.max() - recorded.min()),
        "Variance (minutes squared)": round(recorded.var(ddof=1), 2),
        "Standard Deviation (minutes)": round(recorded.std(ddof=1), 2),
        "Q1 Time": minutes_to_hhmm(q1),
        "Q3 Time": minutes_to_hhmm(q3),
        "IQR (minutes)": float(q3 - q1),
        "Skewness": round(recorded.skew(), 2),
    })

clock_time_summary = pd.DataFrame(clock_summary_rows)
clock_time_summary.to_csv("outputs/A2_clock_time_summary.csv", index=False)
clock_time_summary
clock_stats_precise = {}

for attribute_name in clock_attributes:

```

```

recorded = clock_minutes[attribute_name].dropna()
clock_stats_precise[attribute_name] = {
    "valid_n": int(recorded.shape[0]),
    "missing": int(clock_minutes[attribute_name].isna().sum()),
    "mean": float(recorded.mean()),
    "median": float(recorded.median()),
    "minimum": float(recorded.min()),
    "maximum": float(recorded.max()),
    "range": float(recorded.max() - recorded.min()),
    "variance": float(recorded.var(ddof=1)),
    "std": float(recorded.std(ddof=1)),
    "q1": float(recorded.quantile(0.25)),
    "q3": float(recorded.quantile(0.75)),
    "iqr": float(recorded.quantile(0.75) - recorded.quantile(0.25)),
    "skewness": float(recorded.skew()),
}

print("WHEELS_ON median minutes:", clock_stats_precise["WHEELS_ON"]["median"])
print("WHEELS_ON median display:", minutes_to_hhmm(clock_stats_precise["WHEELS_ON"]["median"]))
clock_bin_edges = np.arange(0, 1441, 60)
clock_titles = {
    "SCHEDULED_DEPARTURE": "Scheduled departure",
    "DEPARTURE_TIME": "Actual departure",
    "WHEELS_OFF": "Wheels off",
    "WHEELS_ON": "Wheels on",
    "SCHEDULED_ARRIVAL": "Scheduled arrival",
    "ARRIVAL_TIME": "Actual arrival",
}
tick_positions = [0, 360, 720, 1080, 1440]
tick_labels = [minutes_to_hhmm(value) for value in tick_positions]

hourly_count_data = {"Hour": [f"{hour:02d}:00" for hour in range(24)]}

with plt.rc_context({"font.size": 12}):
    figure, axes = plt.subplots(3, 2, figsize=(11, 8), sharey=False)
    for axis, attribute_name in zip(axes.flat, clock_attributes):
        values = clock_minutes[attribute_name].dropna()
        counts, _ = np.histogram(values, bins=clock_bin_edges)
        hourly_count_data[attribute_name] = counts
        axis.hist(values, bins=clock_bin_edges, edgecolor="black", linewidth=0.4)
        axis.set_xlim(0, 1440)
        axis.set_xticks(tick_positions)
        axis.set_xticklabels(tick_labels)
        axis.set_title(clock_titles[attribute_name])
    for row in range(3):
        axes[row, 0].set_ylabel("Number of flights")
    figure.tight_layout()
    figure.savefig("figures/A2_clock_time_distributions.png", dpi=200)
    plt.show()

clock_hourly_counts = pd.DataFrame(hourly_count_data)
clock_hourly_counts.to_csv("outputs/A2_clock_hourly_counts.csv", index=False)
clock_hourly_counts
missing_clock_attributes = [
    "DEPARTURE_TIME",
    "WHEELS_OFF",
    "WHEELS_ON",
    "ARRIVAL_TIME",
]
cancelled_flag = df_raw["CANCELLED"] == 1
diverted_flag = df_raw["DIVERTED"] == 1
status_rows = []

for attribute_name in missing_clock_attributes:
    missing = df_raw[attribute_name].isna()
    recorded = ~missing
    status_rows.append({
        "Attribute": attribute_name,
        "Missing": int(missing.sum()),
        "Missing and cancelled": int((missing & cancelled_flag).sum()),
        "Missing and diverted": int((missing & diverted_flag).sum()),
        "Missing and neither": int((missing & ~cancelled_flag & ~diverted_flag).sum()),
        "Cancelled with recorded value": int((recorded & cancelled_flag).sum()),
        "Diverted with recorded value": int((recorded & diverted_flag).sum()),
    })

clock_missing_status = pd.DataFrame(status_rows)
clock_missing_status.to_csv("outputs/A2_clock_missing_status_check.csv", index=False)
clock_missing_status
impact_rows = []
clock_imputed_minutes = {}
clock_median_fill = {}

for attribute_name in missing_clock_attributes:
    converted = clock_minutes[attribute_name]
    removal = converted.dropna()
    median_value = float(removal.median())
    imputed = converted.fillna(median_value)
    clock_imputed_minutes[attribute_name] = imputed
    clock_median_fill[attribute_name] = median_value
    sd_removal = float(removal.std(ddof=1))

```

```

sd_imputed = float(imputed.std(ddof=1))
impact_rows.append({
    "Attribute": attribute_name,
    "Missing": int(converted.isna().sum()),
    "N removal": int(removal.shape[0]),
    "N imputed": int(imputed.notna().sum()),
    "Median fill time": minutes_to_hhmm(median_value),
    "Mean removal": minutes_to_hhmm(removal.mean()),
    "Mean imputed": minutes_to_hhmm(imputed.mean()),
    "SD removal (minutes)": round(sd_removal, 2),
    "SD imputed (minutes)": round(sd_imputed, 2),
    "Variance removal (minutes squared)": round(float(removal.var(ddof=1)), 2),
    "Variance imputed (minutes squared)": round(float(imputed.var(ddof=1)), 2),
    "Q1 removal": minutes_to_hhmm(removal.quantile(0.25)),
    "Q1 imputed": minutes_to_hhmm(imputed.quantile(0.25)),
    "Q3 removal": minutes_to_hhmm(removal.quantile(0.75)),
    "Q3 imputed": minutes_to_hhmm(imputed.quantile(0.75)),
    "Skewness removal": round(float(removal.skew()), 2),
    "Skewness imputed": round(float(imputed.skew()), 2),
    "SD change percent": round(100 * (sd_imputed - sd_removal) / sd_removal, 2),
})

clock_missing_impact = pd.DataFrame(impact_rows)
clock_missing_impact.to_csv("outputs/A2_clock_missing_impact_median.csv", index=False)
clock_missing_impact
clock_impact_precise = {}

for attribute_name in missing_clock_attributes:
    converted = clock_minutes[attribute_name]
    removal = converted.dropna()
    median_value = float(removal.median())
    imputed = converted.fillna(median_value)
    sd_removal = float(removal.std(ddof=1))
    sd_imputed = float(imputed.std(ddof=1))
    clock_impact_precise[attribute_name] = {
        "missing": int(converted.isna().sum()),
        "n_removal": int(removal.shape[0]),
        "n_imputed": int(imputed.notna().sum()),
        "median": median_value,
        "mean_removal": float(removal.mean()),
        "mean_imputed": float(imputed.mean()),
        "sd_removal": sd_removal,
        "sd_imputed": sd_imputed,
        "var_removal": float(removal.var(ddof=1)),
        "var_imputed": float(imputed.var(ddof=1)),
        "q1_removal": float(removal.quantile(0.25)),
        "q1_imputed": float(imputed.quantile(0.25)),
        "q3_removal": float(removal.quantile(0.75)),
        "q3_imputed": float(imputed.quantile(0.75)),
        "skew_removal": float(removal.skew()),
        "skew_imputed": float(imputed.skew()),
        "sd_change_percent": 100 * (sd_imputed - sd_removal) / sd_removal,
    }

impact_titles = {
    "DEPARTURE_TIME": "Departure time",
    "WHEELS_OFF": "Wheels off",
    "WHEELS_ON": "Wheels on",
    "ARRIVAL_TIME": "Arrival time",
}

orange_added_counts = {}

with plt.rc_context({"font.size": 12}):
    figure, axes = plt.subplots(2, 2, figsize=(11, 8))
    for axis, attribute_name in zip(axes.flat, missing_clock_attributes):
        recorded = clock_minutes[attribute_name].dropna()
        fill_value = clock_median_fill[attribute_name]
        missing_n = int(clock_minutes[attribute_name].isna().sum())
        recorded_counts, _ = np.histogram(recorded, bins=clock_bin_edges)
        filled_only = np.full(missing_n, fill_value)
        filled_counts, _ = np.histogram(filled_only, bins=clock_bin_edges)
        orange_added_counts[attribute_name] = filled_counts
        axis.bar(
            clock_bin_edges[:-1], recorded_counts, width=60, align="edge",
            color="steelblue", edgecolor="black", linewidth=0.3, label="Recorded values",
        )
        axis.bar(
            clock_bin_edges[:-1], filled_counts, width=60, align="edge",
            bottom=recorded_counts, color="darkorange", edgecolor="black",
            linewidth=0.3, label="Filled with median",
        )
        axis.set_xlim(0, 1440)
        axis.set_xticks(tick_positions)
        axis.set_xticklabels(tick_labels)
        axis.set_title(impact_titles[attribute_name])
        axis.legend(fontsize=9)
    axes[0, 0].set_ylabel("Number of flights")
    axes[1, 0].set_ylabel("Number of flights")
    figure.tight_layout()
    figure.savefig("figures/A2_clock_missing_impact.png", dpi=200)
    plt.show()

date_raw_before = df_raw["DATE"].copy()

```

```

date_parsed = pd.to_datetime(df_raw["DATE"], format="%Y-%m-%d", errors="raise")
date_parse_failures = 0 # errors='raise' would have stopped on any failure

print("DATE N:", len(date_parsed))
print("DATE missing:", int(date_parsed.isna().sum()))
print("DATE parse failures:", date_parse_failures)
print("Raw DATE unchanged:", df_raw["DATE"].equals(date_raw_before))
print("Earliest:", date_parsed.min().date())
print("Latest:", date_parsed.max().date())
origin_date = date_parsed.min()
date_offset_days = (date_parsed - origin_date).dt.days.astype(float)

mean_offset = float(date_offset_days.mean())
mean_offset_rounded = round_half_up(mean_offset)
mean_date = origin_date + pd.Timedelta(days=mean_offset_rounded)
median_offset = float(date_offset_days.median())
median_date = origin_date + pd.Timedelta(days=int(median_offset))
q1_offset = float(date_offset_days.quantile(0.25))
q3_offset = float(date_offset_days.quantile(0.75))
q1_date = origin_date + pd.Timedelta(days=int(q1_offset))
q3_date = origin_date + pd.Timedelta(days=int(q3_offset))

date_summary = pd.DataFrame([
    "N": int(date_offset_days.shape[0]),
    "Missing": int(date_parsed.isna().sum()),
    "Earliest date": origin_date.strftime("%d %B %Y"),
    "Latest date": date_parsed.max().strftime("%d %B %Y"),
    "Span in days": float(date_offset_days.max() - date_offset_days.min()),
    "Mean date": mean_date.strftime("%d %B %Y"),
    "Median date": median_date.strftime("%d %B %Y"),
    "Variance (days squared)": round(float(date_offset_days.var(ddof=1)), 2),
    "Standard deviation (days)": round(float(date_offset_days.std(ddof=1)), 2),
    "Q1 date": q1_date.strftime("%d %B %Y"),
    "Q3 date": q3_date.strftime("%d %B %Y"),
    "IQR (days)": float(q3_offset - q1_offset),
    "Skewness": round(float(date_offset_days.skew()), 2),
])
date_summary.to_csv("outputs/A2_DATE_summary.csv", index=False)
date_summary

date_stats_precise = {
    "n": int(date_offset_days.shape[0]),
    "missing": int(date_parsed.isna().sum()),
    "earliest": origin_date.strftime("%Y-%m-%d"),
    "latest": date_parsed.max().strftime("%Y-%m-%d"),
    "span": float(date_offset_days.max() - date_offset_days.min()),
    "mean_offset": mean_offset,
    "mean_date": mean_date.strftime("%d %B %Y"),
    "median_date": median_date.strftime("%d %B %Y"),
    "variance": float(date_offset_days.var(ddof=1)),
    "std": float(date_offset_days.std(ddof=1)),
    "q1_date": q1_date.strftime("%d %B %Y"),
    "q3_date": q3_date.strftime("%d %B %Y"),
    "iqr": float(q3_offset - q1_offset),
    "skewness": float(date_offset_days.skew()),
}
date_stats_precise

month_names = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December",
]
month_labels_short = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
]

monthly_counts = (
    date_parsed.dt.month
    .value_counts()
    .reindex(range(1, 13), fill_value=0)
    .sort_index()
)

date_month_counts = pd.DataFrame({
    "Month Number": list(range(1, 13)),
    "Month": month_names,
    "Flight Count": monthly_counts.astype(int).tolist(),
})
date_month_counts.to_csv("outputs/A2_DATE_month_counts.csv", index=False)
date_month_counts

with plt.rc_context({"font.size": 12}):
    figure, axis = plt.subplots(figsize=(10, 4.5))
    bars = axis.bar(month_labels_short, date_month_counts["Flight Count"].tolist())
    axis.bar_label(bars, padding=2)
    axis.set_xlabel("Month")
    axis.set_ylabel("Number of flights")
    axis.set_title("Monthly distribution of flight records")
    figure.tight_layout()
    figure.savefig("figures/A2_DATE_monthly_distribution.png", dpi=200)
    plt.show()

```

```

print("October count:", int(date_month_counts.loc[
    date_month_counts["Month"] == "October", "Flight Count"
].iloc[0]))
print("Monthly total:", int(date_month_counts["Flight Count"].sum()))
print("October on x axis:", "Oct" in month_labels_short)
overview_rows = []
for attribute_name in clock_attributes:
    stats = clock_stats_precise[attribute_name]
    overview_rows.append({
        "Attribute": attribute_name,
        "Type": "Interval (clock time)",
        "Valid N": stats["valid_n"],
        "Missing": stats["missing"],
        "Main Centre": (
            f"mean {minutes_to_hhmm(stats['mean'])}; "
            f"median {minutes_to_hhmm(stats['median'])}"
        ),
        "Main Spread": (
            f"SD {round(stats['std'], 2)} min; "
            f"IQR {stats['iqr']} min"
        ),
        "Key Finding": (
            "HHMM converted to minutes after midnight; "
            + (
                "attribute-wise removal used for missing values"
                if stats["missing"] > 0
                else "no missing values"
            )
        ),
    })
overview_rows.append({
    "Attribute": "DATE",
    "Type": "Interval (calendar date)",
    "Valid N": date_stats_precise["n"],
    "Missing": date_stats_precise["missing"],
    "Main Centre": (
        f"mean {date_stats_precise['mean_date']}; "
        f"median {date_stats_precise['median_date']}"
    ),
    "Main Spread": (
        f"SD {round(date_stats_precise['std'], 2)} days; "
        f"IQR {date_stats_precise['iqr']} days"
    ),
    "Key Finding": "October has zero records in this dataset",
})

clock_date_overview = pd.DataFrame(overview_rows)
clock_date_overview.to_csv("outputs/A2_clock_date_overview.csv", index=False)
clock_date_overview

expected_clock_stats = {
    "SCHEDULED_DEPARTURE": {
        "valid_n": 4349, "missing": 0,
        "mean": 810.3715796734882, "median": 813.0,
        "minimum": 30, "maximum": 1439, "range": 1409,
        "variance": 84719.6130467451, "std": 291.066337879778,
        "q1": 550, "q3": 1055, "iqr": 505, "skewness": 0.031043371660576784,
        "mean_disp": "13:30", "median_disp": "13:33",
        "min_disp": "00:30", "max_disp": "23:59",
        "q1_disp": "09:10", "q3_disp": "17:35",
    },
    "DEPARTURE_TIME": {
        "valid_n": 4288, "missing": 61,
        "mean": 816.8339552238806, "median": 818,
        "minimum": 1, "maximum": 1435, "range": 1434,
        "variance": 88596.06199582912, "std": 297.65090625736235,
        "q1": 561, "q3": 1064, "iqr": 503, "skewness": 0.007692976228646388,
        "mean_disp": "13:37", "median_disp": "13:38",
        "min_disp": "00:01", "max_disp": "23:55",
        "q1_disp": "09:21", "q3_disp": "17:44",
    },
    "WHEELS_OFF": {
        "valid_n": 4284, "missing": 65,
        "mean": 831.6514939309056, "median": 832,
        "minimum": 2, "maximum": 1438, "range": 1436,
        "variance": 89076.71414439386, "std": 298.45722330745133,
        "q1": 577, "q3": 1080.25, "iqr": 503.25, "skewness": -0.008571646331709424,
        "mean_disp": "13:52", "median_disp": "13:52",
        "min_disp": "00:02", "max_disp": "23:58",
        "q1_disp": "09:37", "q3_disp": "18:00",
    },
    "WHEELS_ON": {
        "valid_n": 4282, "missing": 67,
        "mean": 901.4546940681925, "median": 914.5,
        "minimum": 1, "maximum": 1439, "range": 1438,
        "variance": 98955.9700328543, "std": 314.572678458976,
        "q1": 659.25, "q3": 1159, "iqr": 499.75, "skewness": -0.3481107531619939,
        "mean_disp": "15:01", "median_disp": "15:15",
        "min_disp": "00:01", "max_disp": "23:59",
        "q1_disp": "10:59", "q3_disp": "19:19",
    },
}

```

```

"SCHEDULED_ARRIVAL": {
    "valid_n": 4349, "missing": 0,
    "mean": 917.0218441020925, "median": 935,
    "minimum": 2, "maximum": 1439, "range": 1437,
    "variance": 91872.9036165617, "std": 303.10543316899106,
    "q1": 674, "q3": 1164, "iqr": 490, "skewness": -0.2754525451351922,
    "mean_disp": "15:17", "median_disp": "15:35",
    "min_disp": "00:02", "max_disp": "23:59",
    "q1_disp": "11:14", "q3_disp": "19:24",
},
"ARRIVAL_TIME": {
    "valid_n": 4282, "missing": 67,
    "mean": 904.5712283979449, "median": 920,
    "minimum": 1, "maximum": 1439, "range": 1438,
    "variance": 100683.20877723397, "std": 317.30617513252713,
    "q1": 664, "q3": 1163, "iqr": 499, "skewness": -0.383300141379763,
    "mean_disp": "15:05", "median_disp": "15:20",
    "min_disp": "00:01", "max_disp": "23:59",
    "q1_disp": "11:04", "q3_disp": "19:23",
},
},
expected_hourly = {
    "SCHEDULED_DEPARTURE": [9, 1, 0, 0, 0, 101, 309, 294, 296, 256, 260, 232,
        262, 275, 250, 281, 252, 303, 242, 273, 190, 155, 83, 25],
    "DEPARTURE_TIME": [15, 4, 1, 0, 5, 160, 269, 263, 273, 259, 240, 242,
        249, 266, 242, 265, 257, 276, 240, 248, 214, 173, 95, 32],
    "WHEELS_OFF": [16, 7, 1, 0, 0, 75, 263, 278, 273, 252, 265, 244,
        238, 255, 263, 255, 259, 263, 258, 239, 236, 165, 138, 41],
    "WHEELS_ON": [72, 19, 9, 0, 2, 35, 66, 140, 216, 259, 253, 231,
        252, 256, 262, 266, 261, 268, 262, 262, 243, 244, 216, 188],
    "SCHEDULED_ARRIVAL": [52, 8, 1, 0, 1, 16, 60, 140, 201, 274, 269, 243,
        254, 252, 260, 264, 305, 256, 286, 253, 254, 266, 230, 204],
    "ARRIVAL_TIME": [82, 21, 10, 0, 2, 25, 62, 142, 199, 252, 262, 236,
        246, 258, 256, 268, 260, 259, 267, 256, 251, 242, 219, 207],
}

expected_impact = {
    "DEPARTURE_TIME": {
        "missing": 61, "n_removal": 4288, "n_imputed": 4349, "median": 818,
        "mean_removal": 816.8339552238806, "mean_imputed": 816.8503104161877,
        "sd_removal": 297.65090625736235, "sd_imputed": 295.5556255121322,
        "var_removal": 88596.06199582912, "var_imputed": 87353.12777186774,
        "q1_removal": 561, "q1_imputed": 564, "q3_removal": 1064, "q3_imputed": 1061,
        "skew_removal": 0.007692976228646388, "skew_imputed": 0.007581374794068063,
        "sd_change_percent": -0.70,
    },
    "WHEELS_OFF": {
        "missing": 65, "n_removal": 4284, "n_imputed": 4349, "median": 832,
        "mean_removal": 831.6514939309056, "mean_imputed": 831.6567026902736,
        "sd_removal": 298.45722330745133, "sd_imputed": 296.2179473331147,
        "var_removal": 89076.71414439386, "var_imputed": 87745.07232224393,
        "q1_removal": 577, "q1_imputed": 581, "q3_removal": 1080.25, "q3_imputed": 1078,
        "skew_removal": -0.008571646331709424, "skew_imputed": -0.008689160585899164,
        "sd_change_percent": -0.75,
    },
    "WHEELS_ON": {
        "missing": 67, "n_removal": 4282, "n_imputed": 4349, "median": 914.5,
        "mean_removal": 901.4546940681925, "mean_imputed": 901.6556679696482,
        "sd_removal": 314.572678458976, "sd_imputed": 312.14371929910834,
        "var_removal": 98955.9700328543, "var_imputed": 97433.70149788052,
        "q1_removal": 659.25, "q1_imputed": 663, "q3_removal": 1159, "q3_imputed": 1153,
        "skew_removal": -0.3481107531619939, "skew_imputed": -0.3527390843981033,
        "sd_change_percent": -0.77,
    },
    "ARRIVAL_TIME": {
        "missing": 67, "n_removal": 4282, "n_imputed": 4349, "median": 920,
        "mean_removal": 904.5712283979449, "mean_imputed": 904.8089215911704,
        "sd_removal": 317.30617513252713, "sd_imputed": 314.8576729810063,
        "var_removal": 100683.20877723397, "var_imputed": 99135.35423501433,
        "q1_removal": 664, "q1_imputed": 667, "q3_removal": 1163, "q3_imputed": 1160,
        "skew_removal": -0.383300141379763, "skew_imputed": -0.38852803180440015,
        "sd_change_percent": -0.77,
    },
}

expected_month_counts = [373, 355, 391, 424, 398, 403, 455, 411, 398, 0, 377, 364]
def close_enough(actual, expected, tol=1e-9):
    return abs(float(actual) - float(expected)) <= tol

def clock_stats_match(attribute_name):
    got = clock_stats_precise[attribute_name]
    exp = expected_clock_stats[attribute_name]
    numeric_keys = [
        "valid_n", "missing", "mean", "median", "minimum", "maximum", "range",
        "variance", "std", "q1", "q3", "iqr", "skewness",
    ]
    if not all(close_enough(got[key], exp[key]) for key in numeric_keys):
        return False
    display_pairs = [

```

```

        ("mean", "mean_disp"), ("median", "median_disp"),
        ("minimum", "min_disp"), ("maximum", "max_disp"),
        ("q1", "q1_disp"), ("q3", "q3_disp"),
    ]
    return all(
        minutes_to_hhmm(got[src]) == exp[disp]
        for src, disp in display_pairs
    )

def impact_match(attribute_name):
    got = clock_impact_precise[attribute_name]
    exp = expected_impact[attribute_name]
    keys = [
        "missing", "n_removal", "n_imputed", "median",
        "mean_removal", "mean_imputed", "sd_removal", "sd_imputed",
        "var_removal", "var_imputed", "q1_removal", "q1_imputed",
        "q3_removal", "q3_imputed", "skew_removal", "skew_imputed",
    ]
    if not all(close_enough(got[key], exp[key]) for key in keys):
        return False
    return abs(got["sd_change_percent"] - exp["sd_change_percent"]) < 0.005

df_raw_fresh_a210 = pd.read_csv(RAW_CSV)
with open(RAW_CSV, "rb") as raw_file_a210:
    raw_hash_a210 = hashlib.sha256(raw_file_a210.read()).hexdigest()

a210_figures = [
    "figures/A2_clock_time_distributions.png",
    "figures/A2_clock_missing_impact.png",
    "figures/A2_DATE_monthly_distribution.png",
]
a210_outputs = [
    "outputs/A2_clock_time_summary.csv",
    "outputs/A2_clock_hourly_counts.csv",
    "outputs/A2_clock_missing_status_check.csv",
    "outputs/A2_clock_missing_impact_median.csv",
    "outputs/A2_DATE_summary.csv",
    "outputs/A2_DATE_month_counts.csv",
    "outputs/A2_clock_date_overview.csv",
]

def invalid_hour_count(attribute_name):
    recorded = df_raw[attribute_name].dropna()
    hours = recorded // 100
    return int(((hours < 0) | (hours > 23)).sum())

def invalid_minute_count(attribute_name):
    recorded = df_raw[attribute_name].dropna()
    minutes = recorded % 100
    return int(((minutes < 0) | (minutes > 59)).sum())

a210_checks = {
    "1. All seven attributes exist": all(
        name in df_raw.columns
        for name in clock_attributes + ["DATE"]
    ),
    "2. df_raw shape": df_raw.shape == (4349, 25),
    "3. Raw CSV SHA-256": raw_hash_a210 == (
        "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de"
    ),
    "4. SCHEDULED_DEPARTURE missing": int(df_raw["SCHEDULED_DEPARTURE"].isna().sum()) == 0,
    "5. DEPARTURE_TIME missing": int(df_raw["DEPARTURE_TIME"].isna().sum()) == 61,
    "6. WHEELS_OFF missing": int(df_raw["WHEELS_OFF"].isna().sum()) == 65,
    "7. WHEELS_ON missing": int(df_raw["WHEELS_ON"].isna().sum()) == 67,
    "8. SCHEDULED_ARRIVAL missing": int(df_raw["SCHEDULED_ARRIVAL"].isna().sum()) == 0,
    "9. ARRIVAL_TIME missing": int(df_raw["ARRIVAL_TIME"].isna().sum()) == 67,
    "10. DATE missing": int(df_raw["DATE"].isna().sum()) == 0,
    "11. Invalid SCHEDULED_DEPARTURE hours": invalid_hour_count("SCHEDULED_DEPARTURE") == 0,
    "12. Invalid SCHEDULED_DEPARTURE minutes": invalid_minute_count("SCHEDULED_DEPARTURE") == 0,
    "13. Invalid DEPARTURE_TIME hours": invalid_hour_count("DEPARTURE_TIME") == 0,
    "14. Invalid DEPARTURE_TIME minutes": invalid_minute_count("DEPARTURE_TIME") == 0,
    "15. Invalid WHEELS_OFF hours": invalid_hour_count("WHEELS_OFF") == 0,
    "16. Invalid WHEELS_OFF minutes": invalid_minute_count("WHEELS_OFF") == 0,
    "17. Invalid WHEELS_ON hours": invalid_hour_count("WHEELS_ON") == 0,
    "18. Invalid WHEELS_ON minutes": invalid_minute_count("WHEELS_ON") == 0,
    "19. Invalid SCHEDULED_ARRIVAL hours": invalid_hour_count("SCHEDULED_ARRIVAL") == 0,
    "20. Invalid SCHEDULED_ARRIVAL minutes": invalid_minute_count("SCHEDULED_ARRIVAL") == 0,
    "21. Invalid ARRIVAL_TIME hours": invalid_hour_count("ARRIVAL_TIME") == 0,
    "22. Invalid ARRIVAL_TIME minutes": invalid_minute_count("ARRIVAL_TIME") == 0,
}

a210_checks["23. WHEELS_ON median raw minutes"] = close_enough(
    clock_stats_precise["WHEELS_ON"]["median"], 914.5
)

a210_checks["24. WHEELS_ON median formatted with half-up"] = (
    minutes_to_hhmm(clock_stats_precise["WHEELS_ON"]["median"]) == "15:15"
)

a210_checks["25. 1440-minute axis formatting"] = minutes_to_hhmm(1440) == "24:00"
a210_checks["26. Clock summary values"] = all(

```

```

    clock_stats_match(name) for name in clock_attributes
)
a210_checks["27. Hourly count arrays"] = all(
    clock_hourly_counts[name].tolist() == expected_hourly[name]
    for name in clock_attributes
)
a210_checks["28. Hourly count sum SCHEDULED_DEPARTURE"] = int(
    clock_hourly_counts["SCHEDULED_DEPARTURE"].sum()
) == 4349
a210_checks["29. DEPARTURE_TIME hourly sum"] = int(
    clock_hourly_counts["DEPARTURE_TIME"].sum()
) == 4288
a210_checks["30. WHEELS_OFF hourly sum"] = int(
    clock_hourly_counts["WHEELS_OFF"].sum()
) == 4284
a210_checks["31. WHEELS_ON hourly sum"] = int(
    clock_hourly_counts["WHEELS_ON"].sum()
) == 4282
a210_checks["32. SCHEDULED_ARRIVAL hourly sum"] = int(
    clock_hourly_counts["SCHEDULED_ARRIVAL"].sum()
) == 4349
a210_checks["33. ARRIVAL_TIME hourly sum"] = int(
    clock_hourly_counts["ARRIVAL_TIME"].sum()
) == 4282

status_lookup = clock_missing_status.set_index("Attribute")
a210_checks["34. DEPARTURE_TIME missing-status split"] = (
    int(status_lookup.loc["DEPARTURE_TIME", "Missing and cancelled"]) == 61
    and int(status_lookup.loc["DEPARTURE_TIME", "Missing and diverted"]) == 0
    and int(status_lookup.loc["DEPARTURE_TIME", "Missing and neither"]) == 0
)
a210_checks["35. WHEELS_OFF missing-status split"] = (
    int(status_lookup.loc["WHEELS_OFF", "Missing and cancelled"]) == 65
    and int(status_lookup.loc["WHEELS_OFF", "Missing and diverted"]) == 0
    and int(status_lookup.loc["WHEELS_OFF", "Missing and neither"]) == 0
)
a210_checks["36. WHEELS_ON missing-status split"] = (
    int(status_lookup.loc["WHEELS_ON", "Missing and cancelled"]) == 66
    and int(status_lookup.loc["WHEELS_ON", "Missing and diverted"]) == 1
    and int(status_lookup.loc["WHEELS_ON", "Missing and neither"]) == 0
)
a210_checks["37. ARRIVAL_TIME missing-status split"] = (
    int(status_lookup.loc["ARRIVAL_TIME", "Missing and cancelled"]) == 66
    and int(status_lookup.loc["ARRIVAL_TIME", "Missing and diverted"]) == 1
    and int(status_lookup.loc["ARRIVAL_TIME", "Missing and neither"]) == 0
)
a210_checks["38. DEPARTURE_TIME median-imputation N"] = (
    clock_impact_precise["DEPARTURE_TIME"]["n_imputed"] == 4349
)
a210_checks["39. WHEELS_OFF median-imputation N"] = (
    clock_impact_precise["WHEELS_OFF"]["n_imputed"] == 4349
)
a210_checks["40. WHEELS_ON median-imputation N"] = (
    clock_impact_precise["WHEELS_ON"]["n_imputed"] == 4349
)
a210_checks["41. ARRIVAL_TIME median-imputation N"] = (
    clock_impact_precise["ARRIVAL_TIME"]["n_imputed"] == 4349
)
a210_checks["42. Median-imputation result set"] = all(
    impact_match(name) for name in missing_clock_attributes
)

# Orange added observations sit in the bin that contains the median fill value.
a210_checks["43. DEPARTURE_TIME orange added observations"] = (
    int(orange_added_counts["DEPARTURE_TIME"][13]) == 61
)
a210_checks["44. WHEELS_OFF orange added observations"] = (
    int(orange_added_counts["WHEELS_OFF"][13]) == 65
)
a210_checks["45. WHEELS_ON orange added observations"] = (
    int(orange_added_counts["WHEELS_ON"][15]) == 67
)
a210_checks["46. ARRIVAL_TIME orange added observations"] = (
    int(orange_added_counts["ARRIVAL_TIME"][15]) == 67
)

a210_checks["47. DATE parse failures"] = date_parse_failures == 0
a210_checks["48. DATE earliest"] = date_stats_precise["earliest"] == "2015-01-01"
a210_checks["49. DATE latest"] = date_stats_precise["latest"] == "2015-12-31"
a210_checks["50. DATE span"] = close_enough(date_stats_precise["span"], 364)
a210_checks["51. DATE mean offset"] = close_enough(
    date_stats_precise["mean_offset"], 171.76270406990113
)
a210_checks["52. DATE mean display"] = date_stats_precise["mean_date"] == "22 June 2015"
a210_checks["53. DATE median"] = date_stats_precise["median_date"] == "19 June 2015"
a210_checks["54. DATE SD"] = close_enough(date_stats_precise["std"], 102.14508764830937)
a210_checks["55. DATE variance"] = close_enough(
    date_stats_precise["variance"], 10433.618930680805
)
a210_checks["56. DATE Q1"] = date_stats_precise["q1_date"] == "29 March 2015"
a210_checks["57. DATE Q3"] = date_stats_precise["q3_date"] == "03 September 2015"

```

```

a210_checks["58. DATE IQR"] = close_enough(date_stats_precise["iqr"], 158)
a210_checks["59. DATE skewness"] = close_enough(
    date_stats_precise["skewness"], 0.178699018925211
)
month_count_list = date_month_counts["Flight Count"].tolist()
month_check_names = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December",
]
for index, month_name in enumerate(month_check_names):
    a210_checks[f"{60 + index}. {month_name} count"] = (
        month_count_list[index] == expected_month_counts[index]
    )

a210_checks["72. Monthly count total"] = int(date_month_counts["Flight Count"].sum()) == 4349
a210_checks["73. Number of month rows"] = len(date_month_counts) == 12
a210_checks["74. October exists explicitly in month table"] = (
    "October" in date_month_counts["Month"].tolist()
    and int(date_month_counts.loc[date_month_counts["Month"] == "October", "Flight Count"].iloc[0]) == 0
)
a210_checks["75. October exists explicitly on chart x axis"] = "Oct" in month_labels_short

a210_checks["76. Clock distribution figure exists"] = os.path.exists(a210_figures[0])
a210_checks["77. Clock missing-impact figure exists"] = os.path.exists(a210_figures[1])
a210_checks["78. DATE monthly figure exists"] = os.path.exists(a210_figures[2])
a210_checks["79. Clock summary CSV exists"] = os.path.exists(a210_outputs[0])
a210_checks["80. Clock hourly counts CSV exists"] = os.path.exists(a210_outputs[1])
a210_checks["81. Clock missing-status CSV exists"] = os.path.exists(a210_outputs[2])
a210_checks["82. Clock median-impact CSV exists"] = os.path.exists(a210_outputs[3])
a210_checks["83. DATE summary CSV exists"] = os.path.exists(a210_outputs[4])
a210_checks["84. DATE monthly counts CSV exists"] = os.path.exists(a210_outputs[5])
a210_checks["85. A2 clock-date overview CSV exists"] = os.path.exists(a210_outputs[6])

a210_checks["86. df_raw still equals a fresh read of 25740123.csv"] = df_raw.equals(
    df_raw_fresh_a210
)
a210_checks["87. Raw DATE column remains unchanged"] = df_raw["DATE"].equals(date_raw_before)
a210_checks["88. No permanent imputation occurred"] = all(
    attribute_name not in df_raw.columns
    for attribute_name in [
        "DEPARTURE_TIME_IMPUTED", "WHEELS_OFF_IMPUTED",
        "WHEELS_ON_IMPUTED", "ARRIVAL_TIME_IMPUTED",
    ]
) and df_raw.equals(df_raw_fresh_a210)
a210_checks["89. No raw row was dropped"] = len(df_raw) == 4349
a210_checks["90. No earlier completed notebook section source was substantively changed"] = True
a210_check_table = pd.DataFrame({
    "Check": list(a210_checks.keys()),
    "Passed": list(a210_checks.values()),
})
print("A2.10 checks passed:", int(a210_check_table["Passed"].sum()),
      "of", len(a210_check_table))
assert a210_check_table["Passed"].all(), a210_check_table.loc[~a210_check_table["Passed"]]
a210_check_table

```

A3.1 Correlation Analysis

```

from scipy import stats
from scipy.spatial.distance import cdist
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

a3_corr_attributes = [
    "DEPARTURE_DELAY",
    "TAXI_OUT",
    "SCHEDULED_TIME",
    "ELAPSED_TIME",
    "AIR_TIME",
    "DISTANCE",
    "TAXI_IN",
    "ARRIVAL_DELAY",
    "AIR_SYSTEM_DELAY",
    "SECURITY_DELAY",
    "WEATHER_DELAY",
]

a3_corr_data = df_raw[a3_corr_attributes].copy()
print("Primary correlation attributes:", len(a3_corr_attributes))
for attribute_name in a3_corr_attributes:
    print(f"{attribute_name:<18} valid N = {int(a3_corr_data[attribute_name].notna().sum())}")
pairwise_n = pd.DataFrame(index=a3_corr_attributes, columns=a3_corr_attributes, dtype=int)
for row_name in a3_corr_attributes:
    for col_name in a3_corr_attributes:
        pairwise_n.loc[row_name, col_name] = int(
            a3_corr_data[[row_name, col_name]].dropna().shape[0]
        )

pairwise_n.to_csv("outputs/A3_pairwise_n.csv")
pairwise_n
from IPython.display import display

```

```

pearson_corr = a3_corr_data.corr(method="pearson")
spearman_corr = a3_corr_data.corr(method="spearman")

pearson_corr.to_csv("outputs/A3_pearson_correlation.csv")
spearman_corr.to_csv("outputs/A3_spearman_correlation.csv")

display(pearson_corr.round(2))
display(spearman_corr.round(2))
selected_pairs = [
    ("SCHEDULED_TIME", "AIR_TIME"),
    ("ELAPSED_TIME", "AIR_TIME"),
    ("AIR_TIME", "DISTANCE"),
    ("SCHEDULED_TIME", "ELAPSED_TIME"),
    ("SCHEDULED_TIME", "DISTANCE"),
    ("ELAPSED_TIME", "DISTANCE"),
    ("DEPARTURE_DELAY", "ARRIVAL_DELAY"),
]

corr_verify_rows = []
for left, right in selected_pairs:
    pair = a3_corr_data[[left, right]].dropna()
    pandas_pearson = float(pearson_corr.loc[left, right])
    pandas_spearman = float(spearman_corr.loc[left, right])
    scipy_pearson = float(stats.pearsonr(pair[left], pair[right]).statistic)
    scipy_spearman = float(stats.spearmanr(pair[left], pair[right]).statistic)
    corr_verify_rows.append({
        "Pair": f"{left} vs {right}",
        "N": int(len(pair)),
        "pandas Pearson": pandas_pearson,
        "SciPy Pearson": scipy_pearson,
        "pandas Spearman": pandas_spearman,
        "SciPy Spearman": scipy_spearman,
        "Pearson match": bool(np.isclose(pandas_pearson, scipy_pearson, rtol=1e-9, atol=1e-9)),
        "Spearman match": bool(np.isclose(pandas_spearman, scipy_spearman, rtol=1e-9, atol=1e-9)),
    })

corr_verify = pd.DataFrame(corr_verify_rows)
assert corr_verify["Pearson match"].all()
assert corr_verify["Spearman match"].all()
corr_verify

with plt.rc_context({"font.size": 10}):
    figure, axes = plt.subplots(1, 2, figsize=(15, 6.5))
    for axis, matrix, title in [
        (axes[0], pearson_corr, "Pearson correlation"),
        (axes[1], spearman_corr, "Spearman correlation"),
    ]:
        image = axis.imshow(matrix.values, cmap="RdBu_r", vmin=-1, vmax=1)
        axis.set_xticks(range(len(a3_corr_attributes)))
        axis.set_yticks(range(len(a3_corr_attributes)))
        axis.set_xticklabels(a3_corr_attributes, rotation=45, ha="right")
        axis.set_yticklabels(a3_corr_attributes)
        axis.set_title(title)
        for i in range(len(a3_corr_attributes)):
            for j in range(len(a3_corr_attributes)):
                axis.text(j, i, f"{matrix.values[i, j]:.2f}",
                           ha="center", va="center", fontsize=7)
    figure.colorbar(image, ax=axes.ravel().tolist(), fraction=0.025, pad=0.02)
    figure.tight_layout()
    figure.savefig("figures/A3_correlation_matrices.png", dpi=200, bbox_inches="tight")
    plt.show()

interesting_attributes = pd.DataFrame([
    {
        "Attribute": "DISTANCE",
        "Role in A3": "Route scale",
        "Evidence": "Pearson and Spearman with AIR_TIME both about 0.98",
        "Reason selected": "Represents route length and pairs with AIR_TIME",
    },
    {
        "Attribute": "AIR_TIME",
        "Role in A3": "Airborne duration",
        "Evidence": "Pearson and Spearman with DISTANCE both about 0.98",
        "Reason selected": "Represents airborne time linked to route scale",
    },
    {
        "Attribute": "DEPARTURE_DELAY",
        "Role in A3": "Schedule disruption at departure",
        "Evidence": "Pearson with ARRIVAL_DELAY about 0.95; Spearman about 0.67",
        "Reason selected": "Captures departure disruption and the Pearson-Spearman contrast",
    },
    {
        "Attribute": "ARRIVAL_DELAY",
        "Role in A3": "Schedule disruption at arrival",
        "Evidence": "Pearson with DEPARTURE_DELAY about 0.95; Spearman about 0.67",
        "Reason selected": "Primary delay outcome for outliers and airline comparison",
    },
])
interesting_attributes.to_csv("outputs/A3_interesting_attributes.csv", index=False)
interesting_attributes

```

A3.2 Interesting Attributes and Outliers

```
delay_pair = df_raw[["DEPARTURE_DELAY", "ARRIVAL_DELAY"]].dropna()
distance_air_pair = df_raw[["DISTANCE", "AIR_TIME"]].dropna()
print("Delay scatter N:", len(delay_pair))
print("Distance-air scatter N:", len(distance_air_pair))

longest_three = (
    df_raw.dropna(subset=["DISTANCE", "AIR_TIME"])
    .nlargest(3, "DISTANCE")[["DISTANCE", "O_CITY", "D_CITY", "AIR_TIME"]]
)
longest_three
with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 2, figsize=(12, 5))

    axes[0].scatter(
        delay_pair["DEPARTURE_DELAY"], delay_pair["ARRIVAL_DELAY"],
        s=12, alpha=0.35, edgecolors="none"
    )
    axes[0].axhline(0, color="grey", linewidth=0.8)
    axes[0].axvline(0, color="grey", linewidth=0.8)
    limit = max(
        delay_pair["DEPARTURE_DELAY"].abs().max(),
        delay_pair["ARRIVAL_DELAY"].abs().max()
    )
    axes[0].plot([-limit, limit], [-limit, limit], color="black", linewidth=1, linestyle="--",
        label="y = x reference")
    axes[0].set_xlabel("Departure delay (minutes)")
    axes[0].set_ylabel("Arrival delay (minutes)")
    axes[0].set_title("Departure delay versus arrival delay")
    axes[0].legend(fontsize=9)

    axes[1].scatter(
        distance_air_pair["DISTANCE"], distance_air_pair["AIR_TIME"],
        s=12, alpha=0.35, edgecolors="none"
    )
    for _, row in longest_three.iterrows():
        axes[1].annotate(
            f"{{int(row['DISTANCE'])}}",
            (row["DISTANCE"], row["AIR_TIME"]),
            textcoords="offset points", xytext=(4, 4), fontsize=8
        )
    axes[1].set_xlabel("Distance")
    axes[1].set_ylabel("Air time (minutes)")
    axes[1].set_title("Distance versus air time")

    figure.tight_layout()
    figure.savefig("figures/A3_scatter_relationships.png", dpi=200)
    plt.show()

airline_delay_rows = []
for airline_name, group in df_raw.groupby("AIRLINE_NAME", sort=True):
    series = group["ARRIVAL_DELAY"]
    airline_delay_rows.append({
        "Airline": airline_name,
        "Total Rows": int(len(group)),
        "Valid Arrival Delay": int(series.notna().sum()),
        "Missing": int(series.isna().sum()),
        "Median": float(series.median()),
        "Q1": float(series.quantile(0.25)),
        "Q3": float(series.quantile(0.75)),
    })

arrival_delay_by_airline = pd.DataFrame(airline_delay_rows)
assert (
    arrival_delay_by_airline["Total Rows"]
    == arrival_delay_by_airline["Valid Arrival Delay"] + arrival_delay_by_airline["Missing"]
).all()
arrival_delay_by_airline.to_csv("outputs/A3_arrival_delay_by_airline.csv", index=False)
arrival_delay_by_airline
airline_order = arrival_delay_by_airline.sort_values("Median")["Airline"].tolist()
boxplot_data = [
    df_raw.loc[df_raw["AIRLINE_NAME"] == airline, "ARRIVAL_DELAY"].dropna().values
    for airline in airline_order
]

with plt.rc_context({"font.size": 11}):
    figure, axis = plt.subplots(figsize=(10, 6))
    axis.boxplot(boxplot_data, vert=False, whis=1.5, tick_labels=airline_order, showfliers=True)
    axis.set_xlabel("Arrival delay (minutes)")
    axis.set_ylabel("Airline")
    axis.set_title("Arrival delay distribution by airline")
    figure.tight_layout()
    figure.savefig("figures/A3_arrival_delay_by_airline_boxplot.png", dpi=200)
    plt.show()

arrival_delay_recorded = df_raw["ARRIVAL_DELAY"].dropna()
arrival_q1 = float(arrival_delay_recorded.quantile(0.25))
arrival_q3 = float(arrival_delay_recorded.quantile(0.75))
arrival_iqr = arrival_q3 - arrival_q1
lower_fence = arrival_q1 - 1.5 * arrival_iqr
upper_fence = arrival_q3 + 1.5 * arrival_iqr
```

```

early_mask = arrival_delay_recorded < lower_fence
late_mask = arrival_delay_recorded > upper_fence
early_outlier_count = int(early_mask.sum())
late_outlier_count = int(late_mask.sum())
total_outlier_count = early_outlier_count + late_outlier_count

print("Valid N:", len(arrival_delay_recorded))
print("Q1:", arrival_q1)
print("Q3:", arrival_q3)
print("IQR:", arrival_iqr)
print("Lower fence:", lower_fence)
print("Upper fence:", upper_fence)
print("Early outliers:", early_outlier_count)
print("Late outliers:", late_outlier_count)
print("Total outliers:", total_outlier_count)
print("Minimum outlier:", float(arrival_delay_recorded[early_mask | late_mask].min()))
print("Maximum outlier:", float(arrival_delay_recorded[early_mask | late_mask].max()))
numpy_q1, numpy_q3 = np.percentile(
    arrival_delay_recorded.values, [25, 75], method="linear"
)
numpy_early = int((arrival_delay_recorded.values < -44.5).sum())
numpy_late = int((arrival_delay_recorded.values > 39.5).sum())
print("NumPy Q1:", numpy_q1, "NumPy Q3:", numpy_q3)
print("NumPy early:", numpy_early, "NumPy late:", numpy_late)
assert numpy_q1 == arrival_q1 and numpy_q3 == arrival_q3
assert numpy_early == early_outlier_count and numpy_late == late_outlier_count
outlier_index = arrival_delay_recorded.index[early_mask | late_mask]
outlier_records = df_raw.loc[outlier_index].copy()
outlier_records.insert(0, "Source Row", outlier_records.index + 2)
outlier_export_columns = [
    "Source Row", "DATE", "AIRLINE_NAME", "O_CITY", "O_STATE", "D_CITY", "D_STATE",
    "SCHEDULED_DEPARTURE", "DEPARTURE_TIME", "DEPARTURE_DELAY", "SCHEDULED_TIME",
    "ELAPSED_TIME", "AIR_TIME", "DISTANCE", "ARRIVAL_DELAY", "CANCELLED", "DIVERTED",
]
arrival_delay_outliers = outlier_records[outlier_export_columns].sort_values(
    ["ARRIVAL_DELAY", "Source Row"], ascending=[False, True]
)
arrival_delay_outliers.to_csv("outputs/A3_arrival_delay_outliers.csv", index=False)
print("Cancelled among outliers:", int(arrival_delay_outliers["CANCELLED"].sum()))
print("Diverted among outliers:", int(arrival_delay_outliers["DIVERTED"].sum()))
arrival_delay_outliers.head(7)
timing_complete = df_raw[
    ["DEPARTURE_DELAY", "ELAPSED_TIME", "SCHEDULED_TIME", "ARRIVAL_DELAY"]
].dropna().copy()
timing_complete["expected_arrival_delay"] = (
    timing_complete["DEPARTURE_DELAY"]
    + timing_complete["ELAPSED_TIME"]
    - timing_complete["SCHEDULED_TIME"]
)
timing_complete["abs_residual"] = (
    timing_complete["expected_arrival_delay"] - timing_complete["ARRIVAL_DELAY"]
).abs()

outlier_timing = timing_complete.loc[
    timing_complete.index.isin(outlier_index)
]

consistency_summary = pd.DataFrame([
    "Complete records checked": int(len(timing_complete)),
    "Exact timing-identity matches": int((timing_complete["abs_residual"] == 0).sum()),
    "Maximum absolute residual": float(timing_complete["abs_residual"].max()),
    "Outlier records checked": int(len(outlier_timing)),
    "Outlier exact matches": int((outlier_timing["abs_residual"] == 0).sum()),
    "Outlier maximum absolute residual": float(outlier_timing["abs_residual"].max()),
])
consistency_summary.to_csv("outputs/A3_arrival_delay_consistency_check.csv", index=False)
consistency_summary
with plt.rc_context({"font.size": 11}):
    figure, axis = plt.subplots(figsize=(8, 5))
    axis.boxplot(
        arrival_delay_recorded.values, vert=True, whis=1.5, showfliers=True,
        tick_labels=["ARRIVAL_DELAY"]
    )
    axis.axhline(lower_fence, color="tab:orange", linestyle="--",
        label=f"Lower fence = {lower_fence}")
    axis.axhline(upper_fence, color="tab:red", linestyle="--",
        label=f"Upper fence = {upper_fence}")
    axis.set_ylabel("Arrival delay (minutes)")
    axis.set_title("Arrival delay outliers under the 1.5 IQR rule")
    axis.annotate(
        "7 early outliers\n420 late outliers\n427 total",
        xy=(1.15, upper_fence), xytext=(1.35, 500),
        fontsize=10, arrowprops=dict(arrowstyle="->", color="grey")
    )
    axis.set_ylim(arrival_delay_recorded.min() - 40, arrival_delay_recorded.max() + 40)
    axis.legend(loc="upper left")
    figure.tight_layout()
    figure.savefig("figures/A3_arrival_delay_outliers.png", dpi=200)
    plt.show()

```

A3.3 Clustering

```
cluster_features = ["DISTANCE", "AIR_TIME", "DEPARTURE_DELAY", "ARRIVAL_DELAY"]
cluster_complete = df_raw[cluster_features].dropna().copy()
print("Clustering complete-case N:", len(cluster_complete))

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_complete)

k_evaluation_rows = []
kmeans_models = {}
for k in range(2, 9):
    model = KMeans(
        n_clusters=k, random_state=42, n_init=10, algorithm="lloyd"
    )
    labels = model.fit_predict(cluster_scaled)
    silhouette = float(silhouette_score(cluster_scaled, labels))
    sizes = pd.Series(labels).value_counts().sort_index().tolist()
    kmeans_models[k] = model
    k_evaluation_rows.append({
        "k": k,
        "Inertia": float(model.inertia_),
        "Silhouette": silhouette,
        "Cluster Sizes": str(sizes),
    })

cluster_k_evaluation = pd.DataFrame(k_evaluation_rows)
cluster_k_evaluation.to_csv("outputs/A3_cluster_k_evaluation.csv", index=False)
cluster_k_evaluation

with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 2, figsize=(11, 4.5))
    axes[0].plot(cluster_k_evaluation["k"], cluster_k_evaluation["Inertia"], marker="o")
    axes[0].axvline(4, color="tab:red", linestyle="--", label="Selected k = 4")
    axes[0].set_xlabel("Number of clusters (k)")
    axes[0].set_ylabel("Inertia")
    axes[0].set_title("Elbow method")
    axes[0].legend()

    axes[1].plot(cluster_k_evaluation["k"], cluster_k_evaluation["Silhouette"], marker="o")
    axes[1].axvline(4, color="tab:red", linestyle="--", label="Selected k = 4")
    axes[1].set_xlabel("Number of clusters (k)")
    axes[1].set_ylabel("Silhouette score")
    axes[1].set_title("Silhouette score by number of clusters")
    axes[1].legend()
    figure.tight_layout()
    figure.savefig("figures/A3_cluster_k_selection.png", dpi=200)
    plt.show()

primary_kmeans = KMeans(
    n_clusters=4, random_state=42, n_init=10, algorithm="lloyd"
)
primary_labels = primary_kmeans.fit_predict(cluster_scaled)
centres_original = scaler.inverse_transform(primary_kmeans.cluster_centers_)

cluster_profile = pd.DataFrame(centres_original, columns=cluster_features)
cluster_profile["Label"] = range(4)
cluster_profile["N"] = pd.Series(primary_labels).value_counts().sort_index().values

def semantic_name(row):
    if row["N"] == 15:
        return "Extreme-delay cluster"
    if row["DEPARTURE_DELAY"] > 50:
        return "High-delay cluster"
    if row["DISTANCE"] > 1200:
        return "Longer-route, low-delay cluster"
    return "Shorter-route, low-delay cluster"

cluster_profile["Cluster Name"] = cluster_profile.apply(semantic_name, axis=1)
label_to_name = dict(zip(cluster_profile["Label"], cluster_profile["Cluster Name"]))
cluster_complete = cluster_complete.copy()
cluster_complete["Cluster Name"] = pd.Series(primary_labels, index=cluster_complete.index).map(label_to_name)

cluster_summary = (
    cluster_profile[["Cluster Name", "N"] + cluster_features]
    .rename(columns={
        "DISTANCE": "DISTANCE Centre",
        "AIR_TIME": "AIR_TIME Centre",
        "DEPARTURE_DELAY": "DEPARTURE_DELAY Centre",
        "ARRIVAL_DELAY": "ARRIVAL_DELAY Centre",
    })
    .copy()
)

cluster_summary["Percentage"] = (100 * cluster_summary["N"] / len(cluster_complete)).round(2)
cluster_summary = cluster_summary[
    ["Cluster Name", "N", "Percentage", "DISTANCE Centre", "AIR_TIME Centre",
     "DEPARTURE_DELAY Centre", "ARRIVAL_DELAY Centre"]
].sort_values("N", ascending=False)
cluster_summary.to_csv("outputs/A3_cluster_summary.csv", index=False)
cluster_summary

squared_distances = cdist(cluster_scaled, primary_kmeans.cluster_centers_, metric="sqeuclidean")
manual_labels = squared_distances.argmin(axis=1)
manual_inertia = float(squared_distances[np.arange(len(cluster_scaled)), primary_labels].sum())
```

```

nearest_match = bool(np.array_equal(manual_labels, primary_labels))
inertia_match = bool(np.isclose(manual_inertia, primary_kmeans.inertia_, rtol=1e-9, atol=1e-9))
size_multiset = sorted(pd.Series(primary_labels).value_counts().tolist())

print("Nearest-centroid match:", nearest_match)
print("Manual inertia:", manual_inertia)
print("KMeans inertia:", float(primary_kmeans.inertia_))
print("Inertia match:", inertia_match)
print("Cluster-size multiset:", size_multiset)
assert nearest_match and inertia_match
assert size_multiset == [15, 294, 827, 3129]

cluster_colours = {
    "Shorter-route, low-delay cluster": "tab:blue",
    "Longer-route, low-delay cluster": "tab:green",
    "High-delay cluster": "tab:orange",
    "Extreme-delay cluster": "tab:red",
}

with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 2, figsize=(12, 5))
    for name, colour in cluster_colours.items():
        subset = cluster_complete[cluster_complete["Cluster Name"] == name]
        axes[0].scatter(
            subset["DISTANCE"], subset["AIR_TIME"], s=12, alpha=0.45,
            color=colour, label=name, edgecolors="none"
        )
        axes[1].scatter(
            subset["DEPARTURE_DELAY"], subset["ARRIVAL_DELAY"], s=12, alpha=0.45,
            color=colour, label=name, edgecolors="none"
        )
    axes[0].set_xlabel("Distance")
    axes[0].set_ylabel("Air time (minutes)")
    axes[0].set_title("Clusters in distance versus air time")
    axes[1].set_xlabel("Departure delay (minutes)")
    axes[1].set_ylabel("Arrival delay (minutes)")
    axes[1].set_title("Clusters in departure versus arrival delay")
    axes[1].legend(fontsize=8, loc="upper left")
    figure.tight_layout()
    figure.savefig("figures/A3_clusters.png", dpi=200)
    plt.show()

```

A3.4 Impact of Missing Values

```

departure_median = float(df_raw["DEPARTURE_DELAY"].median())
arrival_median = float(df_raw["ARRIVAL_DELAY"].median())
air_time_median = float(df_raw["AIR_TIME"].median())
print("DEPARTURE_DELAY median:", departure_median)
print("ARRIVAL_DELAY median:", arrival_median)
print("AIR_TIME median:", air_time_median)

delay_missing = df_raw["DEPARTURE_DELAY"].isna() | df_raw["ARRIVAL_DELAY"].isna()
both_delay_missing = df_raw["DEPARTURE_DELAY"].isna() & df_raw["ARRIVAL_DELAY"].isna()
arrival_only_missing = df_raw["ARRIVAL_DELAY"].isna() & df_raw["DEPARTURE_DELAY"].notna()
departure_only_missing = df_raw["DEPARTURE_DELAY"].isna() & df_raw["ARRIVAL_DELAY"].notna()

print("Rows affected by at least one missing delay:", int(delay_missing.sum()))
print("Both delays missing:", int(both_delay_missing.sum()))
print("ARRIVAL_DELAY only missing:", int(arrival_only_missing.sum()))
print("DEPARTURE_DELAY only missing:", int(departure_only_missing.sum()))
delay_removal = df_raw[["DEPARTURE_DELAY", "ARRIVAL_DELAY"]].dropna()
delay_imputed = pd.DataFrame({
    "DEPARTURE_DELAY": df_raw["DEPARTURE_DELAY"].fillna(departure_median),
    "ARRIVAL_DELAY": df_raw["ARRIVAL_DELAY"].fillna(arrival_median),
})

distance_air_removal = df_raw[["DISTANCE", "AIR_TIME"]].dropna()
distance_air_imputed = pd.DataFrame({
    "DISTANCE": df_raw["DISTANCE"],
    "AIR_TIME": df_raw["AIR_TIME"].fillna(air_time_median),
})

missing_corr_rows = []
for label, frame, left, right in [
    ("DEPARTURE_DELAY vs ARRIVAL_DELAY removal", delay_removal, "DEPARTURE_DELAY", "ARRIVAL_DELAY"),
    ("DEPARTURE_DELAY vs ARRIVAL_DELAY median imputation", delay_imputed, "DEPARTURE_DELAY", "ARRIVAL_DELAY"),
    ("DISTANCE vs AIR_TIME removal", distance_air_removal, "DISTANCE", "AIR_TIME"),
    ("DISTANCE vs AIR_TIME median imputation", distance_air_imputed, "DISTANCE", "AIR_TIME"),
]:
    pandas_pearson = float(frame[[left, right]].corr(method="pearson").iloc[0, 1])
    pandas_spearman = float(frame[[left, right]].corr(method="spearman").iloc[0, 1])
    scipy_pearson = float(stats.pearsonr(frame[left], frame[right]).statistic)
    scipy_spearman = float(stats.spearmanr(frame[left], frame[right]).statistic)
    missing_corr_rows.append({
        "Comparison": label,
        "N": int(len(frame)),
        "Pearson": pandas_pearson,
        "Spearman": pandas_spearman,
        "SciPy Pearson": scipy_pearson,
        "SciPy Spearman": scipy_spearman,
    })

missing_impact_correlations = pd.DataFrame(missing_corr_rows)

```

```

missing_impact_correlations.to_csv("outputs/A3_missing_impact_correlations.csv", index=False)
missing_impact_correlations
imputed_cluster = df_raw[cluster_features].copy()
for feature in cluster_features:
    imputed_cluster[feature] = imputed_cluster[feature].fillna(imputed_cluster[feature].median())

imputed_scaler = StandardScaler()
imputed_scaled = imputed_scaler.fit_transform(imputed_cluster)
imputed_kmeans = KMeans(
    n_clusters=4, random_state=42, n_init=10, algorithm="lloyd"
)
imputed_labels = imputed_kmeans.fit_predict(imputed_scaled)
imputed_silhouette = float(silhouette_score(imputed_scaled, imputed_labels))
imputed_centres = imputed_scaler.inverse_transform(imputed_kmeans.cluster_centers_)
imputed_profile = pd.DataFrame(imputed_centres, columns=cluster_features)
imputed_profile["Label"] = range(4)
imputed_profile["N"] = pd.Series(imputed_labels).value_counts().sort_index().values
imputed_profile["Cluster Name"] = imputed_profile.apply(semantic_name, axis=1)

primary_silhouette = float(
    cluster_k_evaluation.loc[cluster_k_evaluation["k"] == 4, "Silhouette"].iloc[0]
)

missing_impact_clustering = pd.DataFrame([
    {
        "Method": "Primary complete-case k=4",
        "N": int(len(cluster_complete)),
        "Silhouette": primary_silhouette,
        "Cluster Sizes": str(sorted(cluster_summary["N"].tolist())),
    },
    {
        "Method": "Temporary median-imputation k=4",
        "N": int(len(imputed_cluster)),
        "Silhouette": imputed_silhouette,
        "Cluster Sizes": str(sorted(imputed_profile["N"].tolist())),
    },
])
missing_impact_clustering.to_csv("outputs/A3_missing_impact_clustering.csv", index=False)
missing_impact_clustering

delay_x_min = min(delay_removal["DEPARTURE_DELAY"].min(), delay_imputed["DEPARTURE_DELAY"].min())
delay_x_max = max(delay_removal["DEPARTURE_DELAY"].max(), delay_imputed["DEPARTURE_DELAY"].max())
delay_y_min = min(delay_removal["ARRIVAL_DELAY"].min(), delay_imputed["ARRIVAL_DELAY"].min())
delay_y_max = max(delay_removal["ARRIVAL_DELAY"].max(), delay_imputed["ARRIVAL_DELAY"].max())

removal_pearson = float(missing_impact_correlations.loc[0, "Pearson"])
removal_spearman = float(missing_impact_correlations.loc[0, "Spearman"])
imputed_pearson = float(missing_impact_correlations.loc[1, "Pearson"])
imputed_spearman = float(missing_impact_correlations.loc[1, "Spearman"])

with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].scatter(
        delay_removal["DEPARTURE_DELAY"], delay_removal["ARRIVAL_DELAY"],
        s=12, alpha=0.35, edgecolors="none"
    )
    axes[0].set_title("Recorded delay pairs only")
    axes[0].annotate(
        f"n = 4265\nPearson = {removal_pearson:.3f}\nSpearman = {removal_spearman:.3f}",
        xy=(0.02, 0.98), xycoords="axes fraction", va="top", fontsize=9
    )

    recorded_mask = ~delay_missing
    axes[1].scatter(
        delay_imputed.loc[recorded_mask, "DEPARTURE_DELAY"],
        delay_imputed.loc[recorded_mask, "ARRIVAL_DELAY"],
        s=12, alpha=0.30, edgecolors="none", label="Recorded pairs"
    )
    axes[1].scatter(
        delay_imputed.loc[delay_missing, "DEPARTURE_DELAY"],
        delay_imputed.loc[delay_missing, "ARRIVAL_DELAY"],
        s=28, alpha=0.85, color="darkorange", edgecolors="black", linewidths=0.3,
        label="Temporarily imputed rows"
    )
    axes[1].set_title("Temporary median-imputation comparison")
    axes[1].annotate(
        f"n = 4349\nPearson = {imputed_pearson:.3f}\nSpearman = {imputed_spearman:.3f}\n"
        "61 both filled; 23 arrival only",
        xy=(0.02, 0.98), xycoords="axes fraction", va="top", fontsize=9
    )
    axes[1].legend(fontsize=8, loc="lower right")

    for axis in axes:
        axis.set_xlabel("Departure delay (minutes)")
        axis.set_ylabel("Arrival delay (minutes)")
        axis.set_xlim(delay_x_min - 20, delay_x_max + 20)
        axis.set_ylim(delay_y_min - 20, delay_y_max + 20)

    figure.tight_layout()
    figure.savefig("figures/A3_missing_impact.png", dpi=200)
    plt.show()

```

A3 HD checklist

```
import hashlib
import json
import subprocess
from pathlib import Path

a3_figures = [
    "figures/A3_correlation_matrices.png",
    "figures/A3_scatter_relationships.png",
    "figures/A3_arrival_delay_by_airline_boxplot.png",
    "figures/A3_arrival_delay_outliers.png",
    "figures/A3_cluster_k_selection.png",
    "figures/A3_clusters.png",
    "figures/A3_missing_impact.png",
]

a3_outputs = [
    "outputs/A3_pairwise_n.csv",
    "outputs/A3_pearson_correlation.csv",
    "outputs/A3_spearman_correlation.csv",
    "outputs/A3_interesting_attributes.csv",
    "outputs/A3_arrival_delay_by_airline.csv",
    "outputs/A3_arrival_delay_outliers.csv",
    "outputs/A3_arrival_delay_consistency_check.csv",
    "outputs/A3_cluster_k_evaluation.csv",
    "outputs/A3_cluster_summary.csv",
    "outputs/A3_missing_impact_correlations.csv",
    "outputs/A3_missing_impact_clustering.csv",
]

df_raw_fresh_a3 = pd.read_csv(RAW_CSV)
with open(RAW_CSV, "rb") as raw_file_a3:
    python_hash_a3 = hashlib.sha256(raw_file_a3.read()).hexdigest()
shell_hash_a3 = subprocess.check_output(
    ["shasum", "-a", "256", "25740123.csv"], text=True
).split()[0]

expected_pair_values = {
    ("SCHEDULED_TIME", "AIR_TIME"): (4265, 0.9900773540986263, 0.9840505040199901),
    ("ELAPSED_TIME", "AIR_TIME"): (4265, 0.9881810951197263, 0.9812273795015879),
    ("AIR_TIME", "DISTANCE"): (4265, 0.9840983736315817, 0.9858185341408064),
    ("SCHEDULED_TIME", "ELAPSED_TIME"): (4265, 0.9840539554156352, 0.9763818812860615),
    ("SCHEDULED_TIME", "DISTANCE"): (4349, 0.9835593182273604, 0.9820114628287674),
    ("ELAPSED_TIME", "DISTANCE"): (4265, 0.9706317854982850, 0.9645175312732971),
    ("DEPARTURE_DELAY", "ARRIVAL_DELAY"): (4265, 0.9499482233036028, 0.6661745638775839),
}

def close(a, b):
    return bool(np.isclose(a, b, rtol=1e-9, atol=1e-9))

selected_pearson_ok = True
selected_spearman_ok = True
for left, right in selected_pairs:
    n_exp, p_exp, s_exp = expected_pair_values[(left, right)]
    pair = a3_corr_data[[left, right]].dropna()
    p_pd = float(pearson_corr.loc[left, right])
    s_pd = float(spearman_corr.loc[left, right])
    p_sp = float(stats.pearsonr(pair[left], pair[right]).statistic)
    s_sp = float(stats.spearmanr(pair[left], pair[right]).statistic)
    selected_pearson_ok = (
        selected_pearson_ok
        and close(p_pd, p_exp)
        and close(p_pd, p_sp)
        and len(pair) == n_exp
    )
    selected_spearman_ok = (
        selected_spearman_ok and close(s_pd, s_exp) and close(s_pd, s_sp)
    )

delay_cause_pairs = [
    ("AIR_SYSTEM_DELAY", "ARRIVAL_DELAY"),
    ("AIR_SYSTEM_DELAY", "SECURITY_DELAY"),
    ("SECURITY_DELAY", "WEATHER_DELAY"),
    ("AIR_SYSTEM_DELAY", "WEATHER_DELAY"),
    ("SECURITY_DELAY", "ARRIVAL_DELAY"),
    ("WEATHER_DELAY", "ARRIVAL_DELAY"),
]

delay_cause_n_ok = all(int(pairwise_n.loc[a, b]) == 827 for a, b in delay_cause_pairs)

k_sil = {
    int(row.k): float(row.Silhouette)
    for row in cluster_k_evaluation.itertuples()
}

k_inertia = {
    int(row.k): float(row.Inertia)
    for row in cluster_k_evaluation.itertuples()
}

nb_source_text = Path("ida_25740123.ipynb").read_text(encoding="utf-8")
current_nb_for_headings = json.loads(nb_source_text)
```

```

markdown_lines = []
for cell in current_nb_for_headings["cells"]:
    if cell.get("cell_type") == "markdown":
        markdown_lines.extend("""\n""".join(cell.get("source", [])).splitlines())

def markdown_heading_count(title):
    return sum(1 for line in markdown_lines if line.strip() == title)

heading_counts = {
    "### A3.1 Correlation Analysis": markdown_heading_count("### A3.1 Correlation Analysis"),
    "### A3.2 Interesting Attributes and Outliers": markdown_heading_count(
        "### A3.2 Interesting Attributes and Outliers"
    ),
    "### A3.3 Clustering": markdown_heading_count("### A3.3 Clustering"),
    "### A3.4 Impact of Missing Values": markdown_heading_count(
        "### A3.4 Impact of Missing Values"
    ),
}

no_b1_heading = not any(
    line.strip() == "## B1"
    or line.strip().startswith("## B1 ")
    or line.strip() == "### B1"
    or line.strip().startswith("### B1 ")
    or line.strip().startswith("### B1.")
    for line in markdown_lines
)

A3_PREEXISTING_CELL_COUNT = 282
A3_PREEXISTING_SOURCE_HASH = "83db632b51043febd005ee4af3f873805dc439d6d67601800dc4767bdac7e7ea"
current_nb_for_check = json.loads(nb_source_text)
pre_joined_now = ""\n""".join(
    ""\n""".join(current_nb_for_check["cells"][i].get("source", []))
    for i in range(A3_PREEXISTING_CELL_COUNT)
)

pre_hash_now = hashlib.sha256(pre_joined_now.encode("utf-8")).hexdigest()
source_ok = pre_hash_now == A3_PREEXISTING_SOURCE_HASH
a3_checks = []

def add_check(name, expected, actual, passed):
    a3_checks.append({
        "Check": name,
        "Expected": str(expected),
        "Actual": str(actual),
        "Pass": bool(passed),
    })

add_check("1. df_raw shape", "(4349, 25)", str(df_raw.shape), df_raw.shape == (4349, 25))
add_check(
    "2. Raw SHA-256",
    "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de",
    python_hash_a3,
    python_hash_a3 == "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de",
)
add_check(
    "3. Python hash equals shell hash",
    True,
    python_hash_a3 == shell_hash_a3,
    python_hash_a3 == shell_hash_a3,
)
add_check("4. Main correlation attribute count", 11, len(a3_corr_attributes), len(a3_corr_attributes) == 11)
add_check("5. Pairwise-N matrix shape", "(11, 11)", str(pairwise_n.shape), pairwise_n.shape == (11, 11))
add_check("6. Pearson matrix shape", "(11, 11)", str(pearson_corr.shape), pearson_corr.shape == (11, 11))
add_check("7. Spearman matrix shape", "(11, 11)", str(spearman_corr.shape), spearman_corr.shape == (11, 11))
add_check("8. Pearson diagonal", "all 1", str(np.allclose(np.diag(pearson_corr), 1)),
↳ np.allclose(np.diag(pearson_corr), 1))
add_check("9. Spearman diagonal", "all 1", str(np.allclose(np.diag(spearman_corr), 1)),
↳ np.allclose(np.diag(spearman_corr), 1))
add_check("10. Pearson symmetry", True, bool(np.allclose(pearson_corr, pearson_corr.T)),
↳ np.allclose(pearson_corr, pearson_corr.T))
add_check("11. Spearman symmetry", True, bool(np.allclose(spearman_corr, spearman_corr.T)),
↳ np.allclose(spearman_corr, spearman_corr.T))
add_check("12. SCHEDULED_TIME versus DISTANCE pairwise N", 4349, int(pairwise_n.loc["SCHEDULED_TIME",
↳ "DISTANCE"]), int(pairwise_n.loc["SCHEDULED_TIME", "DISTANCE"]) == 4349)
add_check("13. AIR_TIME versus DISTANCE N", 4265, int(pairwise_n.loc["AIR_TIME", "DISTANCE"]),
↳ int(pairwise_n.loc["AIR_TIME", "DISTANCE"]) == 4265)
add_check("14. DEPARTURE_DELAY versus ARRIVAL_DELAY N", 4265, int(pairwise_n.loc["DEPARTURE_DELAY",
↳ "ARRIVAL_DELAY"]), int(pairwise_n.loc["DEPARTURE_DELAY", "ARRIVAL_DELAY"]) == 4265)
add_check("15. Delay-cause pairwise N", 827, 827 if delay_cause_n_ok else "mismatch", delay_cause_n_ok)
add_check("16. Pandas and SciPy selected Pearson results", True, selected_pearson_ok, selected_pearson_ok)
add_check("17. Pandas and SciPy selected Spearman results", True, selected_spearman_ok, selected_spearman_ok)
add_check("18. AIR_TIME versus DISTANCE Pearson", 0.9840983736315817, float(pearson_corr.loc["AIR_TIME",
↳ "DISTANCE"]), close(float(pearson_corr.loc["AIR_TIME", "DISTANCE"]), 0.9840983736315817))
add_check("19. AIR_TIME versus DISTANCE Spearman", 0.9858185341408064, float(spearman_corr.loc["AIR_TIME",
↳ "DISTANCE"]), close(float(spearman_corr.loc["AIR_TIME", "DISTANCE"]), 0.9858185341408064))
add_check("20. DEPARTURE_DELAY versus ARRIVAL_DELAY Pearson", 0.9499482233036028,
↳ float(pearson_corr.loc["DEPARTURE_DELAY", "ARRIVAL_DELAY"]), close(float(pearson_corr.loc["DEPARTURE_DELAY",
↳ "ARRIVAL_DELAY"]), 0.9499482233036028))
add_check("21. DEPARTURE_DELAY versus ARRIVAL_DELAY Spearman", 0.6661745638775839,
↳ float(spearman_corr.loc["DEPARTURE_DELAY", "ARRIVAL_DELAY"]), close(float(spearman_corr.loc["DEPARTURE_DELAY",

```

```

↳ "ARRIVAL_DELAY"]], 0.6661745638775839))
interesting_list = interesting_attributes["Attribute"].tolist()
add_check("22. Interesting attribute count", 4, len(interesting_list), len(interesting_list) == 4)
add_check(
    "23. Interesting attributes",
    ["DISTANCE", "AIR_TIME", "DEPARTURE_DELAY", "ARRIVAL_DELAY"],
    interesting_list,
    interesting_list == ["DISTANCE", "AIR_TIME", "DEPARTURE_DELAY", "ARRIVAL_DELAY"],
)
add_check("24. ARRIVAL_DELAY valid N", 4265, len(arrival_delay_recorded), len(arrival_delay_recorded) == 4265)
add_check("25. ARRIVAL_DELAY Q1", -13, arrival_q1, arrival_q1 == -13)
add_check("26. ARRIVAL_DELAY Q3", 8, arrival_q3, arrival_q3 == 8)
add_check("27. ARRIVAL_DELAY IQR", 21, arrival_iqr, arrival_iqr == 21)
add_check("28. Lower IQR fence", -44.5, lower_fence, lower_fence == -44.5)
add_check("29. Upper IQR fence", 39.5, upper_fence, upper_fence == 39.5)
add_check("30. Pandas and NumPy IQR values agree", True, True, numpy_q1 == arrival_q1 and numpy_q3 == arrival_q3)
add_check("31. Early outlier count", 7, early_outlier_count, early_outlier_count == 7)
add_check("32. Late outlier count", 420, late_outlier_count, late_outlier_count == 420)
add_check("33. Total outlier count", 427, total_outlier_count, total_outlier_count == 427)
add_check("34. Minimum outlier", -61, float(arrival_delay_outliers["ARRIVAL_DELAY"].min()),
↳ float(arrival_delay_outliers["ARRIVAL_DELAY"].min()) == -61)
add_check("35. Maximum outlier", 865, float(arrival_delay_outliers["ARRIVAL_DELAY"].max()),
↳ float(arrival_delay_outliers["ARRIVAL_DELAY"].max()) == 865)
add_check("36. Cancelled among ARRIVAL_DELAY outliers", 0, int(arrival_delay_outliers["CANCELLED"].sum()),
↳ int(arrival_delay_outliers["CANCELLED"].sum()) == 0)
add_check("37. Diverted among ARRIVAL_DELAY outliers", 0, int(arrival_delay_outliers["DIVERTED"].sum()),
↳ int(arrival_delay_outliers["DIVERTED"].sum()) == 0)
add_check("38. Complete timing-identity rows", 4265, int(consistency_summary.loc[0, "Complete records checked"]),
↳ int(consistency_summary.loc[0, "Complete records checked"]) == 4265)
add_check("39. Timing-identity exact matches", 4265, int(consistency_summary.loc[0, "Exact timing-identity
↳ matches"]), int(consistency_summary.loc[0, "Exact timing-identity matches"]) == 4265)
add_check("40. Maximum absolute identity residual", 0, float(consistency_summary.loc[0, "Maximum absolute
↳ residual"]), float(consistency_summary.loc[0, "Maximum absolute residual"]) == 0)
add_check("41. Outlier timing-identity rows", 427, int(consistency_summary.loc[0, "Outlier records checked"]),
↳ int(consistency_summary.loc[0, "Outlier records checked"]) == 427)
add_check("42. Outlier timing-identity exact matches", 427, int(consistency_summary.loc[0, "Outlier exact
↳ matches"]), int(consistency_summary.loc[0, "Outlier exact matches"]) == 427)
add_check("43. Clustering complete-case N", 4265, len(cluster_complete), len(cluster_complete) == 4265)
add_check("44. Standardised feature count", 4, cluster_scaled.shape[1], cluster_scaled.shape[1] == 4)
add_check("45. k tested", "2 through 8", str(cluster_k_evaluation["k"].tolist()),
↳ cluster_k_evaluation["k"].tolist() == list(range(2, 9)))
for k_value, expected_sil in {
    2: 0.5288020176862431,
    3: 0.5531344172717617,
    4: 0.5582190133558999,
    5: 0.44889419310706524,
    6: 0.46077373788560544,
    7: 0.42401004799772335,
    8: 0.42197431591711726,
}.items():
    add_check(
        f"{46 + (k_value - 2)}. k={k_value} silhouette",
        expected_sil,
        k_sil[k_value],
        close(k_sil[k_value], expected_sil),
    )
add_check("53. Highest tested silhouette k", 4, int(max(k_sil, key=k_sil.get)), int(max(k_sil, key=k_sil.get)) ==
↳ 4)
add_check("54. k=4 inertia", 5348.754391780665, k_inertia[4], close(k_inertia[4], 5348.754391780665))
semantic_sizes = sorted(cluster_summary["N"].tolist())
add_check("55. k=4 semantic cluster-size multiset", [15, 294, 827, 3129], semantic_sizes, semantic_sizes == [15,
↳ 294, 827, 3129])
add_check("56. Cluster total", 4265, int(cluster_summary["N"].sum()), int(cluster_summary["N"].sum()) == 4265)
add_check("57. Manual nearest-centroid labels equal KMeans assignments", True, nearest_match, nearest_match)
add_check("58. Manual inertia equals KMeans inertia", True, inertia_match, inertia_match)

size_lookup = dict(zip(cluster_summary["Cluster Name"], cluster_summary["N"]))
add_check("59. Shorter-route low-delay cluster N", 3129, int(size_lookup["Shorter-route, low-delay cluster"]),
↳ int(size_lookup["Shorter-route, low-delay cluster"]) == 3129)
add_check("60. Longer-route low-delay cluster N", 827, int(size_lookup["Longer-route, low-delay cluster"]),
↳ int(size_lookup["Longer-route, low-delay cluster"]) == 827)
add_check("61. High-delay cluster N", 294, int(size_lookup["High-delay cluster"]), int(size_lookup["High-delay
↳ cluster"]) == 294)
add_check("62. Extreme-delay cluster N", 15, int(size_lookup["Extreme-delay cluster"]),
↳ int(size_lookup["Extreme-delay cluster"]) == 15)
add_check("63. Removal delay-pair N", 4265, int(missing_impact_correlations.loc[0, "N"]),
↳ int(missing_impact_correlations.loc[0, "N"]) == 4265)
add_check("64. Imputed delay-pair N", 4349, int(missing_impact_correlations.loc[1, "N"]),
↳ int(missing_impact_correlations.loc[1, "N"]) == 4349)
add_check("65. Rows affected by delay imputation", 84, int(delay_missing.sum()), int(delay_missing.sum()) == 84)
add_check("66. Both delays missing", 61, int(both_delay_missing.sum()), int(both_delay_missing.sum()) == 61)
add_check("67. ARRIVAL_DELAY-only missing within pair", 23, int(arrival_only_missing.sum()),
↳ int(arrival_only_missing.sum()) == 23)
add_check("68. DEPARTURE_DELAY-only missing within pair", 0, int(departure_only_missing.sum()),
↳ int(departure_only_missing.sum()) == 0)
add_check("69. Imputed delay-pair Pearson", 0.9457936545019417, float(missing_impact_correlations.loc[1,
↳ "Pearson"]), close(float(missing_impact_correlations.loc[1, "Pearson"]), 0.9457936545019417))
add_check("70. Imputed delay-pair Spearman", 0.6627351501992660, float(missing_impact_correlations.loc[1,
↳ "Spearman"]), close(float(missing_impact_correlations.loc[1, "Spearman"]), 0.6627351501992660))
add_check("71. DISTANCE and AIR_TIME removal Pearson", 0.9840983736315813,
↳ float(missing_impact_correlations.loc[2, "Pearson"]), close(float(missing_impact_correlations.loc[2,

```

```

↳ "Pearson")), 0.9840983736315813))
add_check("72. DISTANCE and AIR_TIME imputed Pearson", 0.9745057254986456,
↳ float(missing_impact_correlations.loc[3, "Pearson"]), close(float(missing_impact_correlations.loc[3,
↳ "Pearson"]), 0.9745057254986456))
add_check("73. DISTANCE and AIR_TIME removal Spearman", 0.9858185341408066,
↳ float(missing_impact_correlations.loc[2, "Spearman"]), close(float(missing_impact_correlations.loc[2,
↳ "Spearman"]), 0.9858185341408066))
add_check("74. DISTANCE and AIR_TIME imputed Spearman", 0.9763241139194703,
↳ float(missing_impact_correlations.loc[3, "Spearman"]), close(float(missing_impact_correlations.loc[3,
↳ "Spearman"]), 0.9763241139194703))
add_check("75. Primary clustering silhouette", 0.5582190133558999, primary_silhouette, close(primary_silhouette,
↳ 0.5582190133558999))
add_check("76. Temporary imputed clustering silhouette", 0.5577294625652738, imputed_silhouette,
↳ close(imputed_silhouette, 0.5577294625652738))
imputed_sizes = sorted(imputed_profile["N"].tolist())
add_check("77. Temporary imputed cluster-size multiset", [15, 297, 835, 3202], imputed_sizes, imputed_sizes ==
↳ [15, 297, 835, 3202])
for idx, path in enumerate(a3_figures, start=78):
    add_check(f"{idx}. {path} exists", True, os.path.exists(path), os.path.exists(path))
add_check("85. All 11 expected CSV outputs exist", True, all(os.path.exists(p) for p in a3_outputs),
↳ all(os.path.exists(p) for p in a3_outputs))
add_check("86. df_raw equals a fresh read of 25740123.csv", True, df_raw.equals(df_raw_fresh_a3),
↳ df_raw.equals(df_raw_fresh_a3))
add_check("87. Raw row count unchanged", 4349, len(df_raw), len(df_raw) == 4349)
add_check("88. Raw column count unchanged", 25, df_raw.shape[1], df_raw.shape[1] == 25)
add_check("89. No permanent imputation", True, True, df_raw.equals(df_raw_fresh_a3))
add_check("90. No raw values changed", True, True, df_raw.equals(df_raw_fresh_a3))
add_check("91. No pre-existing notebook cell source through A2 was changed", True, source_ok, source_ok)
add_check("92. A3 headings exist exactly once each", True, all(v == 1 for v in heading_counts.values()), all(v ==
↳ 1 for v in heading_counts.values()))
add_check("93. No B1 analysis was created", True, no_b1_heading, no_b1_heading)

a3_check_table = pd.DataFrame(a3_checks)
print("A3 checks passed:", int(a3_check_table["Pass"].sum()), "of", len(a3_check_table))
assert a3_check_table["Pass"].all(), a3_check_table.loc[~a3_check_table["Pass"]]
a3_check_table

```

A3.5 Revised Correlation Figures and Supporting Tables

```

def save_correlation_matrix(matrix, method_name, file_name):
    """Draw one correlation matrix with the colour bar outside the grid."""
    with plt.rc_context({"font.size": 11}):
        figure, axis = plt.subplots(figsize=(10, 8.6))
        image = axis.imshow(matrix.to_numpy(), cmap="RdBu_r", vmin=-1, vmax=1)
        labels = list(matrix.columns)
        axis.set_xticks(range(len(labels)))
        axis.set_yticks(range(len(labels)))
        axis.set_xticklabels(labels, rotation=45, ha="right", fontsize=10)
        axis.set_yticklabels(labels, fontsize=10)
        for i in range(len(labels)):
            for j in range(len(labels)):
                value = matrix.iat[i, j]
                shade = "white" if abs(value) > 0.6 else "black"
                axis.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=10, color=shade)
        axis.set_title(f"{method_name} correlation between the numeric attributes", fontsize=14, pad=12)
        bar = figure.colorbar(image, ax=axis, fraction=0.046, pad=0.03)
        bar.set_label("Correlation coefficient", fontsize=11)
        figure.tight_layout()
        path = f"figures/A3_{method_name.lower()}_matrix.png"
        figure.savefig(path, dpi=200)
        plt.show()
        return path

pearson_matrix = pearson_corr
spearman_matrix = spearman_corr

pearson_matrix_path = save_correlation_matrix(
    pearson_matrix, "Pearson", "A3_pearson_matrix.png"
)
spearman_matrix_path = save_correlation_matrix(
    spearman_matrix, "Spearman", "A3_spearman_matrix.png"
)
print(pearson_matrix_path)
print(spearman_matrix_path)
print("Old combined figure still present:", os.path.exists("figures/A3_correlation_matrices.png"))

outlier_rate_rows = []
for airline_name, group in df_raw.groupby("AIRLINE_NAME", sort=True):
    recorded = group["ARRIVAL_DELAY"].dropna()
    outlier_count = int(((recorded < lower_fence) | (recorded > upper_fence)).sum())
    outlier_rate_rows.append({
        "Airline": airline_name,
        "Recorded arrival delays": int(len(recorded)),
        "Outliers": outlier_count,
        "Rate percent": round(100 * outlier_count / len(recorded), 2),
        "Median arrival delay": float(recorded.median()),
    })

outlier_rate_by_airline = pd.DataFrame(outlier_rate_rows).sort_values(
    "Rate percent", ascending=True
).reset_index(drop=True)
outlier_rate_by_airline.to_csv("outputs/A3_outlier_rate_by_airline.csv", index=False)
outlier_rate_by_airline

```

```

delay_pair_complete = df_raw[["DEPARTURE_DELAY", "ARRIVAL_DELAY"]].dropna()
arrival_below = int(
    (delay_pair_complete["ARRIVAL_DELAY"] < delay_pair_complete["DEPARTURE_DELAY"]).sum()
)
arrival_equal = int(
    (delay_pair_complete["ARRIVAL_DELAY"] == delay_pair_complete["DEPARTURE_DELAY"]).sum()
)

delay_pair_recovery = pd.DataFrame([
    "Pairs": int(len(delay_pair_complete)),
    "Arrival delay below departure delay": arrival_below,
    "Share percent": round(100 * arrival_below / len(delay_pair_complete), 2),
    "Arrival delay equal to departure delay": arrival_equal,
])
delay_pair_recovery.to_csv("outputs/A3_delay_pair_recovery.csv", index=False)
delay_pair_recovery
from PIL import Image

expected_outlier_rates = {
    "Delta Air Lines Inc.": (832, 49, 5.89, -7.0),
    "Southwest Airlines Co.": (1179, 106, 8.99, -4.0),
    "American Airlines Inc.": (720, 80, 11.11, -4.0),
    "Skywest Airlines Inc.": (558, 64, 11.47, -4.0),
    "Atlantic Southeast Airlines": (506, 65, 12.85, -3.0),
    "United Air Lines Inc.": (470, 63, 13.40, -6.0),
}

a35_checks = []

def add_a35(name, expected, actual, passed):
    a35_checks.append({
        "Check": name,
        "Expected": str(expected),
        "Actual": str(actual),
        "Pass": bool(passed),
    })

rate_lookup = outlier_rate_by_airline.set_index("Airline")
for airline, (rec, outs, rate, med) in expected_outlier_rates.items():
    row = rate_lookup.loc[airline]
    add_a35(
        f"{airline} outlier rate",
        (rec, outs, rate, med),
        (
            int(row["Recorded arrival delays"]),
            int(row["Outliers"]),
            float(row["Rate percent"]),
            float(row["Median arrival delay"]),
        ),
        int(row["Recorded arrival delays"]) == rec
        and int(row["Outliers"]) == outs
        and float(row["Rate percent"]) == rate
        and float(row["Median arrival delay"]) == med,
    )

add_a35(
    "Outlier counts sum to 427",
    427,
    int(outlier_rate_by_airline["Outliers"].sum()),
    int(outlier_rate_by_airline["Outliers"].sum()) == 427,
)
add_a35(
    "Recorded counts sum to 4265",
    4265,
    int(outlier_rate_by_airline["Recorded arrival delays"].sum()),
    int(outlier_rate_by_airline["Recorded arrival delays"].sum()) == 4265,
)
add_a35(
    "Delay recovery pairs",
    (4265, 3127, 73.32, 115),
    (
        int(delay_pair_recovery.loc[0, "Pairs"]),
        int(delay_pair_recovery.loc[0, "Arrival delay below departure delay"]),
        float(delay_pair_recovery.loc[0, "Share percent"]),
        int(delay_pair_recovery.loc[0, "Arrival delay equal to departure delay"]),
    ),
    int(delay_pair_recovery.loc[0, "Pairs"]) == 4265
    and int(delay_pair_recovery.loc[0, "Arrival delay below departure delay"]) == 3127
    and float(delay_pair_recovery.loc[0, "Share percent"]) == 73.32
    and int(delay_pair_recovery.loc[0, "Arrival delay equal to departure delay"]) == 115,
)

pearson_png = "figures/A3_pearson_matrix.png"
spearman_png = "figures/A3_spearman_matrix.png"
add_a35("Pearson matrix figure exists", True, os.path.exists(pearson_png), os.path.exists(pearson_png))
add_a35("Spearman matrix figure exists", True, os.path.exists(spearman_png), os.path.exists(spearman_png))
add_a35(
    "Old combined correlation figure kept",
    True,
    os.path.exists("figures/A3_correlation_matrices.png"),
    os.path.exists("figures/A3_correlation_matrices.png"),
)

```

```

)

saved_pearson = pd.read_csv("outputs/A3_pearson_correlation.csv", index_col=0)
saved_spearman = pd.read_csv("outputs/A3_spearman_correlation.csv", index_col=0)
add_a35(
    "Pearson figure matrix matches saved CSV",
    True,
    bool(np.allclose(pearson_matrix.to_numpy(), saved_pearson.to_numpy())),
    np.allclose(pearson_matrix.to_numpy(), saved_pearson.to_numpy()),
)
add_a35(
    "Spearman figure matrix matches saved CSV",
    True,
    bool(np.allclose(spearman_matrix.to_numpy(), saved_spearman.to_numpy())),
    np.allclose(spearman_matrix.to_numpy(), saved_spearman.to_numpy()),
)

add_a35(
    "Outlier table still has 427 rows",
    427,
    len(arrival_delay_outliers),
    len(arrival_delay_outliers) == 427,
)
expected_source_rows = set((outlier_index + 2).tolist())
actual_source_rows = set(arrival_delay_outliers["Source Row"].tolist())
add_a35(
    "Outlier source rows unchanged",
    True,
    expected_source_rows == actual_source_rows,
    expected_source_rows == actual_source_rows,
)
sort_ok = (
    arrival_delay_outliers["ARRIVAL_DELAY"].is_monotonic_decreasing
    or arrival_delay_outliers["ARRIVAL_DELAY"].eq(
        arrival_delay_outliers["ARRIVAL_DELAY"].sort_values(ascending=False).values
    ).all()
)
# Strict deterministic order check
resort_check = arrival_delay_outliers.sort_values(
    ["ARRIVAL_DELAY", "Source Row"], ascending=[False, True]
).reset_index(drop=True)
current_order = arrival_delay_outliers.reset_index(drop=True)
add_a35(
    "Outlier table sorted by ARRIVAL_DELAY then Source Row",
    True,
    bool(resort_check.equals(current_order)),
    resort_check.equals(current_order),
)

df_raw_fresh_a35 = pd.read_csv(RAW_CSV)
with open(RAW_CSV, "rb") as raw_file_a35:
    hash_a35 = hashlib.sha256(raw_file_a35.read()).hexdigest()
add_a35("df_raw equals fresh CSV read", True, df_raw.equals(df_raw_fresh_a35), df_raw.equals(df_raw_fresh_a35))
add_a35(
    "CSV SHA-256 unchanged",
    "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de",
    hash_a35,
    hash_a35 == "676681651e11de53308116df0fc5b2ef72c5b967ec779d0159f747edd10499de",
)

# PNG files are readable
for path in [pearson_png, spearman_png]:
    with Image.open(path) as image:
        add_a35(f"{path} readable", True, image.size[0] > 0, image.size[0] > 0)

a35_check_table = pd.DataFrame(a35_checks)
print("A3.5 checks passed:", int(a35_check_table["Pass"].sum()), "of", len(a35_check_table))
assert a35_check_table["Pass"].all(), a35_check_table.loc[~a35_check_table["Pass"]]
a35_check_table

B1.1 Choice of the Number of Bins
import inspect
import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Font
from openpyxl.utils import get_column_letter
from sklearn.preprocessing import (
    KBinsDiscretizer,
    MinMaxScaler,
    OneHotEncoder,
    StandardScaler,
)

os.makedirs("figures", exist_ok=True)
os.makedirs("outputs", exist_ok=True)

assert df_raw.shape == (4349, 25)

```

```

assert int(df_raw["ELAPSED_TIME"].isna().sum()) == 84
assert int(df_raw["ELAPSED_TIME"].notna().sum()) == 4265
assert int(df_raw["ARRIVAL_DELAY"].isna().sum()) == 84
assert int(df_raw["ARRIVAL_DELAY"].notna().sum()) == 4265
assert int(df_raw["DISTANCE"].isna().sum()) == 0
assert int(df_raw["AIRLINE_NAME"].isna().sum()) == 0
assert int(df_raw["AIRLINE_NAME"].nunique()) == 6
print("Section B source checks passed.")

elapsed_valid = df_raw["ELAPSED_TIME"].dropna()
n_elapsed = int(elapsed_valid.shape[0])
elapsed_min = float(elapsed_valid.min())
elapsed_max = float(elapsed_valid.max())
elapsed_range = elapsed_max - elapsed_min
elapsed_std = float(elapsed_valid.std(ddof=1))
elapsed_q1 = float(elapsed_valid.quantile(0.25))
elapsed_q3 = float(elapsed_valid.quantile(0.75))
elapsed_iqr = elapsed_q3 - elapsed_q1

print("n =", n_elapsed)
print("minimum =", elapsed_min)
print("maximum =", elapsed_max)
print("range =", elapsed_range)
print("sample SD =", elapsed_std)
print("Q1 =", elapsed_q1)
print("Q3 =", elapsed_q3)
print("IQR =", elapsed_iqr)
cube_root_n = n_elapsed ** (1 / 3)

raw_square_root = float(np.sqrt(n_elapsed))
raw_sturges = float(1 + 3.3 * np.log10(n_elapsed))
raw_rice = float(2 * cube_root_n)
raw_scott = float(elapsed_range / (3.5 * elapsed_std / cube_root_n))
raw_fd = float(elapsed_range / (2 * elapsed_iqr / cube_root_n))

raw_results = {
    "Square-root": raw_square_root,
    "Sturges": raw_sturges,
    "Rice": raw_rice,
    "Scott": raw_scott,
    "Freedman-Diaconis": raw_fd,
}

integer_bins = {name: int(np.ceil(value)) for name, value in raw_results.items()}

for name, value in raw_results.items():
    print(f"{name}: raw = {value:.10f}, integer bins = {integer_bins[name]}")

def equal_width_empty_count(values, n_bins):
    binner = KBinsDiscretizer(
        n_bins=n_bins, encode="ordinal", strategy="uniform", subsample=None
    )
    codes = binner.fit_transform(values.to_numpy().reshape(-1, 1)).ravel().astype(int)
    counts = np.bincount(codes, minlength=n_bins)
    return int((counts == 0).sum()), counts

trial_bin_counts = [10, 13, 15, 20, 33, 38, 58, 66]
empty_by_bins = {}
for n_bins in trial_bin_counts:
    empty_count, _ = equal_width_empty_count(elapsed_valid, n_bins)
    empty_by_bins[n_bins] = empty_count
    print(f"{n_bins} bins -> {empty_count} empty")

interpretations = {
    "Square-root": "Too many bins for this sample size; comparison only.",
    "Sturges": "13 bins; no empty equal-width bins.",
    "Rice": "33 bins; several empty equal-width bins.",
    "Scott": "38 bins; several empty equal-width bins.",
    "Freedman-Diaconis": "58 bins; many empty equal-width bins.",
}

rule_rows = []
for name, raw_value in raw_results.items():
    n_bins = integer_bins[name]
    empty_count = empty_by_bins[n_bins]
    rule_rows.append({
        "Rule": name,
        "Raw result": round(raw_value, 10),
        "Integer bins": n_bins,
        "Equal-width empty-bin count": empty_count,
        "Short interpretation": interpretations[name],
    })

for n_bins in trial_bin_counts:
    if n_bins not in integer_bins.values():
        rule_rows.append({
            "Rule": f"Trial ({n_bins})",
            "Raw result": np.nan,
            "Integer bins": n_bins,
            "Equal-width empty-bin count": empty_by_bins[n_bins],
            "Short interpretation": "Nearby trial value for comparison.",
        })

```

```

b1_rule_comparison = pd.DataFrame(rule_rows)
b1_rule_comparison.to_csv("outputs/B1_bin_rule_comparison.csv", index=False)
print("Saved outputs/B1_bin_rule_comparison.csv")
b1_rule_comparison
plot_candidates = [10, 13, 20, 33, 38, 58]
with plt.rc_context({"font.size": 10}):
    figure, axes = plt.subplots(2, 3, figsize=(12, 7), sharey=False)
    for axis, n_bins in zip(axes.ravel(), plot_candidates):
        empty_count, counts = equal_width_empty_count(elapsed_valid, n_bins)
        axis.bar(np.arange(1, n_bins + 1), counts, color="steelblue", edgecolor="black", linewidth=0.3)
        axis.set_title(f"{n_bins} bins ({empty_count} empty)")
        axis.set_xlabel("Bin")
        axis.set_ylabel("Count")
    figure.suptitle("Equal-width ELAPSED_TIME bin-count comparison")
    figure.tight_layout()
    figure.savefig("figures/B1_bin_count_comparison.png", dpi=200, bbox_inches="tight")
plt.show()

```

```

SELECTED_N_BINS = 13
print("Approved bin count selected:", SELECTED_N_BINS)

```

B1.2 Equal-Width Binning and Smoothing

```

kbins_kwargs = dict(n_bins=SELECTED_N_BINS, encode="ordinal", subsample=None)
if "quantile_method" in inspect.signature(KBinsDiscretizer.__init__).parameters:
    print("sklearn version supports quantile_method and subsample.")

elapsed_matrix = elapsed_valid.to_numpy().reshape(-1, 1)
width_binner = KBinsDiscretizer(strategy="uniform", **kbins_kwargs)
width_codes0 = width_binner.fit_transform(elapsed_matrix).ravel().astype(int)
width_edges = width_binner.bin_edges[0]
width_counts = np.bincount(width_codes0, minlength=SELECTED_N_BINS)
width_means = np.array([
    float(elapsed_valid.to_numpy()[width_codes0 == i].mean())
    for i in range(SELECTED_N_BINS)
])

width_summary = pd.DataFrame({
    "Bin": [f"Bin {i}" for i in range(1, SELECTED_N_BINS + 1)],
    "Left edge": width_edges[:-1],
    "Right edge": width_edges[1:],
    "Actual minimum": [
        float(elapsed_valid.to_numpy()[width_codes0 == i].min()) for i in range(SELECTED_N_BINS)
    ],
    "Actual maximum": [
        float(elapsed_valid.to_numpy()[width_codes0 == i].max()) for i in range(SELECTED_N_BINS)
    ],
    "Count": width_counts,
    "Bin mean": width_means,
})
width_summary.to_csv("outputs/B1_equal_width_summary.csv", index=False)
print("Equal-width edges:", width_edges.tolist())
print("Equal-width counts:", width_counts.tolist(), "sum =", int(width_counts.sum()))
print("Saved outputs/B1_equal_width_summary.csv")
width_summary

```

B1.3 Equal-Depth Binning and Smoothing

```

depth_kwargs = dict(kbins_kwargs)
if "quantile_method" in inspect.signature(KBinsDiscretizer.__init__).parameters:
    depth_kwargs["quantile_method"] = "linear"

depth_binner = KBinsDiscretizer(strategy="quantile", **depth_kwargs)
depth_codes0 = depth_binner.fit_transform(elapsed_matrix).ravel().astype(int)
depth_edges = depth_binner.bin_edges[0]
depth_counts = np.bincount(depth_codes0, minlength=SELECTED_N_BINS)
depth_means = np.array([
    float(elapsed_valid.to_numpy()[depth_codes0 == i].mean())
    for i in range(SELECTED_N_BINS)
])

depth_summary = pd.DataFrame({
    "Bin": [f"Bin {i}" for i in range(1, SELECTED_N_BINS + 1)],
    "Left edge": depth_edges[:-1],
    "Right edge": depth_edges[1:],
    "Actual minimum": [
        float(elapsed_valid.to_numpy()[depth_codes0 == i].min()) for i in range(SELECTED_N_BINS)
    ],
    "Actual maximum": [
        float(elapsed_valid.to_numpy()[depth_codes0 == i].max()) for i in range(SELECTED_N_BINS)
    ],
    "Count": depth_counts,
    "Bin mean": depth_means,
})
depth_summary.to_csv("outputs/B1_equal_depth_summary.csv", index=False)
print("Equal-depth edges:", depth_edges.tolist())
print("Equal-depth counts:", depth_counts.tolist(), "sum =", int(depth_counts.sum()))
print("Saved outputs/B1_equal_depth_summary.csv")
depth_summary
with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)
    axes[0].bar(np.arange(1, 14), width_counts, color="steelblue", edgecolor="black", linewidth=0.4)
    axes[0].set_title("Equal-width (13 bins)")

```

```

axes[0].set_xlabel("Bin")
axes[0].set_ylabel("Count")
axes[1].bar(np.arange(1, 14), depth_counts, color="darkorange", edgecolor="black", linewidth=0.4)
axes[1].set_title("Equal-depth (13 bins)")
axes[1].set_xlabel("Bin")
figure.suptitle("Selected ELAPSED_TIME binning comparison")
figure.tight_layout()
figure.savefig("figures/B1_selected_binning.png", dpi=200, bbox_inches="tight")
plt.show()

b1_output = pd.DataFrame({
    "ELAPSED_TIME": df_raw["ELAPSED_TIME"],
    "Equal-width bin": pd.Series(np.nan, index=df_raw.index, dtype="float"),
    "Equal-width smoothed mean": pd.Series(np.nan, index=df_raw.index, dtype="float"),
    "Equal-depth bin": pd.Series(np.nan, index=df_raw.index, dtype="float"),
    "Equal-depth smoothed mean": pd.Series(np.nan, index=df_raw.index, dtype="float"),
})

valid_elapsed_index = elapsed_valid.index
b1_output.loc[valid_elapsed_index, "Equal-width bin"] = width_codes0 + 1
b1_output.loc[valid_elapsed_index, "Equal-width smoothed mean"] = width_means[width_codes0]
b1_output.loc[valid_elapsed_index, "Equal-depth bin"] = depth_codes0 + 1
b1_output.loc[valid_elapsed_index, "Equal-depth smoothed mean"] = depth_means[depth_codes0]

print(b1_output.head(10))
print("Missing counts in derived B1 columns:")
print(b1_output.isna().sum())

B2.1 Min-Max Normalisation
distance = df_raw["DISTANCE"].astype(float)
distance_n = int(distance.shape[0])
distance_min = float(distance.min())
distance_max = float(distance.max())
distance_mean = float(distance.mean())
distance_sd_pop = float(distance.std(ddof=0))
distance_sd_sample = float(distance.std(ddof=1))

minmax_scaler = MinMaxScaler(feature_range=(0, 1))
distance_minmax = minmax_scaler.fit_transform(distance.to_numpy().reshape(-1, 1)).ravel()

print("n =", distance_n)
print("minimum =", distance_min)
print("maximum =", distance_max)
print("mean =", distance_mean)
print("population SD (ddof=0) =", distance_sd_pop)
print("sample SD (ddof=1) =", distance_sd_sample)
print("min-max minimum =", float(distance_minmax.min()))
print("min-max maximum =", float(distance_minmax.max()))

B2.2 Z-Score Normalisation
zscore_scaler = StandardScaler()
distance_zscore = zscore_scaler.fit_transform(distance.to_numpy().reshape(-1, 1)).ravel()

print("z-score mean =", float(distance_zscore.mean()))
print("z-score population SD =", float(distance_zscore.std(ddof=0)))
print("z-score minimum =", float(distance_zscore.min()))
print("z-score maximum =", float(distance_zscore.max()))

b2_output = pd.DataFrame({
    "DISTANCE": distance,
    "Min-max normalised DISTANCE": distance_minmax,
    "Z-score normalised DISTANCE": distance_zscore,
})
print(b2_output.head())

b2_summary = pd.DataFrame({
    "source N": distance_n,
    "minimum": distance_min,
    "maximum": distance_max,
    "mean": distance_mean,
    "population SD": distance_sd_pop,
    "sample SD": distance_sd_sample,
    "min-max minimum": float(distance_minmax.min()),
    "min-max maximum": float(distance_minmax.max()),
    "z-score mean": float(distance_zscore.mean()),
    "z-score population SD": float(distance_zscore.std(ddof=0)),
    "z-score minimum": float(distance_zscore.min()),
    "z-score maximum": float(distance_zscore.max()),
})
b2_summary.to_csv("outputs/B2_normalisation_summary.csv", index=False)
print("Saved outputs/B2_normalisation_summary.csv")
b2_summary

hist_bins = 30
with plt.rc_context({"font.size": 11}):
    figure, axes = plt.subplots(1, 3, figsize=(12, 3.8))
    axes[0].hist(distance, bins=hist_bins, color="steelblue", edgecolor="black", linewidth=0.3)
    axes[0].set_title("Original DISTANCE")
    axes[0].set_xlabel("Miles")
    axes[0].set_ylabel("Count")
    axes[1].hist(distance_minmax, bins=hist_bins, color="seagreen", edgecolor="black", linewidth=0.3)
    axes[1].set_title("Min-max DISTANCE")
    axes[1].set_xlabel("Scaled value")
    axes[2].hist(distance_zscore, bins=hist_bins, color="darkorange", edgecolor="black", linewidth=0.3)

```

```

axes[2].set_title("Z-score DISTANCE")
axes[2].set_xlabel("Z-score")
figure.suptitle("DISTANCE normalisation comparison")
figure.tight_layout()
figure.savefig("figures/B2_distance_normalisation.png", dpi=200, bbox_inches="tight")
plt.show()

```

B3. Discretisation

```

arrival_delay_category = pd.cut(
    df_raw["ARRIVAL_DELAY"],
    bins=[-np.inf, 15, 60, np.inf],
    right=False,
    labels=["Small Delay", "Medium Delay", "Long Delay"],
)

b3_output = pd.DataFrame({
    "ARRIVAL_DELAY": df_raw["ARRIVAL_DELAY"],
    "ARRIVAL_DELAY category": arrival_delay_category.astype(object),
})
# Keep Excel-friendly blanks for missing categories.
b3_output.loc[b3_output["ARRIVAL_DELAY"].isna(), "ARRIVAL_DELAY category"] = pd.NA

category_order = ["Small Delay", "Medium Delay", "Long Delay", "Missing"]
freq_small = int((arrival_delay_category == "Small Delay").sum())
freq_medium = int((arrival_delay_category == "Medium Delay").sum())
freq_long = int((arrival_delay_category == "Long Delay").sum())
freq_missing = int(df_raw["ARRIVAL_DELAY"].isna().sum())
recorded_n = 4265

b3_frequency = pd.DataFrame([
    {
        "Category": "Small Delay",
        "Frequency": freq_small,
        "Percentage of all 4349 rows": round(100 * freq_small / 4349, 2),
        "Percentage of 4265 recorded values": round(100 * freq_small / recorded_n, 2),
    },
    {
        "Category": "Medium Delay",
        "Frequency": freq_medium,
        "Percentage of all 4349 rows": round(100 * freq_medium / 4349, 2),
        "Percentage of 4265 recorded values": round(100 * freq_medium / recorded_n, 2),
    },
    {
        "Category": "Long Delay",
        "Frequency": freq_long,
        "Percentage of all 4349 rows": round(100 * freq_long / 4349, 2),
        "Percentage of 4265 recorded values": round(100 * freq_long / recorded_n, 2),
    },
    {
        "Category": "Missing",
        "Frequency": freq_missing,
        "Percentage of all 4349 rows": round(100 * freq_missing / 4349, 2),
        "Percentage of 4265 recorded values": np.nan,
    },
])
b3_frequency.to_csv("outputs/B3_arrival_delay_frequency.csv", index=False)

small_mask = arrival_delay_category == "Small Delay"
small_values = df_raw.loc[small_mask, "ARRIVAL_DELAY"]
print("Small Delay composition:")
print("negative =", int((small_values < 0).sum()))
print("zero =", int((small_values == 0).sum()))
print("1 to 14 =", int(((small_values >= 1) & (small_values <= 14)).sum()))
print(b3_frequency)
with plt.rc_context({"font.size": 11}):
    figure, axis = plt.subplots(figsize=(8, 4.5))
    categories = ["Small Delay", "Medium Delay", "Long Delay"]
    frequencies = [freq_small, freq_medium, freq_long]
    bars = axis.bar(categories, frequencies, color=["#4c78a8", "#f58518", "#e45756"], edgecolor="black")
    axis.set_ylabel("Frequency")
    axis.set_title("ARRIVAL_DELAY discretisation")
    for bar, value in zip(bars, frequencies):
        axis.text(bar.get_x() + bar.get_width() / 2, value + 40, str(value), ha="center", va="bottom")
    axis.text(
        0.98, 0.95,
        f"Missing ARRIVAL_DELAY values: {freq_missing}",
        transform=axis.transAxes, ha="right", va="top",
        bbox=dict(boxstyle="round", facecolor="white", edgecolor="grey"),
    )
    figure.tight_layout()
    figure.savefig("figures/B3_arrival_delay_categories.png", dpi=200, bbox_inches="tight")
    plt.show()

```

B4. Binarisation

```

airline_encoder = OneHotEncoder(
    sparse_output=False,
    dtype=int,
    handle_unknown="error",
)
airline_matrix = airline_encoder.fit_transform(df_raw[["AIRLINE_NAME"]])
airline_columns = list(airline_encoder.get_feature_names_out(["AIRLINE_NAME"]))

```

```

b4_output = pd.DataFrame(airline_matrix, columns=airline_columns, index=df_raw.index)
b4_output.insert(0, "AIRLINE_NAME", df_raw["AIRLINE_NAME"].to_numpy())

column_sums = b4_output[airline_columns].sum().astype(int)
row_sums = b4_output[airline_columns].sum(axis=1)

b4_check = pd.DataFrame({
    "Airline": [name.replace("AIRLINE_NAME_", "") for name in airline_columns],
    "Encoded column": airline_columns,
    "Column sum": column_sums.to_numpy(),
})
b4_check.to_csv("outputs/B4_airline_encoding_check.csv", index=False)

print("One-hot columns:", airline_columns)
print(b4_check)
print("Unique row sums:", sorted(row_sums.unique().tolist()))
print(b4_output.head())
excel_path = "ida_a2_25740123.xlsx"

original_data_sheet = df_raw.copy()
b1_sheet = b1_output.copy()
b2_sheet = b2_output.copy()
b3_sheet = b3_output.copy()
b4_sheet = b4_output.copy()

with pd.ExcelWriter(excel_path, engine="openpyxl") as writer:
    original_data_sheet.to_excel(writer, sheet_name="Original Data", index=False)
    b1_sheet.to_excel(writer, sheet_name="B1 Binning", index=False)
    b2_sheet.to_excel(writer, sheet_name="B2 Normalisation", index=False)
    b3_sheet.to_excel(writer, sheet_name="B3 Discretisation", index=False)
    b4_sheet.to_excel(writer, sheet_name="B4 Binarisation", index=False)

workbook = load_workbook(excel_path)
expected_sheets = [
    "Original Data",
    "B1 Binning",
    "B2 Normalisation",
    "B3 Discretisation",
    "B4 Binarisation",
]
assert workbook.sheetnames == expected_sheets

for sheet_name in expected_sheets:
    worksheet = workbook[sheet_name]
    worksheet.freeze_panes = "A2"
    for cell in worksheet[1]:
        cell.font = Font(bold=True)
    for column_cells in worksheet.columns:
        max_length = 0
        column_letter = get_column_letter(column_cells[0].column)
        for cell in column_cells[:50]:
            if cell.value is not None:
                max_length = max(max_length, len(str(cell.value)))
        worksheet.column_dimensions[column_letter].width = min(max(12, max_length + 2), 40)

workbook.save(excel_path)
print("Saved", excel_path)
print("Sheets:", workbook.sheetnames)

def check(name, condition):
    status = "PASS" if condition else "FAIL"
    print(f"{status}: {name}")
    return condition

all_pass = True

print("GLOBAL")
all_pass &= check("dataset shape 4349 x 25", df_raw.shape == (4349, 25))
all_pass &= check(
    "original row order preserved",
    df_raw["DISTANCE"].iloc[0] == 423 and df_raw["DISTANCE"].iloc[-1] == b2_output["DISTANCE"].iloc[-1],
)

print("B1")
all_pass &= check("4265 valid ELAPSED_TIME", n_elapsed == 4265)
all_pass &= check("84 missing ELAPSED_TIME", int(df_raw["ELAPSED_TIME"].isna().sum()) == 84)
all_pass &= check("Square-root raw", abs(raw_square_root - 65.3069674690) < 1e-8)
all_pass &= check("Sturges raw", abs(raw_sturges - 12.9787328172) < 1e-8)
all_pass &= check("Rice raw", abs(raw_rice - 32.4341863416) < 1e-8)
all_pass &= check("Scott raw", abs(raw_scott - 37.5162040901) < 1e-8)
all_pass &= check("Freedman-Diaconis raw", abs(raw_fd - 57.6497885279) < 1e-8)
all_pass &= check("13 bins selected", SELECTED_N_BINS == 13)
all_pass &= check("equal-width counts sum 4265", int(width_counts.sum()) == 4265)
all_pass &= check("equal-depth counts sum 4265", int(depth_counts.sum()) == 4265)
all_pass &= check("equal-width has 13 bins", len(width_counts) == 13)
all_pass &= check("equal-depth has 13 bins", len(depth_counts) == 13)
all_pass &= check(
    "B1 derived columns have exactly 84 missing values",
    all(int(b1_output[col].isna().sum()) == 84 for col in b1_output.columns if col != "ELAPSED_TIME"),
)
all_pass &= check(

```

```

    "expected first-row checks pass",
    b1_output.loc[0, "ELAPSED_TIME"] == 72
    and b1_output.loc[0, "Equal-width bin"] == 2
    and abs(b1_output.loc[0, "Equal-width smoothed mean"] - 93.019204) < 1e-5
    and b1_output.loc[0, "Equal-depth bin"] == 3
    and abs(b1_output.loc[0, "Equal-depth smoothed mean"] - 75.733728) < 1e-5
    and b1_output.loc[1, "ELAPSED_TIME"] == 100
    and b1_output.loc[1, "Equal-width bin"] == 2
    and b1_output.loc[1, "Equal-depth bin"] == 6
    and b1_output.loc[2, "ELAPSED_TIME"] == 247
    and b1_output.loc[2, "Equal-width bin"] == 5
    and b1_output.loc[2, "Equal-depth bin"] == 12
    and b1_output.loc[3, "ELAPSED_TIME"] == 102
    and b1_output.loc[4, "ELAPSED_TIME"] == 125
    and b1_output.loc[4, "Equal-width bin"] == 3
    and b1_output.loc[4, "Equal-depth bin"] == 8,
)

print("B2")
all_pass &= check("4349 DISTANCE values", distance_n == 4349)
all_pass &= check("min-max range [0,1]", abs(distance_minmax.min()) < 1e-12 and abs(distance_minmax.max() - 1) <
↳ 1e-12)
all_pass &= check("z mean approximately 0", abs(distance_zscore.mean()) < 1e-10)
all_pass &= check("z population SD approximately 1", abs(distance_zscore.std(ddof=0) - 1) < 1e-10)
all_pass &= check(
    "expected first-row checks pass",
    abs(distance_minmax[0] - 0.072416599) < 1e-8
    and abs(distance_zscore[0] + 0.662362) < 1e-5
    and abs(distance_minmax[1] - 0.099877949) < 1e-8
    and abs(distance_zscore[1] + 0.431792) < 1e-5,
)

print("B3")
all_pass &= check("Small 3438", freq_small == 3438)
all_pass &= check("Medium 557", freq_medium == 557)
all_pass &= check("Long 270", freq_long == 270)
all_pass &= check("Missing 84", freq_missing == 84)
all_pass &= check("total 4349", freq_small + freq_medium + freq_long + freq_missing == 4349)

print("B4")
all_pass &= check("six columns", len(airline_columns) == 6)
all_pass &= check("every value is 0 or 1", set(np.unique(airline_matrix)).issubset({0, 1}))
all_pass &= check("every row sum is 1", bool((row_sums == 1).all()))
all_pass &= check(
    "column sums equal expected",
    column_sums.tolist() == [733, 521, 839, 569, 1207, 480],
)

print("EXCEL")
wb_check = load_workbook(excel_path, read_only=True, data_only=True)
all_pass &= check("required five sheets", wb_check.sheetnames == expected_sheets)
excel_row_ok = True
for sheet_name in expected_sheets:
    ws = wb_check[sheet_name]
    # header + 4349 data rows
    excel_row_ok &= (ws.max_row == 4350)
all_pass &= check("correct row counts", excel_row_ok)
ws0 = wb_check["Original Data"]
header = [cell.value for cell in next(ws0.iter_rows(min_row=1, max_row=1))]
all_pass &= check("no unexpected index column", header[0] in df_raw.columns and "Unnamed" not in str(header[0]))
first_distance = None
last_distance = None
for i, row in enumerate(ws0.iter_rows(min_row=2, values_only=True), start=0):
    distance_idx = header.index("DISTANCE")
    if i == 0:
        first_distance = row[distance_idx]
    if i == 4348:
        last_distance = row[distance_idx]
all_pass &= check(
    "original row order preserved",
    first_distance == df_raw["DISTANCE"].iloc[0] and last_distance == df_raw["DISTANCE"].iloc[-1],
)
wb_check.close()

print("OVERALL:", "PASS" if all_pass else "FAIL")
if not all_pass:
    raise SystemExit("Section B validation failed.")

```